

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ ĐIỆN TỬ



NGUYỄN TẤN LỘC
TRẦN LƯU NGỌC THẠCH

**THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG NHÀ
THÔNG MINH ĐIỀU KHIỂN BẰNG GIỌNG
NÓI TRÊN NỀN TẢNG IOT VÀ AI**

Chuyên ngành: IOT VÀ TRÍ TUỆ NHÂN TẠO ỨNG DỤNG
Mã chuyên ngành: 7510304

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2025

THÔNG TIN CHUNG

Họ và tên sinh viên : Nguyễn Tấn LộcMSHV: 21118191.....
Lớp : DHIOT17CKhóa: K17.....
Email : nguyentanloc2204@gmail.com.....
Họ và tên sinh viên : Trần Lưu Ngọc Thạch.....MSHV: 21131741.....
Lớp : DHIOT17CKhóa: K17.....
Email : wib2405@gmail.com.....
Chuyên ngành : IOT và Trí tuệ nhân tạo ứng dụng Mã chuyên ngành: 7510304.....
SĐT : 0388488904 (Lộc) / 0862256676 (Thạch).....
Địa chỉ liên hệ : 579/104, Quang Trung, P. Thông Tây Hội, TP. HCM.....
Tên đề tài : THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG NHÀ THÔNG MINH
ĐIỀU KHIỂN BẰNG GIỌNG NÓI TRÊN NỀN TẢNG IOT VÀ AI...
Người hướng dẫn : ThS. Nguyễn Thị Hồng Hà.....
SĐT : 0909500941
Email : nguyenthihongha@iuh.edu.vn.....
Cơ quan công tác : Trường Đại học Công Nghiệp TP. HCM.....
Người hướng dẫn : ThS. Phạm Quang Trí.....
SĐT : 0913982569
Email : phamquangtri@iuh.edu.vn.....
Cơ quan công tác : Trường Đại học Công Nghiệp TP. HCM.....

Tp. Hồ Chí Minh, ngày tháng năm 2025

Người hướng dẫn
(Ký và ghi rõ họ tên)

Sinh viên
(Ký và ghi rõ họ tên)

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the entire width of the page. There are no margins, text, or other markings present.

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the entire width of the page. There are no margins, text, or other markings present.

MỤC LỤC

MỤC LỤC.....	1
DANH MỤC HÌNH ẢNH	5
DANH MỤC BẢNG BIỂU	7
DANH MỤC TỪ VIẾT TẮT.....	8
MỞ ĐẦU.....	9
CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI.....	13
1.1 Giới thiệu chương.....	13
1.2 Bối cảnh đề tài và xu hướng hiện nay	13
1.3 Tổng quan bài toán nhận dạng giọng nói (Keyword spotting).....	14
1.4 Các nghiên cứu liên quan trước đây	14
1.4.1 Nghiên cứu về CNN và DS-CNN cho nhận dạng từ khóa (KWS) trên thiết bị cấu hình thấp	14
1.4.2 Nghiên cứu về nhận dạng giọng nói tiếng Việt	15
1.5 Nhận xét và khoảng trống nghiên cứu.....	16
1.6 Kết luận chương	17
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	18
2.1 Giới thiệu chung	18
2.2 Xử lý tín hiệu tiếng nói trong miền số	18
2.2.1 Lấy mẫu và biểu diễn tín hiệu.....	18
2.2.2 Phân khung và cửa sổ hóa.....	19
2.2.3 Biến đổi Fourier ngắn hạn (STFT)	19
2.3 Trích xuất đặc trưng cho KWS: Mel Filterbank và Log-Mel	19
2.3.1 Mel scale và bộ lọc tam giác.....	19
2.3.2 Log – mel spectrogram	20
2.3.3 Liên hệ với MFCC	20
2.4 Chuẩn hóa đặc trưng.....	20
2.5 Cơ sở mô hình học sâu cho Keyword Spotting.....	20
2.5.1 Bài toán phân loại đa lớp	20

2.5.2	Kiến trúc CNN và biến thể tối ưu	20
2.5.3	Hàm mất mát và tối ưu.....	21
2.6	Hậu Xử Lý Quyết Định (Decision Logic) Cho Kws Chạy Thực Tế.....	21
2.7	Triển khai suy luận trên thiết bị nhúng (Tflite Micro)	22
2.7.1	Tổng quan TensorFlow Lite Micro.....	22
2.7.2	Lượng tử hóa INT8 (Quantization).....	22
2.7.3	Quản lý bộ nhớ: Tensor Arena.....	22
2.8	Cơ sở truyền thông MQTT và HTTPS/TLS/WEBSOCKET, cơ chế thời gian thực qua Firebase và cơ chế gửi thông báo bằng FCM.....	23
2.8.1	Giao thức truyền thông không dây MQTT	23
2.8.2	Khái niệm HTTP và HTTPS.....	23
2.8.3	Khái niệm WebSocket	23
2.8.4	Khái niệm Firebase Admin SDK và Firebase Client SDK trong kiến trúc hệ thống.....	24
2.8.5	Khái niệm Firebase Cloud Messaging (FCM).....	24
2.8.6	Cơ chế gửi và nhận thông báo FCM trong hệ thống.....	25
CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....		26
3.1	Giới thiệu Tổng quan về Hệ thống	26
3.2	Lựa chọn phương án thiết kế tổng thể.....	26
3.3	Kiến trúc Tổng quát của Hệ thống	27
3.4	Giao thức truyền tải dữ liệu trong hệ thống	28
3.5	Phân tích và thiết kế phần cứng hệ thống.....	28
3.5.1	Phân tích yêu cầu kỹ thuật	28
3.5.2	Lựa chọn và Mô tả linh kiện	30
3.5.3	Module Giao tiếp và Truyền thông.....	31
3.5.4	Các cơ cấu chấp hành và đầu ra (Output).....	32
3.5.5	Nguồn cấp	33
3.5.6	Nguồn cách ly dành cho mạch điều khiển Relay bằng B0505S 2W ..	35
3.5.7	Số lượng cảm biến và thiết bị chấp hành cho từng Node	36
3.5.8	Sơ đồ khối phần cứng cho Node (Hardware Block Diagram) tổng thể	37
3.5.9	Sơ đồ nguồn cấp cho Node tổng thể	39

3.5.10	Sơ đồ nguyên lý (Schematic Diagram) cho Node	39
3.5.11	Thiết kế Bố cục Mạch In (PCB Layout)	43
3.5.12	Hình ảnh Bố cục Mạch In Hoàn chỉnh	45
3.6	Phân tích và thiết kế thuật toán nhận diện giọng nói	46
3.6.1	Quá trình huấn luyện mô hình	46
3.6.2	Triển khai chi tiết theo sơ đồ khối	46
3.6.3	Quá trình đánh giá kết quả mô hình.....	49
3.6.4	Quá trình xử lý giọng nói dựa vào mô hình đã huấn luyện	54
3.7	Phân tích và thiết kế thuật toán điều phối IOT của hệ thống	60
3.7.1	Các nhóm chức năng trong thiết kế thuật toán điều phối cần có	60
3.7.2	Triển khai chi tiết từng nhóm – Nhóm 1: Hạ tầng giao tiếp MQTT...61	
3.7.3	Triển khai chi tiết từng nhóm – Nhóm 2: Cập nhật trạng thái & tình trạng online.....	64
3.7.4	Triển khai chi tiết từng nhóm – Nhóm 3 – Xử lý lệnh điều khiển thủ công (App + Voice).....	67
3.7.5	Triển khai chi tiết từng nhóm – Nhóm 4: Xử lý dữ liệu cảm biến thời gian thực & cảnh báo	71
3.7.6	Triển khai chi tiết từng nhóm – Nhóm 5 – Tự động hoá	77
3.7.7	Kết luận chung quá trình thiết kế thuật toán điều phối cho toàn bộ hệ thống IOT	84
3.8	Thiết kế ứng dụng.....	85
3.8.1	Màn hình đăng nhập.....	85
3.8.2	Màn hình Dashboard.....	85
3.8.3	Màn hình Chi tiết Node (Node Detail Screen)	86
3.8.4	Màn hình Lịch sử Thông báo (Notification History).....	87
3.8.5	Màn hình Cài đặt Hệ thống.....	88
3.8.6	Màn hình quản lý danh sách kịch bản tự động	89
3.8.7	Màn hình Cập nhật/ Tạo kịch bản tự động	89
3.8.8	Màn hình Chế độ hoạt động.....	90
CHƯƠNG 4	THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ.....	91
4.1	Thực nghiệm.....	91

4.1.1	Thiết kế mạch in PCB	91
4.1.2	Lắp đặt/ hàn linh kiện phần cứng các Node.....	91
4.1.3	Thiết kế mô hình tương ứng với các Node	92
4.1.4	Kiểm thử các kết quả tính toán cho mạch nguồn.....	94
4.1.5	Thử nghiệm nhận diện giọng nói và quyết định nhận chính xác	94
4.1.6	Kiểm thử chức năng theo dõi môi trường và điều khiển trạng thái các node	98
4.1.7	Kiểm thử chức năng tự động hóa.....	100
KẾT LUẬN VÀ KIẾN NGHỊ.....		102
TÀI LIỆU THAM KHẢO.....		104
PHỤ LỤC.....		107

DANH MỤC HÌNH ẢNH

Hình 3. 1 Sơ đồ Kiến trúc Tổng quát của Hệ thống và Giao thức truyền tải dữ liệu trong hệ thống	27
Hình 3. 2 Sơ đồ khối phần cứng cho Node tổng thể	38
Hình 3. 3 Sơ đồ nguồn cấp cho Node tổng thể	39
Hình 3. 4 Sơ đồ nguyên lý mạch nguồn giảm áp	39
Hình 3. 5 Sơ đồ nguyên lý mạch điều khiển Relay 5V	40
Hình 3. 6 Sơ đồ nguyên lý mạch chuyển đổi UART sang RS485	41
Hình 3. 7 Sơ đồ nguyên lý mạch nguồn cách ly cho Relay sử dụng B0505S 2W....	42
Hình 3. 8 Sơ đồ nguyên lý mạch giảm áp 5 V xuống 3.3 V	43
Hình 3. 9 Sơ đồ nguyên lý tổng hợp cho Node.....	43
Hình 3. 10 Sơ đồ mạch PCB lớp trên.....	45
Hình 3. 11 Sơ đồ mạch in PCB lớp dưới	45
Hình 3. 12 Sơ đồ khối xử lý tín hiệu giọng nói phục vụ huấn luyện mô hình.....	46
Hình 3. 13 Kết quả đánh giá mô hình	51
Hình 3. 14 Ma trận nhầm lẫn trên tập kiểm tra	52
Hình 3. 15 Ảnh 2D log-Mel spectrogram	53
Hình 3. 16 Lưu đồ thuật toán suy luận tìm xác suất nhận diện các nhãn	55
Hình 3. 17 Thuật toán phát hiện từ khóa kích hoạt.....	56
Hình 3. 18 Thuật toán phát hiện lệnh điều khiển giọng nói từ người dùng	58
Hình 3. 19 Lưu đồ khởi tạo và kết nối MQTT	61
Hình 3. 20 Lưu đồ subscribe topic MQTT	62
Hình 3. 21 Lưu đồ tiếp nhận bản tin và điều phối theo topic.....	62
Hình 3. 22 Lưu đồ publish lệnh qua MQTT	63
Hình 3. 23 Lưu đồ cập nhật online/offline của node	65
Hình 3. 24 Lưu đồ cập nhật trạng thái báo cáo (reported state) của actuator	65
Hình 3. 25 Lưu đồ cập nhật trạng thái của Gateway.....	66
Hình 3. 26 Lưu đồ xử lý lệnh giọng nói từ MQTT	68
Hình 3. 27 Lưu đồ lắng nghe và xử lý lệnh từ App	69
Hình 3. 28 Lưu đồ gửi lệnh điều khiển xuống node qua MQTT	69
Hình 3. 29 Lưu đồ thuật toán hàm đánh giá ngưỡng cảnh báo	72
Hình 3. 30 Lưu đồ thuật toán hàm cập nhật giá trị cảm biến lên Cloud Firebase	73
Hình 3. 31 Lưu đồ thuật toán hàm chống lặp cảnh báo	73
Hình 3. 32 Lưu đồ thuật toán hàm ghi lịch sử sự kiện thông báo	74
Hình 3. 33 Lưu đồ thuật toán hàm gửi thông báo đẩy FCM.....	74
Hình 3. 34 Lưu đồ giải thuật hàm cha: Xử lý dữ liệu cảm biến.....	75
Hình 3. 35 Lưu đồ giải thuật hàm đồng bộ kịch bản từ Cloud Firebase	78

Hình 3. 36 Lưu đồ giải thuật hàm kiểm soát quyền kích hoạt tự động hoá	78
Hình 3. 37 Lưu đồ giải thuật hàm kiểm soát quyền kích hoạt tự động hoá theo chế độ vận hành (Hình tiếp theo của Hình 3.36)	79
Hình 3. 38 Lưu đồ giải thuật hàm đánh giá điều kiện kích hoạt của một trigger	80
Hình 3. 39 Lưu đồ giải thuật kiểm tra một kịch bản (rule) có đủ điều kiện để thực hiện	81
Hình 3. 40 Lưu đồ giải thuật hàm thực thi hành động điều khiển khi rule được kích hoạt.....	82
Hình 3. 41 Lưu đồ thuật toán điều phối toàn bộ logic tự động hóa	83

Hình 4. 1 Mạch in PCB 3D	91
Hình 4. 2 Mạch PCB phần cứng cho Node thực tế.....	91
Hình 4. 3 Mô hình Node phòng khách.....	92
Hình 4. 4 Mô hình Node phòng bếp.....	93
Hình 4. 5 Mô hình Node phòng ngủ	93
Hình 4. 6 Kết đo đo điện áp đầu ra thực tế của mạch	94
Hình 4. 7 Nhận diện giọng nói và quyết định nhấn <i>bật_đèn_phòng_khách</i>	95
Hình 4. 8 Nhận diện giọng nói và quyết định nhấn <i>tắt_đèn_phòng_khách</i>	95
Hình 4. 9 Trạng thái hiển thị của các Node khi Offline.....	99
Hình 4. 10 Trạng thái hiển thị của các Node khi Online	99
Hình 4. 11 Kiểm thử chức năng tự động hóa kịch bản đơn	100
Hình 4. 12 Kiểm thử chức năng tự động hóa kịch bản kép	101

Hình 5. 1 Màn hình đăng nhập.....	110
Hình 5. 2 Màn hình Dashboard	110
Hình 5. 3 Màn hình tùy chọn hiển thị	110
Hình 5. 4 Màn hình chi tiết Node.....	110
Hình 5. 5 Màn hình Lịch sử Thông báo	110
Hình 5. 6 Màn hình cài đặt Hệ thống	110
Hình 5. 7 Màn hình danh sách kịch bản tự động	111
Hình 5. 8 Màn hình tạo và chỉnh sửa kịch bản.....	111
Hình 5. 9 Màn hình chế độ hoạt động.....	111

DANH MỤC BẢNG BIỂU

Bảng 3. 1 Yêu cầu chức năng (Functional Requirements)	29
Bảng 3. 2 Yêu cầu phi chức năng (Non-Functional Requirements)	30
Bảng 3. 3 Thông số kỹ thuật các linh kiện Node	31
Bảng 3. 4 Công suất tiêu thụ tối đa của các linh kiện	33
Bảng 3. 5 Số lượng cảm biến và thiết bị chấp hành cho từng Node	37
Bảng 3. 6 Quy tắc thiết kế mạch in PCB cho Node	44

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ (Tiếng Anh)
CNN	Convolutional Neural Network
DNN	Deep Neural Network
DS-CNN	Depthwise Separable Convolutional Neural Network
FCM	Firebase Cloud Messaging
FPGA	Field-Programmable Gate Array
HTTPS	HyperText Transfer Protocol Secure
KWS	Keyword Spotting
MFCC	Mel-Frequency Cepstral Coefficients
MQTT	Message Queuing Telemetry Transport
SDK	Software Development Kit
TFLM	TensorFlow Lite for Microcontrollers
TLS	Transport Layer Security

MỞ ĐẦU

1. Đặt vấn đề

Trong bối cảnh nhà thông minh (Smart Home) ngày càng phổ biến, nhu cầu điều khiển thiết bị nhanh – thuận tiện – tự nhiên đang tăng mạnh. So với điều khiển thông qua ứng dụng điện thoại hoặc công tắc truyền thống, điều khiển bằng giọng nói giúp con người dùng thao tác rảnh tay, đặc biệt hữu ích khi đang nấu ăn, làm việc, hoặc đối với người lớn tuổi.

Tuy nhiên, nhiều hệ thống điều khiển giọng nói hiện nay phụ thuộc vào internet và các gói dịch vụ đám mây, dẫn đến các hạn chế như: độ trễ, chi phí duy trì, và lo ngại về quyền riêng tư. Bên cạnh đó, tiếng Việt có đặc thù về ngữ âm và vùng miền, làm cho việc triển khai việc nhận dạng lệnh tiếng Việt ổn định trên thiết bị nhúng là một bài toán có tính thực tiễn cao.

Vì vậy, đề tài được chọn nhằm xây dựng một hệ thống nhận dạng giọng nói tiếng Việt chạy trực tiếp trên thiết bị (edge AI), không phụ thuộc mạng, có khả năng điều khiển các thiết bị điện trong nhà theo tập lệnh được định nghĩa trước.

Tính cấp thiết của đề tài nằm ở việc: Tăng trải nghiệm của người dùng với việc điều khiển tự nhiên, đảm bảo tính bảo mật do xử lý tại thiết bị, phù hợp triển khai thực tế nhờ chi phí phần cứng thấp.

2. Mục tiêu của đề tài

Mục tiêu của đề tài là thiết kế và triển khai một hệ thống điều khiển nhà thông minh bằng giọng nói tiếng Việt, trong đó mô hình nhận dạng được tối ưu để chạy trên vi điều khiển.

Cụ thể, đề tài hướng đến các mục tiêu sau:

- Xây dựng bộ điều khiển tiếng Việt (ví dụ: bật/tắt thiết bị, mở đóng...) và thiết kế nhân dữ liệu tương ứng.

- Thu thập, tiền xử lý và chuẩn hóa dữ liệu âm thanh phục vụ huấn luyện mô hình (cắt đoạn, lọc nhiễu, chuẩn độ dài mẫu...).
- Thiết kế và huấn luyện mô hình Keyword Spotting (KWS) với đặc trưng log-mel, tối ưu kích thước để nhúng.
- Chuyển đổi mô hình sang định dạng phù hợp (TFLite/TFLite Micro) và triển khai suy luận trực tiếp trên ESP32.
- Tích hợp hệ thống điều khiển thiết bị (ví dụ Relay) theo kết quả suy luận, đảm bảo hoạt động ổn định trong môi trường thực tế.

3. Đối tượng và phạm vi nghiên cứu

a) Đối tượng nghiên cứu:

- Tín hiệu giọng nói tiếng Việt trong bối cảnh điều khiển lệnh ngắn (keyword/command).
- Quy trình tiền xử lý âm thanh và trích xuất đặc trưng phục vụ nhận dạng lệnh.
- Mô hình học máy tối ưu cho thiết bị nhúng (KWS) và quá trình suy luận trên vi điều khiển.
- Hệ thống phần cứng phục vụ thu âm và điều khiển thiết bị (micro, ESP32, relay).
- Khả năng điều phối lệnh giọng nói điều khiển nhiều Node khác nhau.

b) Phạm vi nghiên cứu:

- Hệ thống không có khả năng lưu trữ các dữ liệu cảm biến do kinh phí thực hiện có hạn, Cloud Firebase yêu cầu trả phí cho vấn đề này.
- Không có các biểu đồ thể hiện giá trị các cảm biến
- Hệ thống tập trung vào việc nhận dạng tập lệnh giới hạn (nhóm lệnh đã định nghĩa trước), không mở rộng sang nhận dạng hội thoại dài hoặc chuyển giọng nói thành văn bản (ASR).
- Đề tài ưu tiên chứng minh tính khả thi và khả năng mở rộng.
- Dữ liệu và đánh giá chủ yếu trong môi trường trong nhà, nhiễu nền mức vừa và nhỏ (quạt, tiếng nói chuyện xa, tiếng sinh hoạt, ...).

- Mô hình được tối ưu để chạy offline trên thiết bị nhúng (ưu tiên độ trễ thấp và bộ nhớ phù hợp), phạm vi triển khai cho điều khiển thiết bị cơ bản như đèn/quạt/rèm thông qua Relay hoặc Module chấp hành tương đương.
- Giới hạn về thời gian và nguồn lực thu thập dữ liệu có thể khiến tính đa dạng giọng nói (vùng miền/độ tuổi) chưa bao phủ toàn bộ.

4. Ý nghĩa thực tiễn của đề tài

Đề tài có ý nghĩa ứng dụng rõ rệt vì tạo ra một giải pháp điều khiển nhà thông minh bằng giọng nói không phụ thuộc vào internet, phù hợp với điều kiện triển khai thực tế (chi phí thấp, dễ lắp đặt).

Cụ thể:

- Tính ứng dụng: Dùng điều khiển các thiết bị lệnh trong nhà bằng lệnh tiếng Việt, thao tác nhanh và tiện lợi.
- Giải quyết nhu cầu thực tế: Tăng tiện ích cho người dùng phổ thông, hỗ trợ người lớn tuổi/ khả năng vận động hạn chế nhờ điều khiển rảnh tay.
- Hiệu quả vận hành: Xử lý tại thiết bị giúp giảm độ trễ, tăng độ ổn định khi mất mạng, đồng thời cải thiện riêng tư và bảo mật dữ liệu giọng nói vì không cần gửi ra ngoài.

5. Kết cấu của báo cáo

Chương 1: Tổng quan về đề tài

Trình bày bối cảnh và lý do thực hiện đề tài, tổng hợp các hướng tiếp cận/ sản phẩm liên quan đã được nghiên cứu và triển khai, đồng thời chỉ ra những hạn chế còn tồn tại. Từ đó xác định khoảng trống và định hướng mà đề tài tập trung giải quyết, làm rõ tính cần thiết và hướng phát triển của hệ thống.

Chương 2: Cơ sở lý thuyết

Trình bày các cơ sở lý thuyết và thuật toán nền tảng phục vụ đề tài, bao gồm nguyên lý hoạt động của các phần tử phân cứng liên quan, các chuẩn giao tiếp truyền nhận

dữ liệu (UART, SPI, I2S...), cũng như các phương pháp xử lý tín hiệu và kỹ thuật nhận dạng phục vụ hệ thống điều khiển bằng giọng nói. Chương này là nền tảng để xây dựng phần thiết kế và triển khai ở chương sau.

Chương 3: Phân tích và thiết kế hệ thống

Phân tích yêu cầu và đề xuất kiến trúc tổng thể của hệ thống. trình bày chi tiết thiết kế phần cứng (Cảm biến, vi điều khiển, khối chấp hành...), thiết kế mô hình/thuật toán xử lý (tiền xử lý, trích đặc trưng, suy luận, cơ chế ra quyết định), và các tính toán – lựa chọn tham số nhằm đảm bảo hệ thống đáp ứng yêu cầu kỹ thuật và khả năng triển khai thực tế.

Chương 4: Thực nghiệm và đánh giá kết quả

Trình bày quá trình triển khai thử nghiệm, thu nhập số liệu và đánh giá kết quả theo các tiêu chí đã đặt ra. Bao gồm hình ảnh hệ thống thực tế, bảng/biểu đồ kết quả, phương pháp đo kiểm và đối sánh giữa thiết kế (Chương 3) với thực nghiệm. Từ kết quả thu được, rút ra nhận xét và mức độ đáp ứng mục tiêu, độ ổn định và những điểm cần cải tiến.

Kết luận và kiến nghị

Tổng hợp các kết quả đạt được, nêu các đóng góp chính của đề tài, đồng thời chỉ ra giới hạn còn tồn tại. trên cơ sở đó, đề xuất hướng phát triển và cải tiến trong các nghiên cứu /triển khai tiếp theo để nâng cao hiệu quả và khả năng ứng dụng thực tế của hệ thống.

CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu chương

Chương 1 trình bày bối cảnh hình thành đề tài và xu hướng nghiên cứu liên quan đến nhận dạng lệnh giọng nói dạng từ vựng giới hạn (Keyword Spotting – KWS) theo hướng chạy trực tiếp trên thiết bị (on-device/edge). Đồng thời, chương này tổng hợp các công trình tiêu biểu về kiến trúc mô hình nhỏ gọn, dữ liệu – cách đánh giá, nền tảng triển khai nhúng, và các phần liên quan đến nhận diện giọng nói tiếng Việt. Từ đó, rút ra khoảng trống và vấn đề thực tế mà đề tài hướng đến giải quyết.

1.2 Bối cảnh đề tài và xu hướng hiện nay

Trong các hệ thống nhà thông minh, điều khiển bằng giọng nói được xem là một kênh tương tác tự nhiên, giảm thao tác thủ công và phù hợp với các tình huống “rảnh tay”. Tuy nhiên, nhiều giải pháp thực tế vẫn phụ thuộc vào dịch vụ đám mây, dẫn đến các vấn đề phổ biến như độ trễ, phụ thuộc kết nối mạng và lo ngại về quyền riêng tư dữ liệu giọng nói.

Xu hướng gần đây là đưa suy luận mô hình xuống thiết bị (TinyML/on-device inference), ưu tiên xử lý tại chỗ để giảm độ trễ và nâng cao tính ổn định. Một nền tảng tiêu biểu cho hướng này là TensorFlow Lite Micro (TFLM), được thiết kế để chạy mô hình học sâu trên các hệ thống nhúng với bộ nhớ hạn chế.

Song song với sự phát triển của TFLM, hệ sinh thái Internet of Things (IoT) trong nhà thông minh cũng ngày càng mở rộng với kiến trúc đa node phân tán. Thay vì một thiết bị điều khiển trung tâm duy nhất, các hệ thống hiện đại thường bao gồm nhiều node cảm biến và thiết bị chấp hành được đặt tại các vị trí khác nhau trong không gian sống, giao tiếp thông qua các giao thức như MQTT hoặc Zigbee. Điều này đặt ra yêu cầu mới là làm thế nào để tích hợp khả năng nhận diện giọng nói offline vào một hệ thống IoT đa node một cách hiệu quả, đảm bảo tính đồng bộ, khả năng mở rộng và phản hồi thời gian thực.

1.3 Tổng quan bài toán nhận dạng giọng nói (Keyword spotting)

Keyword Spotting (KWS) là bài toán phát hiện/nhận dạng một tập từ khóa hoặc lệnh ngắn định nghĩa trước (ví dụ: “bật”, “tắt”, “mở”, “đóng” ...) từ tín hiệu âm thanh. Khác với ASR (nhận dạng câu dài), KWS tập trung vào từ vựng giới hạn, yêu cầu mô hình gọn nhẹ, suy luận nhanh và có cơ chế giảm báo nhầm (false positive) vì hệ thống thường hoạt động chế độ “luôn lắng nghe”.

Pipeline KWS điển hình gồm:

1. Thu âm và cắt cửa sổ tín hiệu theo thời gian;
2. Trích đặc trưng (phổ biến là log-mel/MFCC);
3. Mô hình phân loại (CNN/RNN/DS-CNN...);
4. Hậu xử lý theo ngưỡng, làm mượt theo thời gian và cơ chế “unknown/noise” để hạn chế chặn nhầm.

Trong nghiên cứu KWS, việc đánh đổi giữa độ chính xác - độ trễ - tài nguyên tính toán là trọng tâm, đặc biệt khi triển khai trên thiết bị nhỏ.

1.4 Các nghiên cứu liên quan trước đây

1.4.1 Nghiên cứu về CNN và DS-CNN cho nhận dạng từ khóa (KWS) trên thiết bị cấu hình thấp

Một hướng nghiên cứu kinh điển cho bài toán Keyword Spotting (KWS) trên thiết bị có tài nguyên hạn chế là sử dụng Mạng nơ-ron tích chập (Convolutional Neural Network – CNN). Công trình của Sainath và Parada [1] tại hội nghị Interspeech 2015 đã chứng minh rằng CNN có thể đạt độ chính xác tốt với số lượng tham số ít hơn đáng kể so với mạng DNN truyền thống, phù hợp với các ràng buộc "small-footprint" (giới hạn tham số và phép tính toán) trong bài toán nhận dạng từ khóa.

Để tối ưu hơn nữa cho thiết bị nhúng, nhiều nghiên cứu đã tập trung vào kiến trúc Depthwise Separable Convolution (DS-CNN) — kỹ thuật tách convolution chuẩn thành hai bước depthwise và pointwise - giúp giảm chi phí tính toán một cách đáng kể. Zhang và cộng sự [2] với công trình "Hello Edge" là nghiên cứu tiêu biểu trong hướng này, trình bày kiến trúc DS-CNN cho KWS trên vi điều khiển ARM Cortex-

M, đạt độ chính xác 95.4% trên tập Google Speech Commands. Công trình này nhấn mạnh khả năng mở rộng khi tăng độ sâu mạng nhưng vẫn kiểm soát được bộ nhớ và độ phức tạp tính toán.

Sørensen và cộng sự [3] đã mở rộng hướng nghiên cứu này bằng cách triển khai DS-CNN trên hệ thống nhúng công suất thấp, kết hợp kỹ thuật quantization (lượng tử hóa) để giảm thêm yêu cầu bộ nhớ mà không làm giảm độ chính xác đáng kể. Ngoài ra, nghiên cứu này còn áp dụng data augmentation để cải thiện độ robust của mô hình trong môi trường có nhiễu.

Các nghiên cứu gần đây còn triển khai DS-CNN trên nền tảng FPGA. Bae và cộng sự [4] đề xuất kiến trúc mạng neural nhị phân hóa và tam phân hóa (binarized and ternarized neural networks), đạt độ chính xác 97.1% cho wake-word detection trong khi giảm 49.3% diện tích chip so với các thiết kế trước đó. Xu và Zhang [15] cũng đề xuất kết hợp DS-CNN với kiến trúc ResNet và khối Squeeze-and-Excitation, đạt kết quả state-of-the-art với chỉ khoảng 10,000 tham số.

Về mặt dataset, Warden [5] đã công bố tập dữ liệu Google Speech Commands — hiện là benchmark chuẩn cho các nghiên cứu KWS quốc tế, bao gồm hơn 65,000 đoạn âm thanh một giây cho 30 từ khóa khác nhau, thu thập từ hàng nghìn người nói.

1.4.2 Nghiên cứu về nhận dạng giọng nói tiếng Việt

Với tiếng Việt, nền tảng dữ liệu công khai phục vụ nhận dạng giọng nói (ASR) đã bắt đầu được hình thành. Luong và Vu [8] đã công bố tập dữ liệu VIVOS — một trong những corpus tiếng Việt miễn phí đầu tiên phục vụ nghiên cứu nhận dạng giọng nói, gồm khoảng 15 giờ ghi âm từ 65 người nói.

Trong lĩnh vực nhận dạng giọng nói điều khiển thiết bị, Nguyễn Chí Ngôn và cộng sự [9] tại Đại học Cần Thơ đã sử dụng mạng nơ-ron nhân tạo kết hợp với kỹ thuật trích đặc trưng dựa trên biến đổi Fourier thời gian ngắn (STFT) để nhận dạng các lệnh điều khiển cơ bản ("Trái", "Phải", "Tới", "Lui"), đạt độ chính xác khoảng 90%.

Đây là một trong những công trình tiên phong về nhận dạng giọng nói tiếng Việt cho ứng dụng điều khiển tại Việt Nam.

Phan Duy Hung và cộng sự [10] đã xây dựng tập dữ liệu gồm 15 lệnh điều khiển nhà thông minh bằng tiếng Việt và sử dụng mạng nơ-ron Hồi Quy (Recurrent Neural Network - RNN), đạt độ chính xác 98.19%. Nghiên cứu này cho thấy tiềm năng của học sâu trong bài toán nhận dạng lệnh ngắn tiếng Việt.

Một hướng tiếp cận khác được Nguyen và Cao [11] đề xuất là kết hợp Support Vector Machine (SVM) và CNN để sửa lỗi nhận dạng lệnh tiếng Việt trên smartphone. Phương pháp này sử dụng Google Speech Recognition làm bước tiền xử lý, sau đó áp dụng mô hình học máy để hiệu chỉnh kết quả, giảm tỷ lệ lỗi từ 35% xuống còn 5%.

Ngoài ra, một số nghiên cứu đã khám phá các phương pháp truyền thống như Logic mờ (Fuzzy Logic) [12] cho nhận dạng giọng nói tiếng Việt, tận dụng đặc điểm đơn âm tiết của ngôn ngữ để đơn giản hóa quá trình nhận dạng.

1.5 Nhận xét và khoảng trống nghiên cứu

- Chưa có nghiên cứu tối ưu mô hình DS-CNN cho đặc điểm thanh điệu tiếng Việt. Tiếng Việt có 6 thanh điệu, khác biệt lớn với tiếng Anh, trong khi các mô hình từ [2], [3] được thiết kế cho tiếng Anh.
- Chưa có nghiên cứu tiếng Việt nào giải quyết vấn đề phối hợp đa thiết bị như đề cập trong [7]: thiết bị nào sẽ phản hồi khi có nhiều node cùng thu được lệnh? Làm sao tránh duplicate action?
- Chưa có giải pháp đồng bộ thời gian giữa các node cho tiếng Việt — trong hệ thống đa node, tín hiệu giọng nói đến các microphone ở thời điểm khác nhau do khoảng cách và độ trễ mạng.
- Chưa có nghiên cứu tích hợp end-to-end pipeline hoàn chỉnh từ phát hiện wake-word → nhận dạng lệnh → xác định thiết bị đích → điều khiển thiết bị → phản hồi người dùng. Phần lớn nghiên cứu [9], [10], [13] chỉ tập trung vào một phần của pipeline.

- Chưa hỗ trợ điều khiển theo phòng/vùng — ví dụ: "Bật đèn phòng khách" vs "Bật đèn phòng ngủ" đòi hỏi hệ thống hiểu ngữ cảnh vị trí, hiện chưa có nghiên cứu tiếng Việt nào đề cập.

1.6 Kết luận chương

Chương 1 đã trình bày bối cảnh và xu hướng on – device cho bài toán KWS, tổng hợp các nghiên cứu tiêu biểu về mô hình nhỏ gọn, dữ liệu – đánh giá và triển khai TinyML, đồng thời điếm qua các nghiên cứu liên quan đến tiếng Việt. Từ đó, chương này xác định vấn đề trọng tâm (dữ liệu lệnh tiếng Việt, chống nhiễu/nhận nhầm, tối ưu tài nguyên) làm cơ sở để triển khai cơ sở lý thuyết ở Chương 2 và thiết kế hệ thống ở Chương 3.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Giới thiệu chung

Chương này trình bày các cơ sở lý thuyết và nền tảng kỹ thuật được sử dụng trong đề tài, nhằm xây dựng hệ thống điều khiển thiết bị IoT thông qua nhận dạng giọng nói. Nội dung chương được chia thành hai nhóm chính tương ứng với hai thành phần cốt lõi của hệ thống.

- Nhận dạng lệnh giọng nói dạng từ vựng giới hạn (Keyword Spotting – KWS): Xử lý tín hiệu âm thanh, trích đặc trưng, mô hình phân loại, chuẩn hóa và triển khai suy luận trên thiết bị nhúng.
- Truyền thông dữ liệu theo thời gian thực trong hệ thống IoT. Phần này phân tích các giao thức truyền thông phổ biến như HTTPS/TLS/WebSocket và đồng thời trình bày kiến trúc tích hợp Firebase làm nền tảng trung gian: Mô hình giao tiếp Gateway gửi dữ liệu lên Firebase sử dụng Admin SDK và Firebase gửi dữ liệu xuống ứng dụng sử dụng Client SDK, đảm bảo cập nhật theo thời gian thực, đảm bảo độ trễ thấp và đồng bộ hóa liên tục giữa các thành phần trong hệ thống.

2.2 Xử lý tín hiệu tiếng nói trong miền số

2.2.1 Lấy mẫu và biểu diễn tín hiệu

Tín hiệu giọng nói trong tự nhiên là tín hiệu tương tự (analog), liên tục theo thời gian và biên độ. Để có thể xử lý bằng máy tính hoặc vi điều khiển, tín hiệu này cần được chuyển đổi sang dạng số (digital) thông qua quá trình số hóa, bao gồm hai bước chính:

- **Lấy mẫu (Sampling):** Rời rạc hóa tín hiệu theo thời gian bằng cách đo giá trị biên độ tại các thời điểm cách đều nhau với tần số lấy mẫu f_s . Theo định lý Nyquist-Shannon, tần số lấy mẫu phải lớn hơn hoặc bằng hai lần tần số cao nhất của tín hiệu ($f_s \geq 2f_{max}$) để tránh hiện tượng chồng phổ (aliasing).
- **Lượng tử hóa (Quantization):** Chuyển đổi biên độ liên tục thành các mức giá trị rời rạc, thường sử dụng độ phân giải 16-bit.

Trong bài toán nhận dạng lệnh giọng nói, tần số lấy mẫu $f_s = 16\text{kHz}$ được sử dụng phổ biến. Lựa chọn này đáp ứng đủ dải tần quan trọng của tiếng nói (thông tin ngữ âm tập trung chủ yếu trong khoảng $300\text{Hz} - 3400\text{Hz}$, có thể mở rộng đến 8kHz), đồng thời tiết kiệm tài nguyên tính toán và bộ nhớ so với các tần số cao hơn như 44.1kHz hay 48kHz .

2.2.2 Phân khung và cửa sổ hóa

Tín hiệu giọng nói thay đổi liên tục theo thời gian (không dừng), nhưng trong các khoảng ngắn khoảng $20-40\text{ms}$, tín hiệu có thể xem là tương đối ổn định. Do đó, tín hiệu được chia thành các khung (frame) có độ dài cố định, các khung liên kế thường chồng lấn $25-50\%$ để đảm bảo không mất thông tin tại vùng chuyển tiếp.

Trước khi phân tích phổ, mỗi khung được nhân với hàm cửa sổ (thường dùng cửa sổ Hamming) nhằm làm mượt biên độ tại hai đầu khung, giảm thiểu hiện tượng rò rỉ phổ khi thực hiện biến đổi Fourier.

2.2.3 Biến đổi Fourier ngắn hạn (STFT)

Biến đổi Fourier ngắn hạn (Short - Time Fourier Transform – STFT) là kỹ thuật phân tích phổ tần số của tín hiệu theo từng khung thời gian. Thay vì phân tích toàn bộ tín hiệu một lần, STFT áp dụng biến đổi Fourier trên từng khung đã được cửa sổ hóa, cho phép quan sát sự thay đổi thành phần tần số theo thời gian.

Kết quả của quá trình biến đổi này tạo thành Spectrogram – biểu đồ hai chiều với trục thời gian và trục tần số, cường độ màu thể hiện năng lượng tại mỗi điểm. Spectrogram là cơ sở nền tảng để tạo ra các đặc trưng như Mel-Spectrogram.

2.3 Trích xuất đặc trưng cho KWS: Mel Filterbank và Log-Mel

2.3.1 Mel scale và bộ lọc tam giác

Thính giác con người cảm nhận tần số phi tuyến tính: nhạy hơn ở vùng thấp và "nén" ở vùng cao. Mel scale mô hình hóa hiện tượng này, vì vậy Mel filterbank (thường là bộ lọc tam giác trên trục Mel) được sử dụng rộng rãi trong bài toán tiếng nói. Các triển khai filterbank kinh điển thường tham chiếu Auditory Toolbox.

2.3.2 Log – mel spectrogram

Sau khi tính năng lượng qua các dải Mel, áp dụng hàm log để giảm độ chênh lệch biên độ và làm đặc trưng ổn định hơn trước biến thiên cường độ. Log – mel là đầu vào phù hợp cho CNN trong KWS vì giữ được cấu trúc “ảnh thời gian – tần số”.

2.3.3 Liên hệ với MFCC

MFCC là đặc trưng truyền thống được tạo ra từ Log-mel thông qua biến đổi cosine rời rạc (DCT) để giảm tương quan và giảm chiều. MFCC được giới thiệu sớm và trở thành chuẩn phổ biến trong nhận dạng tiếng nói.

2.4 Chuẩn hóa đặc trưng

Trong thực tế, dữ liệu có thể thay đổi theo người nói, môi trường và mức âm lượng. Chuẩn hóa giúp mô hình học ổn định hơn và giảm lệch phân phối giữa lúc train và lúc chạy thật.

- Chuẩn hóa theo mean/std (z – score): $x' = \frac{x-\mu}{\sigma}$ theo từng kênh mel hoặc toàn bộ tensor đặc trưng.
- Khi khai triển nhúng, μ và σ có thể được “đóng gói” vào firmware để đảm bảo pipeline inference giống training.

2.5 Cơ sở mô hình học sâu cho Keyword Spotting

2.5.1 Bài toán phân loại đa lớp

KWS trong đề tài được mô hình hóa như bài toán phân loại với đầu vào của mô hình là đặc trưng Log-mel spectrogram có dạng tensor 2D (thời gian $\times$ tần số), đầu ra là vector xác suất tương ứng với từng lớp. Lớp có xác suất cao nhất được chọn làm kết quả dự đoán thông qua Softmax để tạo phân phối xác suất.

2.5.2 Kiến trúc CNN và biến thể tối ưu

Mạng nơ-ron tích chập (Convolutional Neural Network – CNN) phù hợp với bài toán KWS vì khả năng trích xuất đặc trưng cục bộ từ dữ liệu dạng "ảnh" như Log-mel spectrogram. Kiến trúc CNN cơ bản bao gồm:

- Lớp tích chập (Convolutional layer): trích xuất đặc trưng cục bộ theo không gian thời gian – tần số.
- Lớp pooling: giảm kích thước đặc trưng, tăng tính bất biến với dịch chuyển nhỏ.
- Lớp fully - connected: kết hợp đặc trưng và đưa ra kết quả phân lớp.

Đối với triển khai trên thiết bị nhúng có tài nguyên hạn chế, các biến thể CNN tối ưu được sử dụng như Depthwise Separable Convolution (DS-CNN): tách tích chập thành hai bước (depthwise và pointwise), giảm đáng kể số lượng tham số và phép tính so với tích chập chuẩn (Phần này Chương 1 đã làm rõ trong mục tổng quan nghiên cứu; chương 2 tập trung nguyên lý).

2.5.3 Hàm mất mát và tối ưu

- Cross – entropy thường dùng cho phân loại đa lớp.
- Các kỹ thuật như Early Stopping, Class Weighting... được dùng để cải thiện tổng quát hóa khi dữ liệu các lớp không cân bằng.

2.6 Hậu Xử Lý Quyết Định (Decision Logic) Cho Kws Chạy Thực Tế

Một hệ KWS chạy thật không chỉ dựa vào "argmax" một lần, mà cần cơ chế giảm nhận nhầm:

- Ngưỡng xác suất (confidence threshold): Chỉ chấp nhận lệnh nếu xác suất lớp mục tiêu vượt ngưỡng.
- Làm mượt theo thời gian: Lấy trung bình/majority vote trên nhiều cửa sổ liên tiếp để giảm nhiễu.
- Cơ chế "cooldown/debounce": Sau khi kích hoạt một lệnh, tạm khóa kích hoạt trong một khoảng ngắn để tránh lặp lệnh do cùng một câu nói.
- Nhóm lớp unknown/noise: Giúp mô hình "từ chối" khi gặp câu nói ngoài tập lệnh, từ đó hạn chế False Positive.

2.7 Triển khai suy luận trên thiết bị nhúng (Tflite Micro)

2.7.1 Tổng quan TensorFlow Lite Micro

TensorFlow Lite Micro (TFLM) [21] là một phiên bản thu gọn của TensorFlow Lite, được thiết kế để chạy các mô hình học sâu trên vi điều khiển và các thiết bị nhúng có tài nguyên hạn chế. TFLM có các đặc điểm chính:

- Không yêu cầu hệ điều hành, có thể chạy trực tiếp trên bare - metal.
- Không sử dụng cấp phát bộ nhớ động (dynamic memory allocation), phù hợp với môi trường nhúng.
- Hỗ trợ các kiến trúc MCU phổ biến như ARM Cortex-M, ESP32, Arduino.
- Kích thước thư viện nhỏ (thường dưới 100KB tùy cấu hình).

2.7.2 Lượng tử hóa INT8 (Quantization)

Mô hình học sâu thường được huấn luyện với trọng số dạng float32, chiếm nhiều bộ nhớ và yêu cầu phép tính dấu phẩy động. Để triển khai trên MCU, mô hình cần được lượng tử hóa (quantization) nhằm:

- Giảm kích thước mô hình (float32 $\rightarrow$ int8: giảm khoảng 4 lần).
- Tăng tốc suy luận nhờ sử dụng phép tính số nguyên.
- Giảm tiêu thụ năng lượng.

2.7.3 Quản lý bộ nhớ: Tensor Arena

TFLM sử dụng cơ chế quản lý bộ nhớ tĩnh thông qua một vùng nhớ gọi là tensor arena. Đây là một mảng byte được cấp phát trước, dùng để chứa:

- Các tensor trung gian trong quá trình suy luận.
- Cấu trúc dữ liệu nội bộ của interpreter.

Kích thước tensor arena cần được ước lượng phù hợp với mô hình. Nếu quá nhỏ, inference sẽ thất bại; nếu quá lớn, sẽ lãng phí RAM vốn hạn chế trên MCU.

2.8 Cơ sở truyền thông MQTT và HTTPS/TLS/WEBSOCKET, cơ chế thời gian thực qua Firebase và cơ chế gửi thông báo bằng FCM

2.8.1 *Giao thức truyền thông không dây MQTT*

MQTT (Message Queuing Telemetry Transport) là một giao thức truyền thông publish/subscribe hoạt động trên tầng ứng dụng (Application Layer) của mô hình OSI, được tối ưu cho thiết bị IoT công suất thấp và mạng không ổn định.

Trong đề tài, MQTT được triển khai trên Raspberry Pi 4B với vai trò MQTT Broker sử dụng dịch vụ Eclipse Mosquitto, còn các node ESP32-S3 đóng vai trò Publisher và Subscriber để gửi/nhận dữ liệu điều khiển.

2.8.2 *Khái niệm HTTP và HTTPS*

HTTP (Hypertext Transfer Protocol) là giao thức tầng ứng dụng theo mô hình trao đổi yêu cầu – phản hồi (request/response) có tính chất stateless. HTTPS là biến thể của HTTP sử dụng kênh truyền bảo mật TLS (Transport Layer Security), trong đó RFC về HTTP cũng định nghĩa rõ hai URI scheme "http" và "https".

Trong phạm vi đề tài, HTTPS được xem là lớp giao tiếp nền tảng để các thành phần (Gateway/ứng dụng) trao đổi dữ liệu với dịch vụ đám mây theo cách phổ biến, dễ tích hợp và có khả năng mở rộng tốt.

2.8.3 *Khái niệm WebSocket*

WebSocket là một giao thức cho phép thiết lập kênh giao tiếp hai chiều (two-way) giữa client và server, bao gồm bước opening handshake và cơ chế message framing, hoạt động trên nền TCP. Mục tiêu của WebSocket là duy trì kết nối và trao đổi dữ liệu liên tục thay vì lặp lại nhiều request/response như HTTP truyền thống.

Trong bối cảnh hệ thống thời gian thực, WebSocket thường được nhắc đến như một cơ chế hỗ trợ cập nhật dữ liệu nhanh và liên tục giữa hai đầu giao tiếp.

2.8.4 Khái niệm Firebase Admin SDK và Firebase Client SDK trong kiến trúc hệ thống

Firebase cung cấp hai nhóm SDK phù hợp với vai trò của từng thành phần trong hệ thống:

- Firebase Admin SDK [18] (Gateway → Firebase): Là bộ thư viện phục vụ môi trường đặc quyền như server/gateway. Tài liệu chính thức nêu rõ để sử dụng Admin SDK cần có Firebase project và service account, đồng thời sử dụng tệp cấu hình thông tin định danh để khởi tạo SDK và giao tiếp với Firebase.
- Firebase Client SDK [19] (Firebase → App): Là bộ thư viện dùng cho phía ứng dụng người dùng (mobile/web). Ứng dụng sử dụng Client SDK để đọc dữ liệu và thiết lập cơ chế lắng nghe thay đổi (onSnapshot () đối với Firestore) nhằm nhận cập nhật theo thời gian thực.

Từ đó, luồng khái quát trong đề tài được mô tả như sau: Gateway cập nhật dữ liệu lên Firebase thông qua Admin SDK; khi dữ liệu thay đổi, ứng dụng nhận cập nhật thông qua cơ chế realtime của Client SDK; việc truyền thông qua Internet được bảo vệ theo lớp HTTPS/TLS.

2.8.5 Khái niệm Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging (FCM) [20] là dịch vụ nhắn tin/đẩy thông báo (push messaging) do Firebase cung cấp, cho phép hệ thống gửi thông báo từ môi trường đáng tin cậy (server/gateway hoặc Cloud Functions) đến thiết bị người dùng (Android/iOS/Web). Một triển khai FCM điển hình gồm hai thành phần chính: phía gửi thông báo (trusted environment) và ứng dụng client nhận thông báo thông qua cơ chế vận chuyển tương ứng với từng nền tảng.

Trong phạm vi đề tài, FCM được xem như cơ chế hỗ trợ gửi thông báo chủ động tới ứng dụng (thay đổi trạng thái thiết bị, cảnh báo, sự kiện quan trọng), bổ sung cho cơ chế đồng bộ dữ liệu Realtime của Firebase.

2.8.6 Cơ chế gửi và nhận thông báo FCM trong hệ thống

Mỗi phiên bản ứng dụng client sau khi tích hợp FCM sẽ nhận được một registration token (định danh thiết bị/instance ứng dụng) dùng để định tuyến thông báo tới đúng thiết bị. Phía gửi (server/gateway) có thể gửi thông báo tới một thiết bị cụ thể thông qua token, nhiều thiết bị bằng danh sách token (multicast), hoặc nhóm thiết bị theo chủ đề (topic).

Trong đề tài, Gateway gửi thông báo theo hai cách phổ biến:

- Firebase Admin SDK: Hỗ trợ gửi đến token, multicast (tối đa 500/lần gọi), hoặc gửi theo danh sách message.
- FCM HTTP v1 API: Cho phép xây dựng request gửi thông báo nhắm đến token, topic hoặc condition.

Phía ứng dụng nhận thông báo qua SDK, sau đó hiển thị hoặc xử lý tùy theo logic ứng dụng. FCM hỗ trợ hai loại thông điệp chính: thông điệp hiển thị (notification message) dùng để thông báo trạng thái/nhắc nhở người dùng, và thông điệp dữ liệu (data message) để ứng dụng tự xử lý như cập nhật giao diện hoặc đồng bộ trạng thái.

CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Giới thiệu Tổng quan về Hệ thống

Trong bối cảnh công nghệ IOT (Internet of Things) đang phát triển mạnh mẽ, việc tự động hóa và giám sát nhà cửa trở thành một nhu cầu thiết yếu của người dùng, đề tài hướng đến giải pháp tích hợp, dễ dàng triển khai và có chi phí hợp lý nhưng vẫn có thể giúp người dùng giám sát các thông số quan trọng (như nhiệt độ, độ ẩm, an ninh) và điều khiển các thiết bị từ xa một cách linh hoạt.

Thiết kế hệ thống sẽ đi sâu vào các điểm chính của đề tài như: Thiết kế phần cứng, thiết kế các giải thuật cho code (bao gồm phần Thuật toán điều phối IOT và Thuật toán nhận diện giọng nói), thiết kế ứng dụng di động (App). Giúp phác thảo rõ hơn một hệ thống hoạt động ổn định và thân thiện với người dùng.

3.2 Lựa chọn phương án thiết kế tổng thể

Phương án thiết kế tổng thể được lựa chọn cho hệ thống nhà thông minh là kiến trúc phân tầng Node – Gateway, sử dụng vi điều khiển ESP32 cho vai trò Node và máy tính nhúng Raspberry Pi cho vai trò Gateway điều phối trung tâm.

Kiến trúc phân tầng cho phép phân tán tải xử lý hiệu quả với sự tách biệt rõ ràng chức năng: Node tập trung vào thu thập dữ liệu, xử lý mức thấp và điều khiển thiết bị chấp hành; Gateway đảm nhiệm các tác vụ xử lý chuyên sâu như tổng hợp dữ liệu, ra quyết định và kết nối với nền tảng Cloud. Cách tiếp cận này mang lại các ưu điểm:

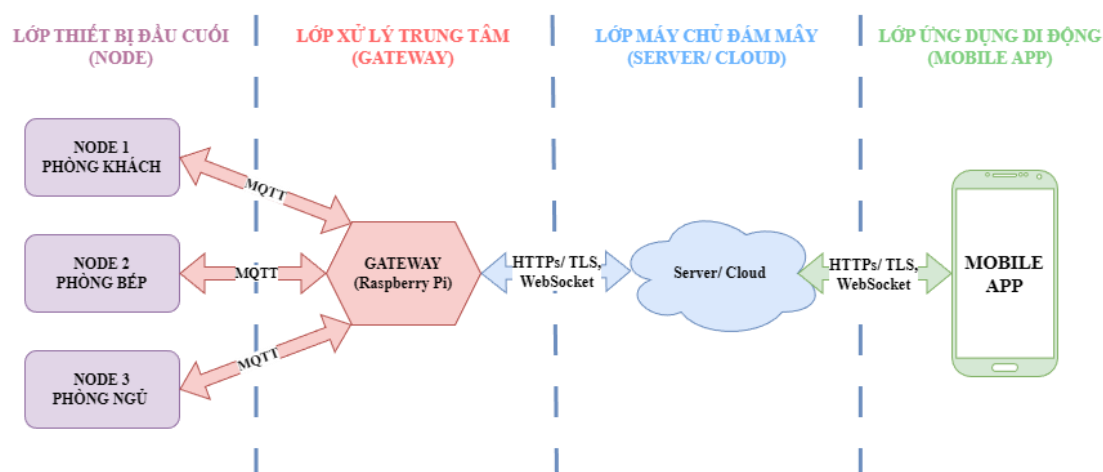
- Giảm tải tính toán, tiêu thụ năng lượng và độ phức tạp phần mềm tại Node.
- Đáp ứng thời gian thực và duy trì hoạt động cục bộ khi mất kết nối Internet.
- Tạo điểm kiểm soát tập trung về bảo mật, hạn chế bề mặt tấn công so với mô hình Node kết nối trực tiếp lên Cloud.
- Mở rộng số lượng Node linh hoạt mà không tăng gánh nặng cho hạ tầng mạng.

3.3 Kiến trúc Tổng quát của Hệ thống

Hệ thống nhà thông minh điều khiển bằng giọng nói được thiết kế theo kiến trúc bốn lớp, bao gồm:

- Lớp thiết bị đầu cuối (Node).
- Lớp xử lý trung tâm (Gateway).
- Lớp máy chủ đám mây (Server/ Cloud).
- Lớp ứng dụng di động (App di động).

Mỗi lớp đảm nhiệm một vai trò riêng biệt, phối hợp hoạt động nhằm đảm bảo hiệu quả và mở rộng linh hoạt. Được thể hiện qua Hình 3.1



Hình 3. 1 Sơ đồ Kiến trúc Tổng quát của Hệ thống và Giao thức truyền tải dữ liệu trong hệ thống

Trong đó:

- Lớp thiết bị đầu cuối (Node): Các Node được triển khai phân tán tại nhiều khu vực trong nhà, đảm nhiệm thu thập dữ liệu từ cảm biến, thu âm và xử lý tín hiệu âm thanh cho chức năng điều khiển giọng nói, đồng thời thực thi điều khiển thiết bị chấp hành theo lệnh từ Gateway.
- Lớp xử lý trung tâm (Gateway): Gateway tiếp nhận dữ liệu từ các Node, quản lý MQTT Broker, xử lý nhận diện lệnh giọng nói và định tuyến lệnh đến Node

đích. Đồng thời, Gateway cập nhật trạng thái hệ thống lên Cloud, đóng vai trò điều phối trung tâm và giảm tải xử lý cho các Node.

- Lớp máy chủ đám mây (Cloud): Nền tảng Firebase lưu trữ dữ liệu thời gian thực và theo dõi trạng thái thiết bị, cho phép người dùng giám sát và điều khiển hệ thống từ xa.
- Lớp ứng dụng di động (Mobile App): Trực quan hóa dữ liệu môi trường và cho phép người dùng gửi lệnh điều khiển thiết bị một cách thuận tiện.

3.4 Giao thức truyền tải dữ liệu trong hệ thống

Hệ thống sử dụng hai nhóm giao thức chính cho việc đảm bảo truyền tải dữ liệu giữa các lớp, chi tiết được thể hiện thông qua Hình 3. 1 Sơ đồ Kiến trúc Tổng quát của Hệ thống và Giao thức truyền tải dữ liệu trong hệ thống.

Giao tiếp Node – Gateway (mạng cục bộ): Sử dụng giao thức MQTT trên nền tảng WiFi/TCP/IP. Gateway được cấu hình như MQTT Broker quản lý luồng dữ liệu, các Node đóng vai trò MQTT Client sử dụng cơ chế Publish/Subscribe để truyền nhận dữ liệu cảm biến và lệnh điều khiển. Giao thức MQTT đảm bảo truyền tải hiệu quả, tiết kiệm năng lượng và độ trễ thấp trong mạng cục bộ.

Giao tiếp Gateway – Cloud – App (qua Internet): Sử dụng Firebase SDK API trên nền tảng HTTPS/TLS và WebSocket. Gateway sử dụng Firebase Admin SDK, ứng dụng di động sử dụng Firebase Client SDK. Cơ chế này đảm bảo đồng bộ dữ liệu thời gian thực, bảo mật khi truyền qua Internet và phân quyền truy cập thông qua các quy tắc bảo mật của Firebase.

3.5 Phân tích và thiết kế phần cứng hệ thống

3.5.1 Phân tích yêu cầu kỹ thuật

Dựa vào những đặc điểm của kiến trúc hệ thống đã được phân tích, phần này làm rõ hơn các yêu cầu cần thiết để hệ thống hoạt động ổn định và hiệu quả.

Các yêu cầu được phân loại thành hai nhóm chính:

- Yêu cầu chức năng (Functional Requirements): Xác định những tác vụ bắt buộc mà phần cứng phải đáp ứng, là cơ sở để lựa chọn các linh kiện cần thiết cho hệ thống.
- Yêu cầu phi chức năng (Non-Functional Requirements): Tập trung vào chất lượng và các ràng buộc của hệ thống, nhằm đảm bảo hiệu suất và tính thẩm mỹ cho hệ thống nhà thông minh.

Chi tiết được thể hiện thông qua hai bảng sau:

Bảng 3. 1 Yêu cầu chức năng (Functional Requirements)

STT	Yêu cầu chức năng	Thông số kỹ thuật yêu cầu
1	Giám sát môi trường	Đo nhiệt độ (0-50 độ C) Đo độ ẩm (0-100%) Đo nồng độ khí Gas
2	Giám sát an ninh	Phát hiện con người ngay cả khi đứng yên
3	Điều khiển 5 thiết bị tải điện	Điều khiển bật/ tắt tải 125V đến 250V (AC) với khả năng chịu tải tối đa 10A
4	Mở rộng khoảng cách truyền dữ liệu	Hỗ trợ chuyển đổi giao tiếp UART sang RS485 để truyền dữ liệu ổn định trên khoảng cách xa, chống nhiễu.
5	Giao tiếp Bus có dây	Hỗ trợ giao tiếp I2C, I2S, UART
6	Thu nhận tín hiệu âm thanh	Thu âm thanh, lấy mẫu cho luồng âm thanh tốc độ cao $\geq 16\text{kHz}$
7	Giao tiếp không dây	Hỗ trợ giao thức Wifi chuẩn 802.11 b/g/n và hoạt động ổn định với MQTT
8	Nền tảng Gateway	Hỗ trợ chạy hệ điều hành Linux đầy đủ để triển khai MQTT Broker và Firebase Admin SDK

Bảng 3. 2 Yêu cầu phi chức năng (Non-Functional Requirements)

STT	Yêu cầu phi chức năng	Tiêu chí kỹ thuật cụ thể
1	Ổn định nguồn cấp	Mạch nguồn phải cung cấp điện áp ổn định 3.3V và 5V (DC).
2	Cách ly với điện áp AC	Có khả năng, chống nhiễu từ Relay và tải AC, đảm bảo cách ly điện (Isolation) giữa mạch điều khiển và tải AC.
3	Kích thước vật lý	Kích thước mạch PCB của Node phải nhỏ gọn, dễ dàng lắp đặt (10cm x 15cm).
4	Khả năng Mở rộng I/O	MCU phải có đủ chân GPIO phục vụ để kết nối các giao thức I2C, I2S, UART.
4	Độ Chính xác Dữ liệu	Các cảm biến môi trường phải có độ chính xác nằm trong ngưỡng chấp nhận được (Ví dụ: Nhiệt độ $\pm 0.5^{\circ}\text{C}$, độ ẩm $\pm 5\%$).

3.5.2 Lựa chọn và Mô tả linh kiện

Các linh kiện được lựa chọn dựa trên tiêu chí: đáp ứng yêu cầu chức năng của hệ thống, tương thích với vi điều khiển ESP32-S3, có độ tin cậy cao và được hỗ trợ tốt bởi cộng đồng. Đặc biệt, ESP32-S3 được chọn vì hỗ trợ TensorFlow Lite Micro phù hợp với phát triển các ứng dụng AI như tác vụ nhận diện giọng nói tại Node, đồng thời có đủ tài nguyên về các giao tiếp ngoại vi quan trọng như I2C, I2S, UART, ADC giúp phát triển các ứng dụng IOT song song với nhận diện giọng nói. Các linh kiện được lựa chọn được thể hiện trong bảng dưới đây:

Bảng 3. 3 Thông số kỹ thuật các linh kiện Node

Linh kiện	Chức năng	Thông số chính	Giao tiếp
ESP32-S3 N16R8	MCU	Dual - core 240MHz, WiFi, BLE 5.0, PSRAM 8MB	-
DHT22	Nhiệt độ, độ ẩm	-40~80°C, 0-100%RH, độ phân giải 0.1	1-Wire
MQ2	Khí gas, khói	300-10. 000 ppm	Analog/Digital
BH1750	Cường độ ánh sáng	1 - 65. 535 lux	I2C
HLK-LD2410B	Hiện diện người	Radar 24GHz, 0-8m	UART/GPIO
INMP441	Thu âm thanh	PCM 24-bit, 60Hz-15kHz, SNR 61dB	I2S

3.5.3 Module Giao tiếp và Truyền thông

Nhằm đáp ứng yêu cầu mở rộng khoảng cách truyền dữ liệu. Module giao tiếp truyền thông trong hệ thống sử dụng chuẩn RS485, thông qua mạch chuyển đổi tín hiệu UART của vi điều khiển sang chuẩn truyền vi sai RS485.

Về mặt kỹ thuật, với điểm mạnh đáp ứng yêu cầu truyền dữ liệu ổn định trên khoảng cách xa. RS485 là chuẩn truyền thông công nghiệp sử dụng cặp tín hiệu vi sai (A/B), cho khả năng chống nhiễu tốt, suy hao thấp và hỗ trợ truyền dữ liệu ở khoảng cách lên đến hàng trăm mét trong môi trường có nhiễu điện từ. Đồng thời cho phép hệ thống hoạt động ở chế độ half-duplex, phù hợp với kiến trúc truyền thông nhiều Node

3.5.4 Các cơ cấu chấp hành và đầu ra (Output)

A) Mạch điều khiển thiết bị công suất cao (Relay)

Để điều khiển các phụ tải AC 220V, hệ thống sử dụng Module Relay Cách ly Quang (Optically Isolated Relay). Giải pháp này đáp ứng hai yêu cầu kỹ thuật cốt lõi:

- An toàn điện: Sử dụng Optocoupler để truyền tín hiệu điều khiển bằng ánh sáng, đảm bảo cách ly vật lý hoàn toàn giữa mạch điều khiển (3.3V) và mạch động lực (220V).
- Chống nhiễu hệ thống: Ngăn chặn xung điện áp ngược và nhiễu điện từ (EMI) phát sinh trong quá trình đóng ngắt tải, bảo vệ vi điều khiển ESP32 và tăng cường độ ổn định cho toàn hệ thống.

B) Cơ cấu điều khiển rèm cửa thông minh

Hệ thống rèm được thiết kế theo mô hình Phân cấp xử lý (Hierarchical Actuation) nhằm tối ưu hóa hiệu suất của Node ESP32 chính.

Nguyên lý hoạt động: Node chính phát lệnh qua giao thức UART, được chuyển đổi sang chuẩn RS485 để truyền dẫn. Tại đầu cuối, một ESP32 phụ (Slave) tiếp nhận, giải mã và trực tiếp điều khiển Motor Driver.

Ưu điểm kỹ thuật:

- Độ tin cậy cao: Chuẩn RS485 sử dụng tín hiệu vi phân (Differential Signaling), giúp chống nhiễu vượt trội và đảm bảo tín hiệu khi truyền tải khoảng cách xa.
- Tối ưu tài nguyên: Việc chuyển giao nhiệm vụ điều khiển vị trí (Position Control) cho ESP32 phụ giúp Node chính tập trung tài nguyên xử lý âm thanh (I2S) và duy trì kết nối MQTT/Gateway.
- Độc lập vận hành: Node rèm đóng vai trò là cơ cấu chấp hành thuần túy, không tham gia vào mạng không dây, giúp giảm tải băng thông Wi-Fi và tăng tính bảo mật nội bộ.

C) Thiết bị báo hiệu và cảnh báo

Sử dụng còi chip (Buzzer) và LED trạng thái để phản hồi tương tác người dùng

3.5.5 Nguồn cấp

Nhằm đảm bảo hiệu suất của hệ thống, và khả năng đáp ứng của nguồn cấp luôn ổn định khi hoạt động ở chế độ tải nặng nhất (bao gồm truyền dữ liệu liên tục thông qua Wifi và tất cả các cơ cấu chấp hành kích hoạt đồng thời). Dựa trên các yêu cầu về phản ứng và chi tiết các linh kiện dễ dàng lập được bảng tiêu thụ dòng điện đỉnh (Bảng 3.4) của mỗi linh kiện.

Bảng 3. 4 Công suất tiêu thụ tối đa của các linh kiện

STT	Thiết bị và linh kiện	V cung cấp (V)	I_max (mA)	P_max = V×I (W)
1	ESP32-S3 (board, Wi-Fi TX peak)	5.00	300.0	$5.00 \times 0.3000 = 1.50 \text{ W}$
2	Microphone INMP441	3.30	5.0	$3.30 \times 0.005 = 0.0165 \text{ W}$
3	DHT22	5.00	2.5	$5.00 \times 0.0025 = 0.0125 \text{ W}$
4	MQ-2 module (heater)	5.00	150.0	$5.00 \times 0.1500 = 0.75 \text{ W}$
5	BH1750	5.00	0.2	$5.00 \times 0.0002 = 0.001 \text{ W}$
6	HLK-LD2410B (mmWave)	5.00	60.0	$5.00 \times 0.0600 = 0.3 \text{ W}$
7	Relay (5 × 80 mA)	5.00	400.0	$5.00 \times 0.4000 = 2.0 \text{ W}$
8	Buzzer (active)	5.00	50.0	$5.00 \times 0.0500 = 0.25 \text{ W}$
9	Các linh kiện phụ trợ (LED báo hiệu)	5.00	20.0	$5.00 \times 0.0200 = 0.10 \text{ W}$

Công suất tổng:

$$P_{\text{tổng}} = 1.5 + 0.0165 + 0.0125 + 0.75 + 0.001 + 0.3 + 2.0 + 0.25 + 0.1 = 4.93 \text{ W}$$

Với tổng công suất tiêu thụ của hệ thống xấp xỉ **4.93 W**, nếu cấp nguồn **5 V trực tiếp**, dòng điện yêu cầu tối thiểu được xác định theo:

$$I_{5V} = \frac{P}{V} = \frac{4.93}{5} = 0.9877 \text{ A} \approx 1 \text{ A} \quad (3-1)$$

Tuy nhiên, việc cấp nguồn 5V có biên điện áp nhỏ và dễ bị ảnh hưởng bởi sụt áp và nhiễu, có thể gây mất ổn định cho vi điều khiển và các cảm biến. Việc lựa chọn chuyển đổi từ điện áp 12V xuống 5V ngay tại mạch giúp đảm bảo điện áp cấp luôn ổn định và nâng cao độ tin cậy khi hệ thống vận hành liên tục nhờ những ưu điểm sau đây:

- Khả năng dự phòng Dòng điện (Current Headroom) xem phân tích thêm tại *Phụ lục A – Phần cứng, mục Khả năng dự phòng Dòng điện* cho ra được kết luận tỷ lệ dòng điện giữa hai phương án cấp nguồn 5V trực tiếp và chuyển đổi điện áp từ 12V xuống 5V là dòng điện phía 12 V chỉ bằng ~46% so với cấp trực tiếp 5 V.
- Độ ổn định Điện áp (Voltage Stability) xem phân tích thêm tại *Phụ lục A – Phần cứng, mục Độ ổn định điện áp* cho ra được kết luận sụt áp trên đường 12V nhỏ hơn 2 lần so với 5V trực tiếp, giúp điện áp đầu vào của Buck Converter ổn định hơn khi tải thay đổi.

Dựa vào những phân tích và tính toán, việc lựa chọn cấp nguồn 12V sau đó giảm điện áp xuống 5V là cần thiết. Cùng với công suất tiêu thụ tại 5V là 4.93W tương ứng với công suất đầu vào 12V ta tính được dòng tối thiểu cần có như sau:

Công suất đầu vào tại bus 12 V:

$$P_{in} = \frac{P_{out}}{\eta} \quad (3-2)$$

Dòng tối thiểu:

$$I_{12V} = \frac{P_{in}}{12} = \frac{P_{out}}{12\eta} \quad (3-3)$$

Trong đó: η là hiệu suất chuyển đổi

- Với $\eta = 90\%$ (0.90):

$$I_{12V} = \frac{4.93}{12 \times 0.90} = \frac{4.93}{10.8} = 0.45648 \text{ A} \approx 0.46 \text{ A} \quad (3-4)$$

- Với $\eta = 96\%$ (0.96):

$$I_{12V} = \frac{4.93}{12 \times 0.96} = \frac{4.93}{11.52} = 0.4281 \text{ A} \approx 0.43 \text{ A} \quad (3-5)$$

Từ các tính toán chi tiết về công suất tiêu thụ đỉnh của Node (bao gồm ESP32 ở chế độ WiFi đỉnh, các Relay kích hoạt đồng thời, và các module khác), dòng điện tối thiểu lý thuyết $I_{min} \approx 0.43\text{--}0.46 \text{ A}$

Tuy nhiên, trong thực tế dòng thiết kế luôn cần một khoảng dự phòng, kết hợp với hiệu suất hoạt động ổn định của các mạch hạ áp thường sẽ là 60% - 70% công suất thiết kế, cùng với kịch bản có thể mở rộng các thiết bị trong tương lai dự phòng $I_{future} = 1 \text{ A}$ tại 12V.

$$I_{design} = I_{min} \times 1.30 \approx 0.46 \times 1.40 \approx 0.644 \text{ A} \quad (3-6)$$

$$I_{total} = I_{future} + I_{design} = 0.644 + 1 = 1.644 \text{ A}$$

Tổng kết lựa chọn: Dựa trên tính toán, cần một bộ chuyển đổi nguồn có khả năng cung cấp dòng điện $\geq 1.644 \text{ A}$. Do đó, việc quyết định lựa chọn IC Buck Converter có dòng danh định tối thiểu tiêu chuẩn là $\geq 2.0 \text{ A}$.

3.5.6 Nguồn cách ly dành cho mạch điều khiển Relay bằng B0505S 2W

Hệ thống sử dụng nguồn cấp độc lập cho khối Relay nhằm tách biệt hoàn toàn với nguồn nuôi vi điều khiển. Giải pháp này giúp triệt tiêu nhiễu điện từ (EMI) và các xung điện áp ngược phát sinh khi đóng cắt tải, từ đó ngăn ngừa hiện tượng sụt áp cục bộ gây reset hệ thống và bảo vệ linh kiện điều khiển khỏi hư hỏng do dòng xung.

Nguồn cách ly B0505S 2W được lựa chọn vì khả năng đáp ứng được tính toán như sau:

$$I_{\max} = \frac{P_{\max}}{U} = \frac{2W}{5V} = 0.40 A \quad (3-7)$$

Tải tối thiểu: Hầu hết các nhà sản xuất (ví dụ MORNSUN, Hi-Link) khuyến nghị tải tối thiểu không dưới 10% công suất định mức (khoảng 40mA đối với phiên bản 2W).

Phạm vi tải lý tưởng: Duy trì tải đầu ra trong khoảng 50% đến 80% công suất định mức. Tức khoảng 200 mA đến 320 mA.

Bỏ qua những linh kiện sử dụng dòng nhỏ khác, dòng điện tối đa của một kênh Relay sẽ là tổng dòng của cuộn dây Relay, transistor khi kích, và LED báo hiệu (Nếu chúng được cấp nguồn từ 5V cách ly) như công thức dưới đây:

$$I_{\text{module_RL}} = I_{\text{coil}} + I_{\text{base}} + I_{\text{LED_tổng}} \quad (3-8)$$

Trong đó:

1: I_{coil} là dòng điện khi kích của Relay, với $I_{\text{coil}} = 70 \text{ mA}$ (theo datasheet)

2: I_{base} là dòng điện khi kích của transistor, với $I_{\text{base}} = 14 \text{ mA}$

3: $I_{\text{LED_tổng}}$ là dòng định mức (hoạt động) thường nằm trong khoảng 10 mA

với 2 LED báo hiệu cho mỗi kênh Relay là 20 mA. Như vậy

$$\begin{aligned} I_{\text{module_RL}} &= 70 \text{ mA} + 14 \text{ mA} + 20 \text{ mA} \\ &= 99.15 \text{ mA} \approx 100 \text{ mA} \end{aligned} \quad (3-9)$$

Tổng kết, với khuyến nghị về phạm vi tải lý tưởng cho một nguồn cách ly B0505S 2W có thể sử dụng tối đa cho 3 kênh Relay.

3.5.7 Số lượng cảm biến và thiết bị chấp hành cho từng Node

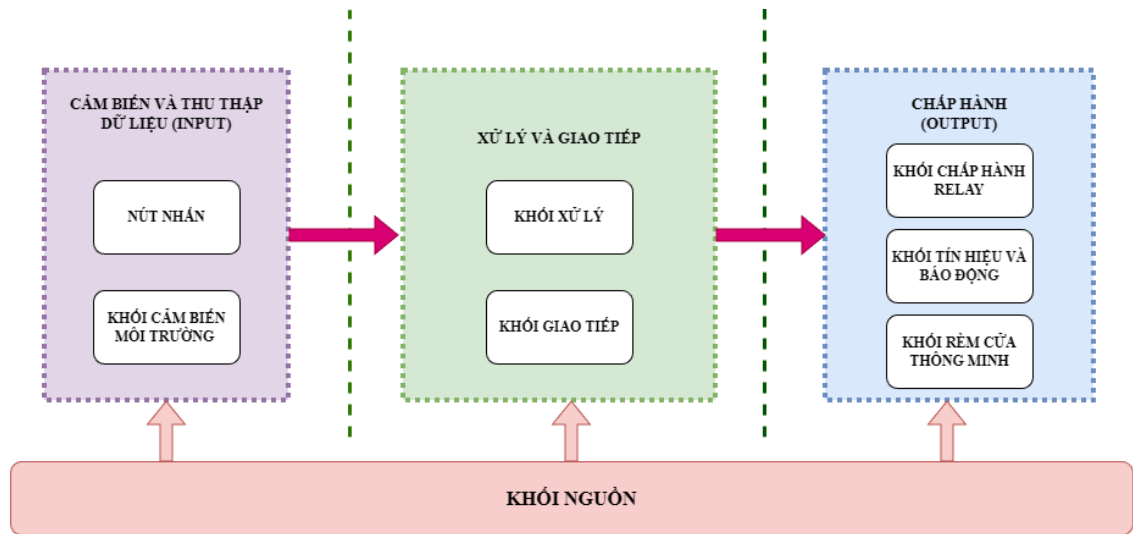
Với các chức năng khác nhau tại các phòng, số lượng các cảm biến và chấp hành cũng có sự thay đổi tương ứng với các phòng đó, dưới đây là bảng liệt kê số lượng các thiết bị cho các Node khác nhau

Bảng 3. 5 Số lượng cảm biến và thiết bị chấp hành cho từng Node

Thành phần	Node Phòng Khách	Node Phòng Bếp	Node Phòng Ngủ
Vi điều khiển	ESP32-S3	ESP32-S3	ESP32-S3
Cảm biến			
DHT22 (Nhiệt độ & Độ ẩm)	1	1	1
INMP441 (Âm thanh)	1	1	1
MQ2 (Khí Gas)	-	1	-
BH1750 (Ánh sáng)	-	-	1
Radar (Hiện diện)	-	-	1
Thiết bị chấp hành			
Relay (Kênh)	5	5	5
Cơ cấu Rèm cửa	1	-	1
Đèn LED RGB	1	1	1
Còi Buzzer	1	1	1
Tổng số linh kiện/Node	10	10	12

3.5.8 Sơ đồ khối phần cứng cho Node (Hardware Block Diagram) tổng thể

Sơ đồ khối phần cứng được xây dựng nhằm trực quan hóa kiến trúc phân tầng đã được lựa chọn, thể hiện rõ mối liên hệ logic và các giao tiếp vật lý giữa các khối chức năng chính. Sơ đồ này tuân thủ các yêu cầu về chức năng cần có, quản lý nguồn và cách ly chấp hành, đồng thời làm nổi bật vai trò của IoT Node (ESP32) trong hệ thống. Việc mô hình hóa này là nền tảng cho giai đoạn thiết kế mạch nguyên lý chi tiết tiếp theo.

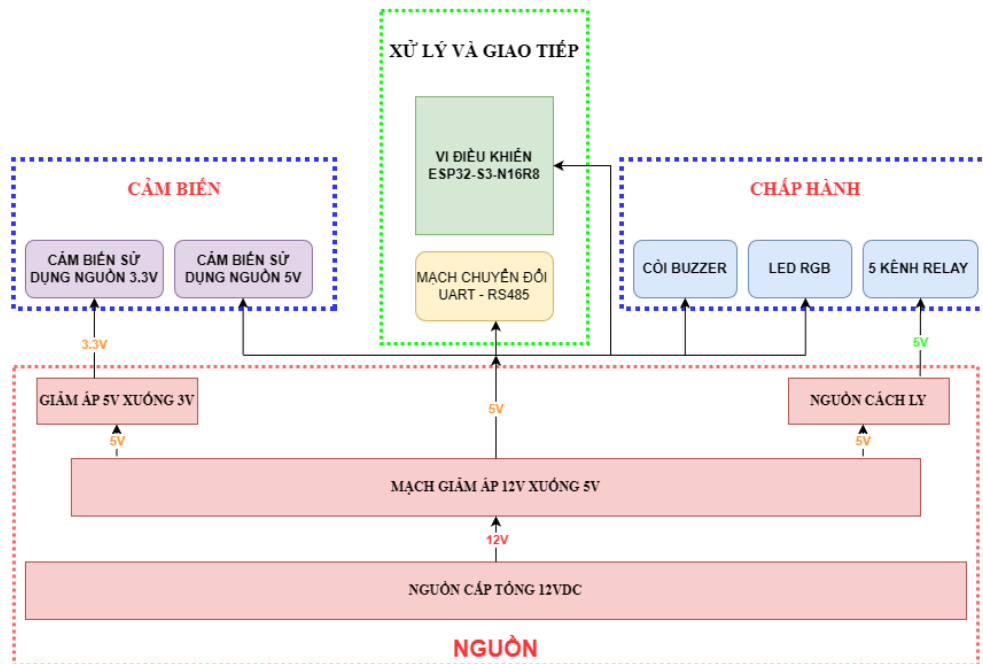


Hình 3. 2 Sơ đồ khối phần cứng cho Node tổng thể

Trong đó:

- Cảm biến và thu thập dữ liệu (Input) là nút nhấn và các cảm biến đo các giá trị vật lý từ môi trường như nhiệt độ, độ ẩm, ánh sáng, khí dễ cháy, âm thanh
- Xử lý và giao tiếp là Vi điều khiển chịu trách nhiệm xử lý và suy luận cùng với module giao tiếp ngoại vi
- Chấp hành (Output) là Relay đóng ngắt điều khiển tải AC 220V tuân thủ các yêu cầu về cách ly nguồn và cách ly tín hiệu; Khối tín hiệu và bảo động giúp người dùng trực quan hơn; Khối rèm cửa thông minh chịu trách nhiệm riêng biệt đóng và mở rèm.
- Khối Nguồn chịu trách nhiệm cấp nguồn cho toàn bộ mạch.

3.5.9 Sơ đồ nguồn cấp cho Node tổng thể

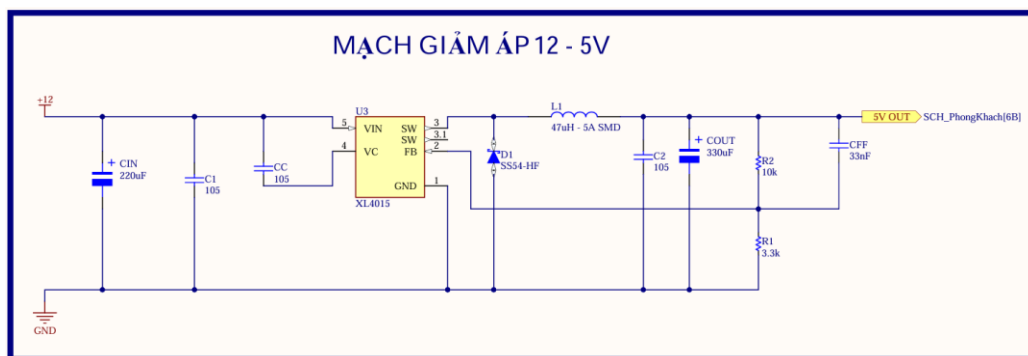


Hình 3. 3 Sơ đồ nguồn cấp cho Node tổng thể

Sơ đồ khối quản lý nguồn được xây dựng để trực quan hóa luồng năng lượng từ nguồn đầu vào 12V đến các tầng ổn áp khác nhau, đồng thời thể hiện cơ chế cách ly điện cho các module chấp hành. Cấu trúc này nhằm đảm bảo rằng mỗi linh kiện, đặc biệt là ESP32 nhạy cảm, đều nhận được điện áp chính xác và ổn định, đồng thời triệt tiêu nhiễu từ tải AC 220V.

3.5.10 Sơ đồ nguyên lý (Schematic Diagram) cho Node

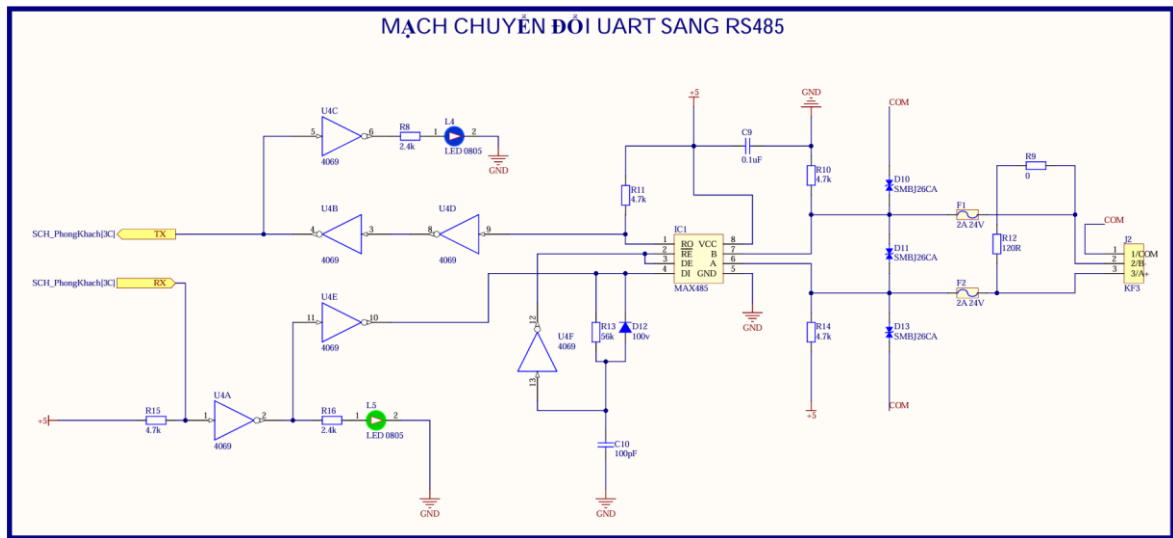
3.5.10.1 Sơ đồ nguyên lý mạch nguồn giảm áp 12V xuống 5V sử dụng IC XL4015



Hình 3. 4 Sơ đồ nguyên lý mạch nguồn giảm áp

Mạch sử dụng IC cách ly quang PC817 làm tầng đệm cách ly giữa ESP32 và Relay. PC817 truyền tín hiệu điều khiển bằng ánh sáng, giúp ngăn chặn hoàn toàn nhiễu điện từ và các xung điện áp cao từ cuộn dây Relay dội ngược về vi điều khiển. Giải pháp này đảm bảo an toàn tuyệt đối cho các chân I/O và duy trì sự ổn định cho hệ thống khi đóng ngắt tải AC công suất lớn. Chi tiết tính toán các linh kiện được thể hiện tại *Phụ lục A – Phần cứng, mục Tính toán các giá trị cho Relay*

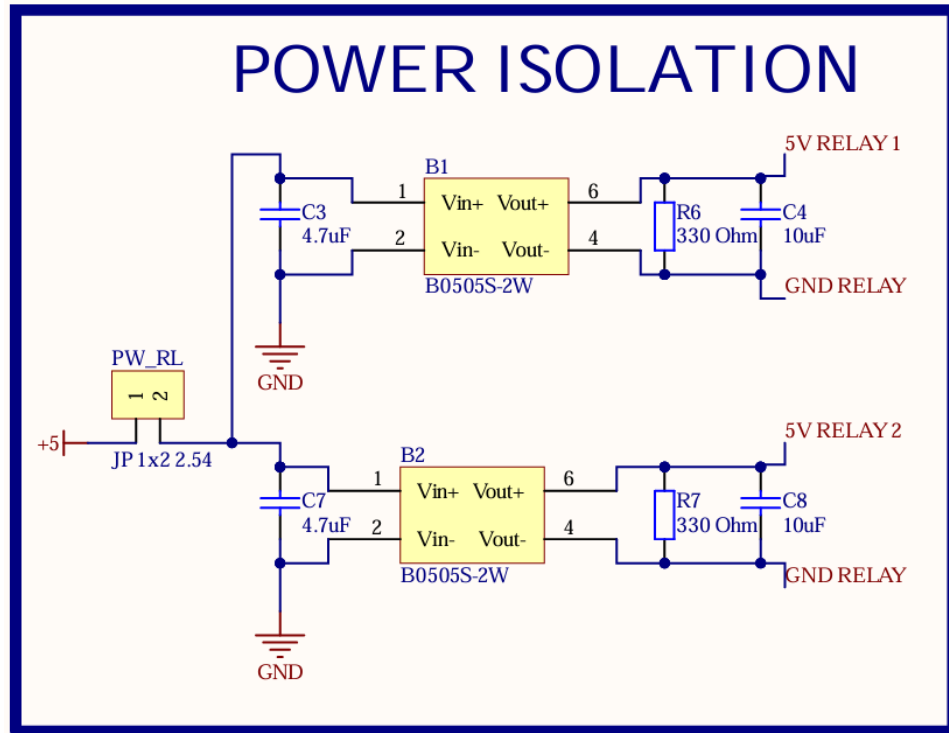
3.5.10.3 Sơ đồ nguyên lý mạch chuyển đổi UART sang RS485



Hình 3. 6 Sơ đồ nguyên lý mạch chuyển đổi UART sang RS485

Mạch chuyển đổi UART-to-RS485 (Auto Direction): Hệ thống sử dụng module chuyển đổi dựa trên IC MAX485 hoạt động ở chế độ bán song công (Half-Duplex). Điểm nổi bật của mạch là cơ chế điều khiển hướng tự động thông qua các cổng đảo logic của IC 4069, giúp chuyển đổi linh hoạt giữa chế độ truyền (DE) và nhận $\overline{RE}$ dựa trên trạng thái tín hiệu TX/RX mà không cần chân điều phối từ vi điều khiển.

3.5.10.4 Sơ đồ nguyên lý mạch nguồn cách ly cho Relay sử dụng B0505S 2W



Hình 3. 7 Sơ đồ nguyên lý mạch nguồn cách ly cho Relay sử dụng B0505S 2W

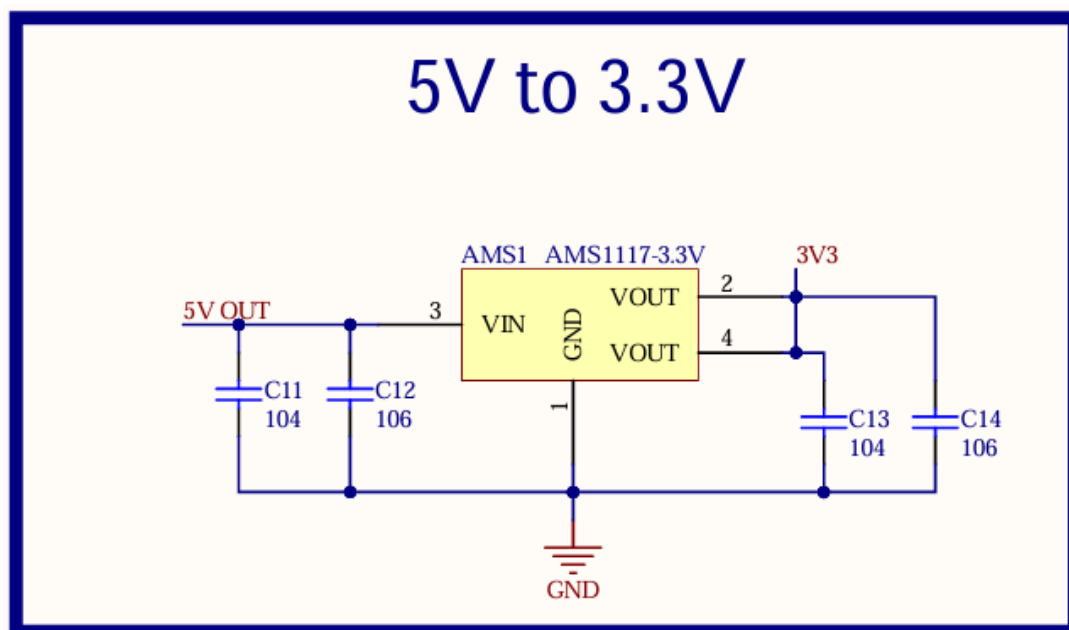
Với khuyến nghị từ nhà sản xuất về việc sử dụng công suất tải đầu ra không thấp hơn 10% so với toàn bộ tải có thể chịu được của nguồn cách ly, dòng điện tối thiểu luôn phải đạt được là 40 mA. Điện trở $R = 330 \text{ Ohm}$ được chọn mắc song song với đầu ra tải cố định của nguồn cách ly, kết hợp với 3 đèn LED báo hiệu nguồn của 3 Relay nhằm đảm bảo đầu ra luôn hơn 40 mA. Được tính toán như sau:

$$\text{Gọi dòng điện qua điện trở tải cố định là } I_{dummy} = \frac{V_{out}}{R_{dummy}} = \frac{5 \text{ V}}{330 \text{ Ohm}} \approx 15.15 \text{ mA}$$

Gọi dòng điện qua 3 LED báo hiệu nguồn của 3 kênh Relay là $I_{LED_total} \approx 30 \text{ mA}$ (Với mỗi LED là $\approx 10 \text{ mA}$). Vậy tổng dòng điện tối thiểu qua tải thực tế:

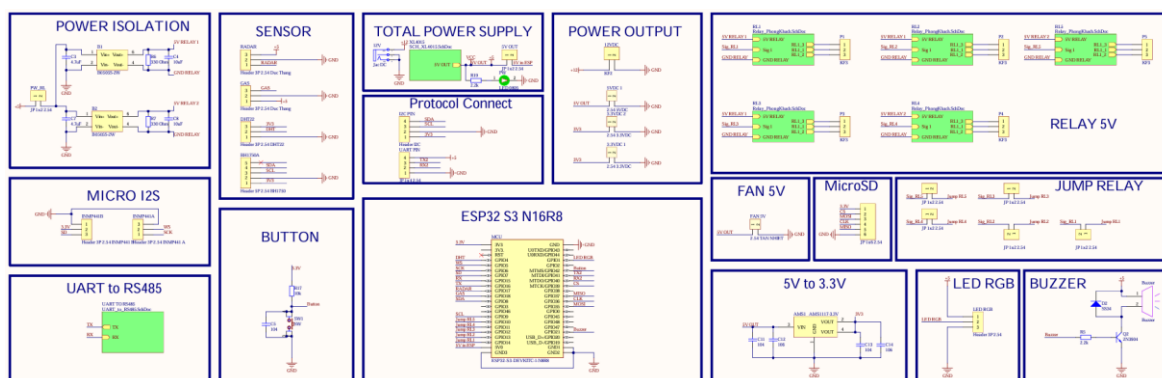
$$I_{actual} = I_{dummy} + I_{LED_total} = 15.15 \text{ mA} + 30 \text{ mA} = 45.15 \text{ mA}$$

3.5.10.5 Sơ đồ nguyên lý mạch giảm áp 5 V xuống 3.3 V sử dụng IC AMS1117



Hình 3. 8 Sơ đồ nguyên lý mạch giảm áp 5 V xuống 3.3 V

3.5.10.6 Sơ đồ nguyên lý tổng hợp



Hình 3. 9 Sơ đồ nguyên lý tổng hợp cho Node

3.5.11 Thiết kế Bố cục Mạch In (PCB Layout)

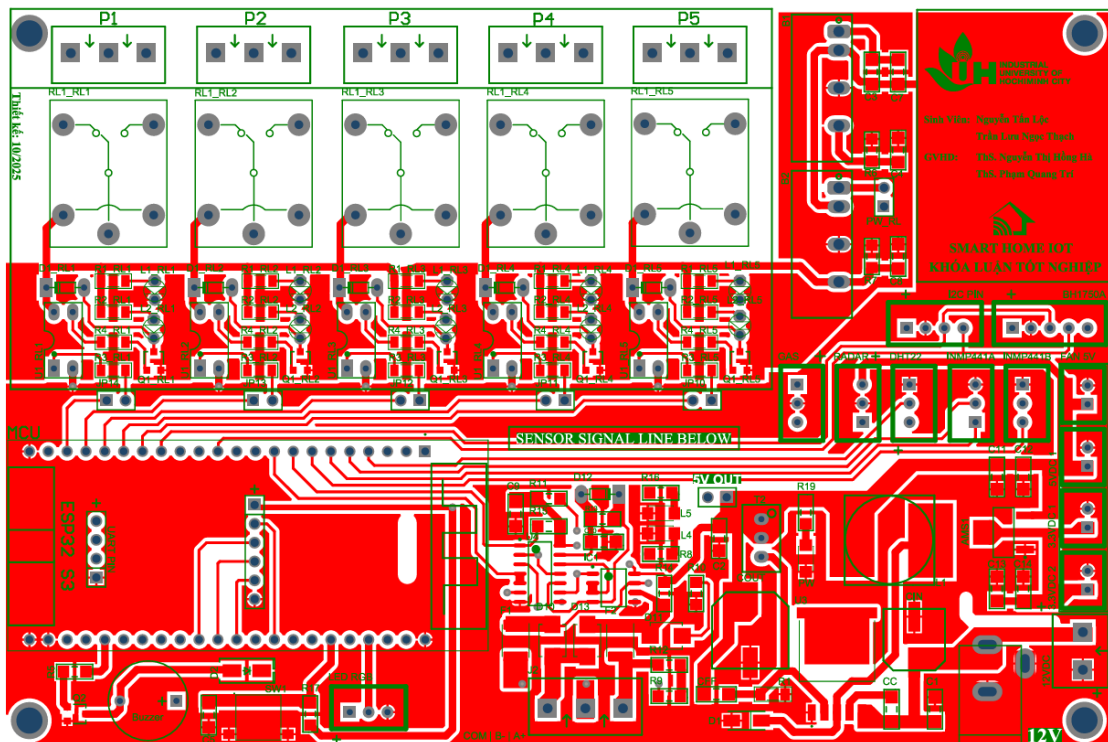
3.5.11.1 Quy tắc Thiết kế (Design Rules)

Sau khi hoàn tất sơ đồ nguyên lý và phân bố chân chi tiết, giai đoạn tiếp theo là thiết kế PCB. Mục tiêu của giai đoạn này không chỉ là kết nối các linh kiện mà còn là tối ưu hóa hiệu suất, đảm bảo tính toàn vẹn tín hiệu (Signal Integrity) và giảm thiểu nhiễu điện từ (EMI) để mạch hoạt động ổn định trong môi trường thực tế.

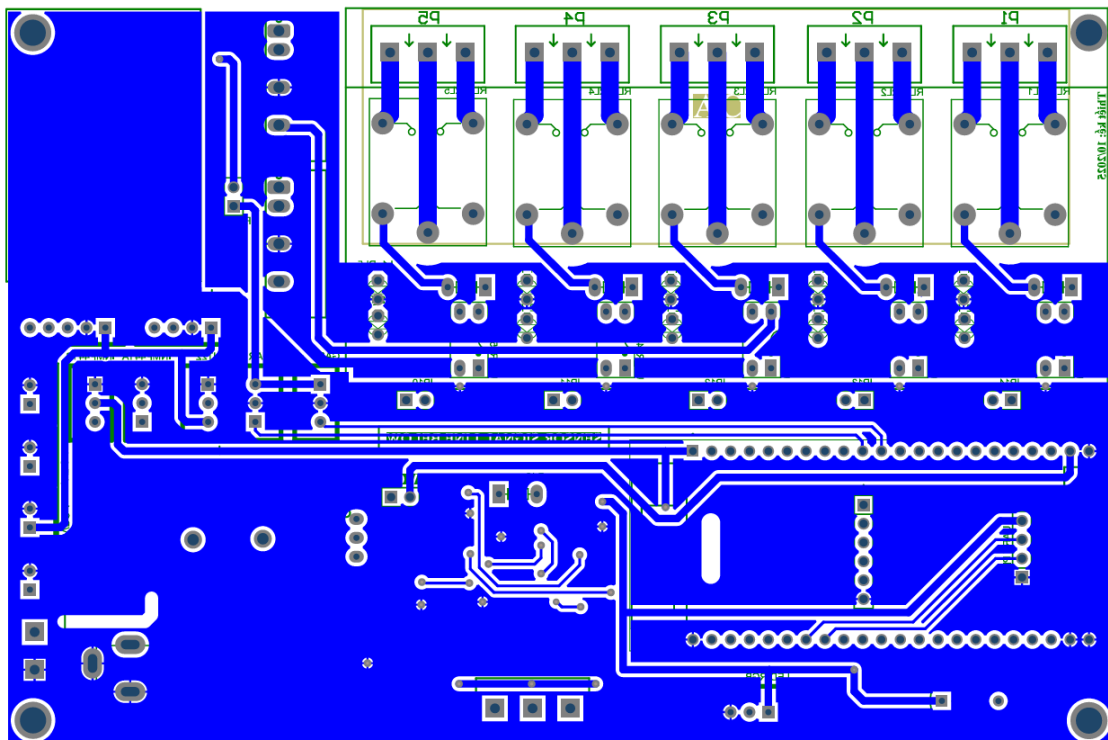
Bảng 3. 6 Quy tắc thiết kế mạch in PCB cho Node

STT	Quy tắc	Nội dung
1	Kích thước mạch, số lớp	Phạm vi thiết kế mạch không vượt quá $15\text{ cm} \times 10\text{ cm}$. Thiết kế mạch 2 lớp.
2	Đảm bảo cách ly điện áp cao	Không phủ đồng khu vực Relay đóng ngắt.
3	Đường mạch chịu tải tốt	Với các đường mạch nguồn 12V, 5V và 3.3V bề rộng tối thiểu 40 mil. Với các đường mạch điện AC bề rộng tối thiểu 95 mil. Với các đường mạch tín hiệu có dòng truyền tải thấp bề rộng tối thiểu 15 mil.
3	Phân chia bố cục chức năng rõ ràng	Các mạch chức năng được phân chia tối ưu bố cục các vùng trong mạch.
4	Có ký hiệu chú thích	Chú thích khu vực giắc cắm các cảm biến, khu vực cấp nguồn, chiều dương của nguồn cấp.

3.5.12 Hình ảnh Bố cục Mạch In Hoàn chỉnh



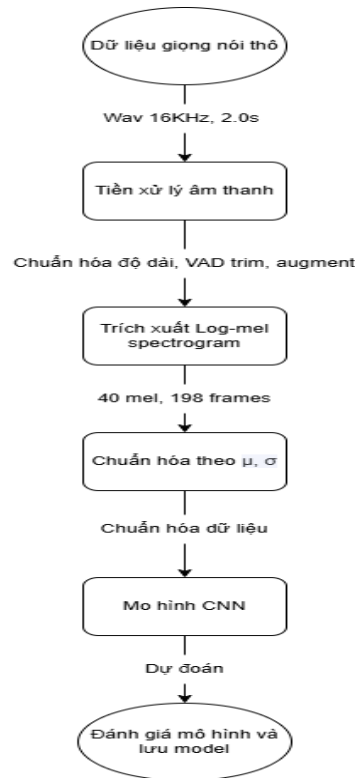
Hình 3. 10 Sơ đồ mạch PCB lớp trên



Hình 3. 11 Sơ đồ mạch in PCB lớp dưới

3.6 Phân tích và thiết kế thuật toán nhận diện giọng nói

3.6.1 Quá trình huấn luyện mô hình



Hình 3. 12 Sơ đồ khối xử lý tín hiệu giọng nói phục vụ huấn luyện mô hình

Sơ đồ biểu diễn chuỗi các bước xử lý từ lúc còn là dữ liệu giọng nói thô cho đến khi đánh giá và lưu model CNN đã huấn luyện. Toàn bộ quá trình này được thực hiện trên máy tính (offline) trước khi mô hình được triển khai trên ESP32-S3.

3.6.2 Triển khai chi tiết theo sơ đồ khối

3.6.2.1 Chuẩn bị và tổ chức dữ liệu đầu vào

- Xác định tập lệnh cần nhận dạng gồm các câu lệnh điều khiển nhà thông minh (bật/tắt đèn, bật/tắt quạt, hey esp,...) và các lớp đặc biệt như noise (nhiều môi trường) và unkonwn (câu nói bất kỳ không thuộc lệnh).
- Thu thập dữ liệu nói trực tiếp từ người dùng trên phần cứng ESP32-S3 kết hợp với micro INMP441, định dạng WAV, 16kHz, mono, độ dài khoảng 1.5-2s cho mỗi mẫu.

- Tổ chức dữ liệu thành các thư mục tương ứng từng nhãn, đặt tên file tuân tự để dễ dàng xử lý và quản lý tự động trong các đoạn mã Python.

3.6.2.2 *Tiền xử lý và chuẩn hóa dữ liệu âm thanh*

- Chuẩn hóa định dạng tất cả các file về: 16kHz, mono, độ sâu 16 bit. Các file sai chuẩn hoặc bị lỗi được loại bỏ.
- Chuẩn hóa độ dài tín hiệu: Cắt bỏ hoặc bổ sung vùng im lặng để mỗi mẫu có cùng số lượng mẫu (tương ứng với thời lượng thiết kế, $2s = 32.000$ mẫu). Những mẫu chênh lệch quá nhiều hoặc gần như im lặng hoàn toàn sẽ bị loại.
- Áp dụng loại bỏ im lặng (VAD hoặc ngưỡng năng lượng RMS) ở đầu và cuối câu lệnh để lệnh để phần tiếng nói nằm gọn trong cửa sổ thời gian
- Chuẩn hóa biên độ (đưa biên độ về dải chuẩn $[-1;1]$) nhằm giảm ảnh hưởng của mức năng lượng khác nhau giữa các lần thu.

3.6.2.3 *Tăng cường dữ liệu*

Để cải thiện khả năng tổng quát hóa và bù lại việc số lượng người nói còn hạn chế, dữ liệu được tăng cường bằng các phép biến đổi trên tín hiệu:

- Thêm nhiễu Gaussian hoặc trộn nhẹ với mẫu nhiễu môi trường đã thu.
- Thay đổi tốc độ nói (time-stretch) trong một biên độ nhỏ để mô phỏng việc nói nhanh/ chậm khác nhau.
- Thay đổi cao độ (pitch shift) một lượng nhỏ để mô phỏng khác biệt nam/nữ.

Sau khi tăng cường, số lượng mẫu mỗi lớp được cân bằng hơn, giúp mô hình tránh việc thiên lệch và các lớp có nhiều dữ liệu.

3.6.2.4 *Trích xuất đặc trưng log-Mel spectrogram*

- Phân khung (Framing): Mỗi đoạn tín hiệu sau tiền xử lý được chia thành các khung ngắn với độ dài cửa sổ cửa sổ 30 ms, bước trượt 10 ms)
- Biến đổi Fourier ngắn hạn (STFT): Thực hiện STFT trên từng khung để chuyển tín hiệu từ miền thời gian sang miền tần số, thu được phổ biên độ (Magnitude Spectrum).

- Trích xuất Log-Mel Spectrogram: Năng lượng trên mỗi dải Mel được tính toán và lấy Logarithm để tạo thành Log-Mel Spectrogram. Đây là ma trận hai chiều (Thời gian x tần số Mel) đóng vai trò là dữ liệu đầu vào cho mô hình CNN.
- Định dạng đầu vào: Kết quả cuối cùng là mỗi mẫu âm thanh được biểu diễn bằng một ma trận kích thước cố định ($F \times 40$), với F là số khung thời gian và 40 là số bộ lọc Mel (Mel filters).

3.6.2.5 Chuẩn hóa đặc trưng theo thống kê của toàn bộ tập dữ liệu

Từ toàn bộ tập log-Mel spectrogram, tính toán trung bình và độ lệch chuẩn cho từng dải Mel trên toàn bộ dữ liệu huấn luyện.

Mọi ma trận log-Mel được chuẩn hóa theo công thức:

$$x_{norm} = \frac{x - \mu}{\sigma} \quad (3-13)$$

Trong đó: μ và σ là các giá trị thống kê đã tính trước cho từng dải Mel.

Hai vector μ và σ này được lưu lại để sử dụng lại trong quá trình suy luận trên ESP32-S3, đảm bảo đặc trưng ở pha huấn luyện và pha triển khai có cùng phân bố.

3.6.2.6 Cấu trúc và cấu hình huấn luyện mô hình CNN

a) Cấu trúc mô hình

- Đầu vào: Ma trận log-Mel chuẩn hóa kích thước $F \times 40$, được coi như ảnh 2D có một kênh.
- Các tầng tích chập 2D với số lượng kênh tăng dần ($16 \rightarrow 32 \rightarrow 48$), kết hợp với Batch Normalization và MaxPooling để trích xuất và cô đọng đặc trưng theo cả trục thời gian và trục tần số.
- Tầng Global Average Pooling nén bản đồ đặc trưng thành một vector, sau đó là một vài lớp Dense sử dụng Softmax với số nút bằng số lệnh.
- Kiến trúc thiết kế sao cho sau khi lượng tử hóa vẫn đáp ứng được giới hạn bộ nhớ và tốc độ suy luận trên ESP32-S3.

b) Cấu hình huấn luyện

- Hàm mất mát: `sparse_categorical_crossentropy`.
- Bộ tối ưu: Adam với tốc độ học ban đầu phù hợp ($1e-3$).
- Dữ liệu được chia thành các tập `train/validation/test`, có thể áp dụng `last_weight` để xử lý mất cân bằng giữa các lớp.
- Sử dụng `EarlyStopping` theo `val_accuracy` với `patience` thích hợp, đồng thời lưu lại mô hình có độ chính xác trên tập `validation` cao nhất.

3.6.2.7 Lưu mô hình đã huấn luyện và tham số phục vụ triển khai

Sau khi huấn luyện hoàn tất, mô hình đạt chất lượng tốt nhất trên tập kiểm định được lưu lại dưới dạng tệp (`.h5` hoặc `SavedModel`).

Mô hình được chuyển đổi sang định dạng TensorFlow Lite, tiến hành lượng tử hóa về kiểu số nguyên 8 bit (`int8`) để giảm kích thước và phù hợp với tài nguyên của vi điều khiển.

Đồng thời, các tham số chuẩn hóa (vector μ , σ) và thông tin cấu hình trích xuất đặc trưng (kích thước log-Mel, số khung, số dải Mel,...) được xuất ra dưới dạng mảng C để nhúng thẳng vào mã nguồn firmware ESP32-S3.

Tập mô hình TFLite và các thông số đi kèm này là đầu vào cho khối “Quá trình xử lý giọng nói dựa trên mô hình huấn luyện” ở các mục sau.

3.6.3 Quá trình đánh giá kết quả mô hình

3.6.3.1 Mục tiêu và kịch bản đánh giá

Sau khi hoàn tất quá trình huấn luyện, mô hình nhận dạng giọng nói cần đánh giá trên một tập dữ liệu độc lập để kiểm chứng khả năng tổng quát và độ tin cậy trước khi triển khai thực tế trên ESP32-S3.

Trong đề tài, tập dữ liệu ban đầu được chia thành:

- Tập huấn luyện (training set): Dùng để cập nhật trọng số mô hình CNN trong quá trình học.

- Tập kiểm tra (validation/test set): Gồm 8636 mẫu, không tham gia quá trình cập nhật trọng số, chỉ dùng để đánh giá hiệu năng cuối cùng của mô hình.

Tập kiểm tra bao gồm đầy đủ các lớp lệnh mà hệ thống cần nhận dạng:

Từ khóa đánh thức: *hey_esp*.

Các lệnh điều khiển phòng khách:

- *bat_den_phong_khach, tat_den_phong_khach,*
- *bat_quat_phong_khach, tat_quat_phong_khach,*
- *dong_rem_phong_khach, mo_rem_phong_khach.*

Các lệnh điều khiển phòng ngủ:

- *bat_den_phong_khach, tat_den_phong_ngu,*
- *bat_quat_phong_khach, tat_quat_phong_ngu,*
- *dong_rem_phong_khach, mo_rem_phong_ngu.*

Các lệnh điều khiển phòng bếp:

- *bat_den_phong_khach, tat_den_phong_bep,*
- *bat_quat_phong_khach, tat_quat_phong_bep.*

Mỗi lớp khoảng 480 – 650 mẫu, đảm bảo tập kiểm tra tương đối cân bằng, không thiên lệch về bất kì lệnh nào.

Quá trình đánh giá gồm:

- Tính các chỉ số định lượng (accuracy, precision, recall, F1-score) trên toàn bộ tập kiểm tra.
- Phân tích ma trận nhầm lẫn để xem mô hình hay nhầm lẫn giữa những lệnh nào.
- Minh họa trực quan đặc trưng log-Mel 2D của một số câu lệnh đối ngược (bật/tắt đèn) để làm rõ cách mô hình phân biệt chúng trên miền đặc trưng.

3.6.3.2 Các chỉ số đánh giá sử dụng

Trong quá trình đánh giá, các chỉ số sau được sử dụng:

- Độ chính xác (Accuracy): Tỷ lệ dự đoán đúng trên toàn bộ tập kiểm tra.

$$Accuracy = \frac{\text{số mẫu dự đoán đúng}}{\text{tổng số mẫu}} \quad (3-14)$$

- Precision của từng lớp: Đánh giá khả năng tránh kích hoạt nhầm lệnh (False Positive).
- Recall của từng lớp: Đánh giá khả năng tránh bỏ sót lệnh của người dùng (False Negative).
- F1-score: Trung bình điều hòa giữa precision và recall, phản ánh đồng thời khả năng dự đoán đúng và mức độ không bỏ sót.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (3-15)$$

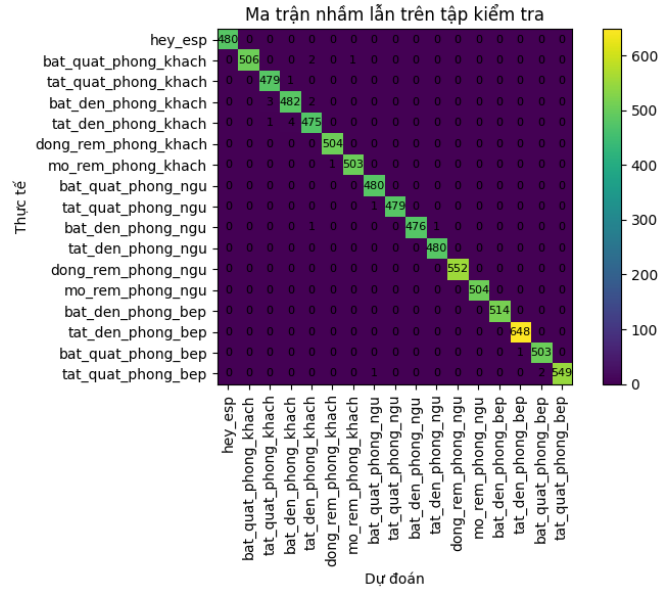
- Macro & Weighted Average: Đánh giá hiệu năng trung bình của mô hình trên tất cả các lớp lệnh, đảm bảo tính cân bằng và khách quan.
- Confusion Matrix (Ma trận nhầm lẫn): Trực quan hóa chi tiết các cặp lệnh thường bị hệ thống nhận diện sai, từ đó đưa ra hướng tối ưu hóa cho từng lớp. Chi tiết các chỉ số được thể hiện như hình dưới đây:

Độ chính xác trên tập kiểm tra (val/test): 0.9975				
	precision	recall	f1-score	support
hey_esp	1.0000	1.0000	1.0000	480
bat_quat_phong_khach	1.0000	0.9941	0.9970	509
tat_quat_phong_khach	0.9917	0.9979	0.9948	480
bat_den_phong_khach	0.9897	0.9897	0.9897	487
tat_den_phong_khach	0.9896	0.9896	0.9896	480
dong_rem_phong_khach	0.9980	1.0000	0.9990	504
mo_rem_phong_khach	0.9980	0.9980	0.9980	504
bat_quat_phong_ngu	0.9959	1.0000	0.9979	480
tat_quat_phong_ngu	1.0000	0.9979	0.9990	480
bat_den_phong_ngu	1.0000	0.9958	0.9979	478
tat_den_phong_ngu	0.9979	1.0000	0.9990	480
dong_rem_phong_ngu	1.0000	1.0000	1.0000	552
mo_rem_phong_ngu	1.0000	1.0000	1.0000	504
bat_den_phong_bep	1.0000	1.0000	1.0000	514
tat_den_phong_bep	0.9985	1.0000	0.9992	648
bat_quat_phong_bep	0.9960	0.9980	0.9970	504
tat_quat_phong_bep	1.0000	0.9946	0.9973	552
accuracy			0.9975	8636
macro avg	0.9974	0.9974	0.9974	8636
weighted avg	0.9975	0.9975	0.9975	8636

Hình 3. 13 Kết quả đánh giá mô hình

3.6.3.3 Phân tích ma trận nhầm lẫn

Để quan sát chi tiết hơn các lỗi phân loại, mô hình được biểu diễn dưới dạng ma trận nhầm lẫn.



Hình 3. 14 Ma trận nhầm lẫn trên tập kiểm tra

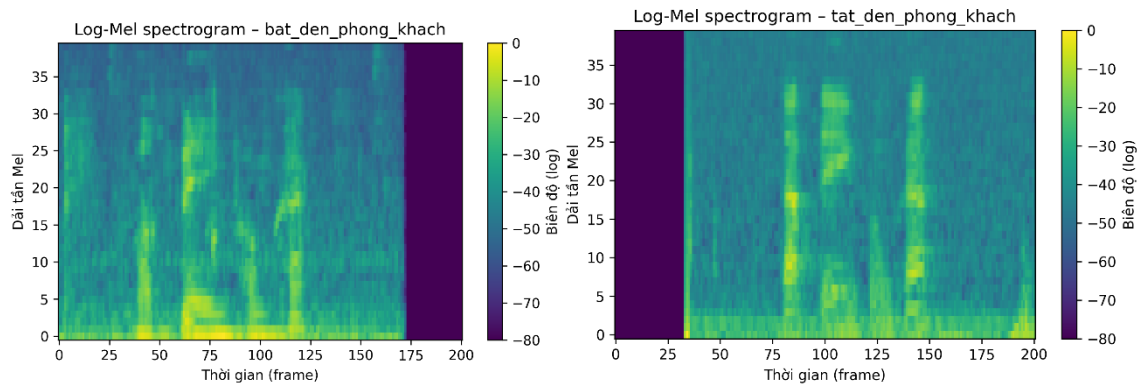
Độ chính xác cục bộ: Các giá trị tập trung chủ yếu trên đường chéo chính với tỉ lệ áp đảo, chứng minh mô hình phân loại chính xác hầu hết các tập lệnh. Nhiều lớp như hey_esp, bat_den_phong_khach đạt tỉ lệ nhận diện đúng tuyệt đối.

Sai số đặc trưng: Một số ít sai sót (từ 1–4 mẫu) chỉ xảy ra giữa các cặp lệnh có ngữ âm tương tự trong cùng bối cảnh, ví dụ như sự nhầm lẫn giữa tiền tố "bật" và "tắt" (Ví dụ: bat_quat_phong_khach và tat_quat_phong_khach).

Khả năng phân biệt bối cảnh: Hoàn toàn không xảy ra hiện tượng "nhầm lẫn chéo" giữa các khu vực khác nhau (phòng khách, phòng ngủ, phòng bếp). Điều này khẳng định mô hình đã học tốt các đặc trưng từ khóa chỉ định vị trí.

3.6.3.4 Minh họa log_Mel 2D của các câu lệnh

Để giải thích rõ hơn tại sao mô hình CNN có thể phân biệt được các lệnh giọng nói, minh họa log-Mel spectrogram (đặc trưng đầu vào của mô hình) cho hai câu lệnh đối ngược “bật đèn phòng khách” và “tắt đèn phòng khách”.



Hình 3. 15 Ảnh 2D log-Mel spectrogram

Bên trong vùng có tín hiệu, dễ dàng nhận thấy ba cụm năng lượng chính:

- Cụm đầu tiên tương ứng với âm tiết “bật”, “tắt”
- Cụm thứ hai tương ứng với “đèn”,
- Cụm thứ 3 kéo dài hơn tương ứng với cụm “phòng khách”.

Cấu trúc tương đồng: Cả hai lệnh đều có phần đuôi "đèn phòng khách" với cấu trúc năng lượng giống nhau, tập trung chủ yếu ở dải tần thấp và trung. Các vùng im lặng (Padding) được thể hiện bằng màu tím đậm để đảm bảo độ dài mẫu chuẩn 2 giây (200 frames).

Đặc trưng phân đoạn: Tín hiệu hiển thị rõ rệt các cụm năng lượng (vệt sáng dọc) tương ứng với từng âm tiết.

- Lệnh "*Bật*": Âm tiết /b/ là phụ âm hữu thanh, tạo ra năng lượng mạnh, tập trung và "mềm" hơn ở dải tần thấp.
- Lệnh "*Tắt*": Âm tiết /t/ là phụ âm vô thanh, năng lượng sắc nét, ngắn hơn và phân bố ở dải tần cao hơn.

Sự khác biệt về thời gian: Spectrogram phản ánh chính xác khoảng lặng (Latent period) trước khi phát âm. Lệnh "Tắt" trong mẫu thử nghiệm có khoảng lặng đầu (frame 0-30) rõ rệt hơn so với lệnh "Bật".

Như vậy, mặc dù phần cuối “đèn phòng khách” của 2 câu lệnh gần như trùng nhau về mặt phổ, mẫu năng lượng ở phần đầu lại khác nhau rõ rệt. Khi log-Mel được đưa

vào mô hình CNN, các lớp tích chập sẽ quét theo cả chiều thời gian và tần số để phát hiện những mẫu cục bộ đặc trưng này (cụm năng lượng tương ứng với “bật” hay “tắt”). Nhờ đó, mô hình có thể phân biệt chính xác hia lệnh đối ngược, dù chúng chỉ khác nhau mỗi từ đơn.

3.6.4 Quá trình xử lý giọng nói dựa vào mô hình đã huấn luyện

3.6.4.1 Mục tiêu của quá trình xử lý giọng nói

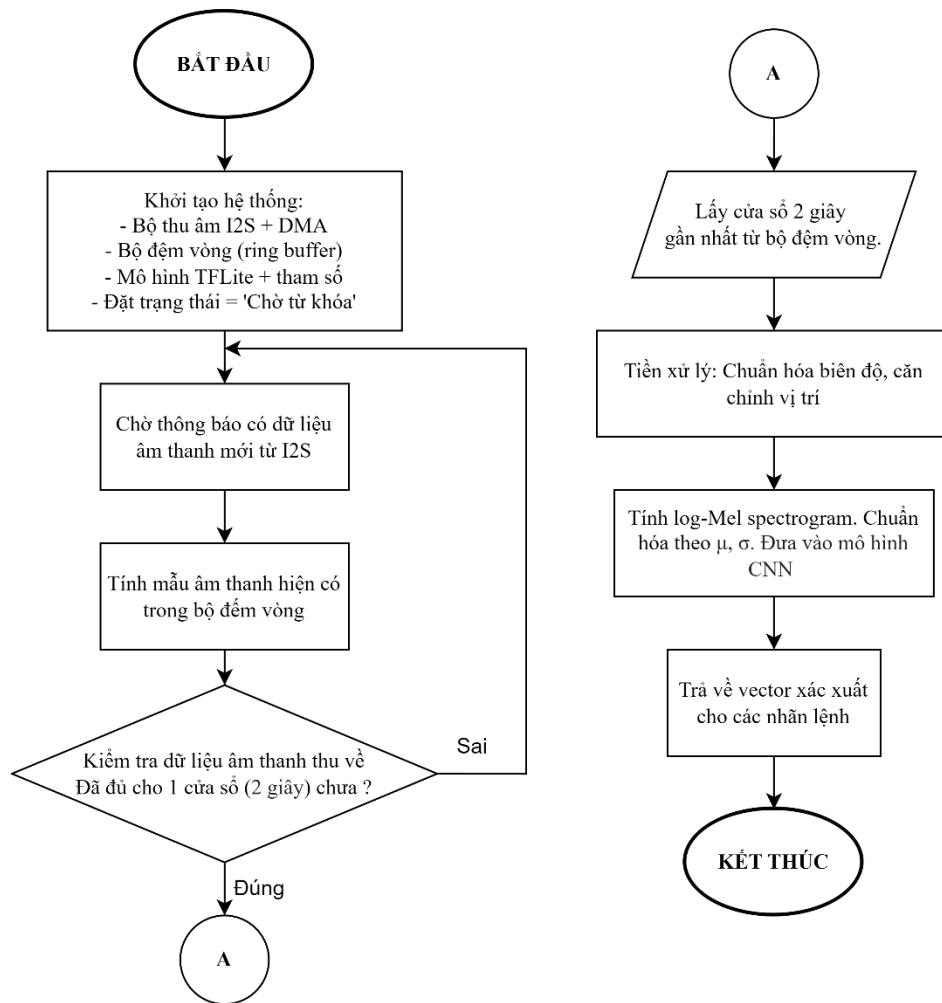
Sau khi mô hình nhận dạng giọng nói đã được huấn luyện và đánh giá, bước tiếp theo là tích hợp mô hình này vào vi điều khiển ESP32-S3 để xử lý tín hiệu trong thời gian thực. Quá trình xử lý giọng nói phải đáp ứng các yêu cầu sau:

- Lắng nghe liên tục từ micro INMP441, không bỏ lỡ từ khóa kích hoạt và câu lệnh của người dùng.
- Phát hiện chính xác từ khóa “hey esp” làm điều kiện kích hoạt hệ thống.
- Nhận dạng câu lệnh điều khiển (bật/tắt đèn, quạt, rèm ở từng khu vực) với độ tin cậy cao, hạn chế tối đa kích hoạt nhầm.
- Đảm bảo độ trễ thấp, phù hợp với thao tác điều khiển thời gian thực.
- Hoạt động được trên tài nguyên hạn chế của ESP32-S3 (Ram, Flash), đồng thời không chặn các tác vụ IoT khác (MQTT, đọc cảm biến, ...).

Để đạt được các mục tiêu trên, thuật toán trên ESP32-S3 được tổ chức thành 2 tầng chính:

- Thu thập và xử lý đặc trưng tiếng nói, đưa ra được cửa sổ xác xuất nhận diện các nhân
- Xử lý logic giọng nói: Trạng thái chờ từ khóa kích hoạt và trạng thái chờ lệnh từ người dùng

3.6.4.2 Thuật toán suy luận tìm xác suất nhận diện các nhãn



Hình 3. 16 Lưu đồ thuật toán suy luận tìm xác suất nhận diện các nhãn

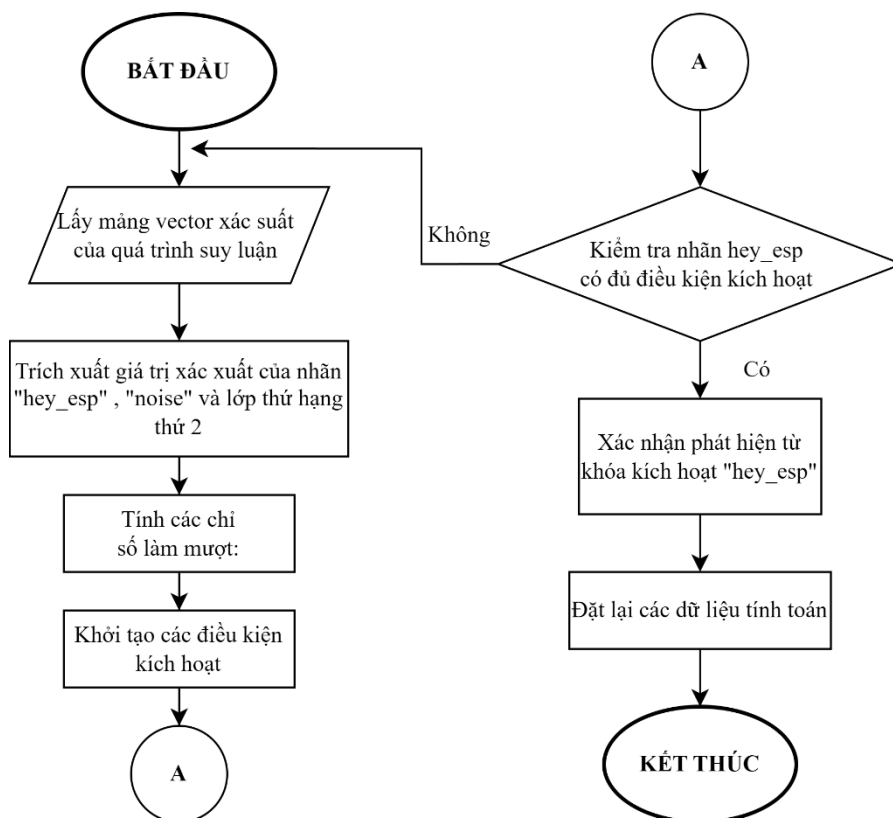
Một vòng lặp suy luận liên tục được thực hiện tuần tự và lặp lại trên các bước chính sau đây:

1. Chờ thông báo có dữ liệu mới từ I2S: task kws_task ngủ và chỉ được đánh thức khi DMA/I2S hoàn tất một buffer mới (thông qua cơ chế taskNotify). Điều này giúp tiết kiệm CPU.
2. Tính số mẫu hiện có trong bộ đệm vòng: từ chỉ số ghi toàn cục và chỉ số đọc của RingBufferReader, hệ số xác định được mẫu số khả dụng.
3. Kiểm tra đủ dữ liệu cho một cửa sổ phân tích:
 - Nếu số mẫu < kích thước cửa sổ (2 giây) vòng lặp quay lại chờ thêm dữ liệu.

- Nếu số mẫu $\geq$ kích thước cửa sổ $\rightarrow$ tiến hành bước tiếp theo.
4. Lấy cửa sổ 2 giây gần nhất: hệ thống trích xuất N mẫu gần nhất từ ring buffer, canh chỉnh vị đọc để tránh lệch pha do trễ DMA, chuẩn hóa biên độ và loại bỏ phần dư thừa.
 5. Tính đặc trưng log-Mel spectrogram và suy luận: cửa sổ âm thanh được chuyển sang miền đặc trưng giống giai đoạn huấn luyện (STFT $\rightarrow$ Mel filterbank 40 dải $\rightarrow$ log năng lượng). Ma trận Log-Mel sau đó được chuẩn hóa theo μ , σ và đưa vào mô hình CNN để thu được vector xác suất $p[i]$ cho toàn bộ các lớp (từ khóa, lệnh điều khiển, nhiễu,...).

Kết quả của bước này là một vector xác suất đại diện cho 2 giây âm thanh gần nhất, được dùng cho phần quyết định tiếp theo của máy trạng thái.

3.6.4.3 Thuật toán phát hiện từ khóa kích hoạt "hey_esp"



Hình 3. 17 Thuật toán phát hiện từ khóa kích hoạt

Quá trình phát hiện từ khóa kích hoạt được tiến hành với những hoạt động chính như sau:

Lấy xác suất từ khóa:

Trích giá trị $p_wake = p[hey_esp]$, (do điều kiện kích hoạt từ khóa chỉ cần phân biệt nhãn kích hoạt và nhãn noise, vì thế các nhãn khác coi như không xét đến), đồng thời lấy xác suất của lớp có thứ hạng thứ hai và “lớp noise” để tính biên độ (margin) và đánh giá mức nhiễu hiện tại của âm thanh môi trường.

Tính các chỉ số làm mượt:

- Cập nhật giá trị trung bình trượt EWMA của xác suất từ khóa.
- Cập nhật bộ đếm bộ phiếu (vote) số lần cửa sổ gần đây mà từ khóa là lớp có xác suất cao nhất.

Kiểm tra các điều kiện kích hoạt: Từ khóa chỉ được chấp nhận khi đồng thời thỏa nhiều điều kiện:

- Xác suất tức thời của từ khóa lớn hơn một ngưỡng tối thiểu.
- Biên độ so với lớp thứ hai đủ lớn, chứng tỏ mô hình phân biệt rõ.
- EWMA của xác suất từ khóa vượt ngưỡng, cho thấy từ khóa được duy trì ổn định trong nhiều cửa sổ liên tiếp.
- Mức nhiễu không được vượt quá số ngưỡng cho phép (không bị nhầm thành lớp noise).
- Khoảng thời gian từ lần kích hoạt trước đó lớn hơn thời gian cooldown, tránh trường hợp kích hoạt liên tiếp.

Quyết định kích hoạt:

Nếu không thỏa các điều kiện trên, hệ thống chỉ cập nhật thông tin làm mượt, trượt cửa sổ theo thời gian và quay lại vòng lặp.

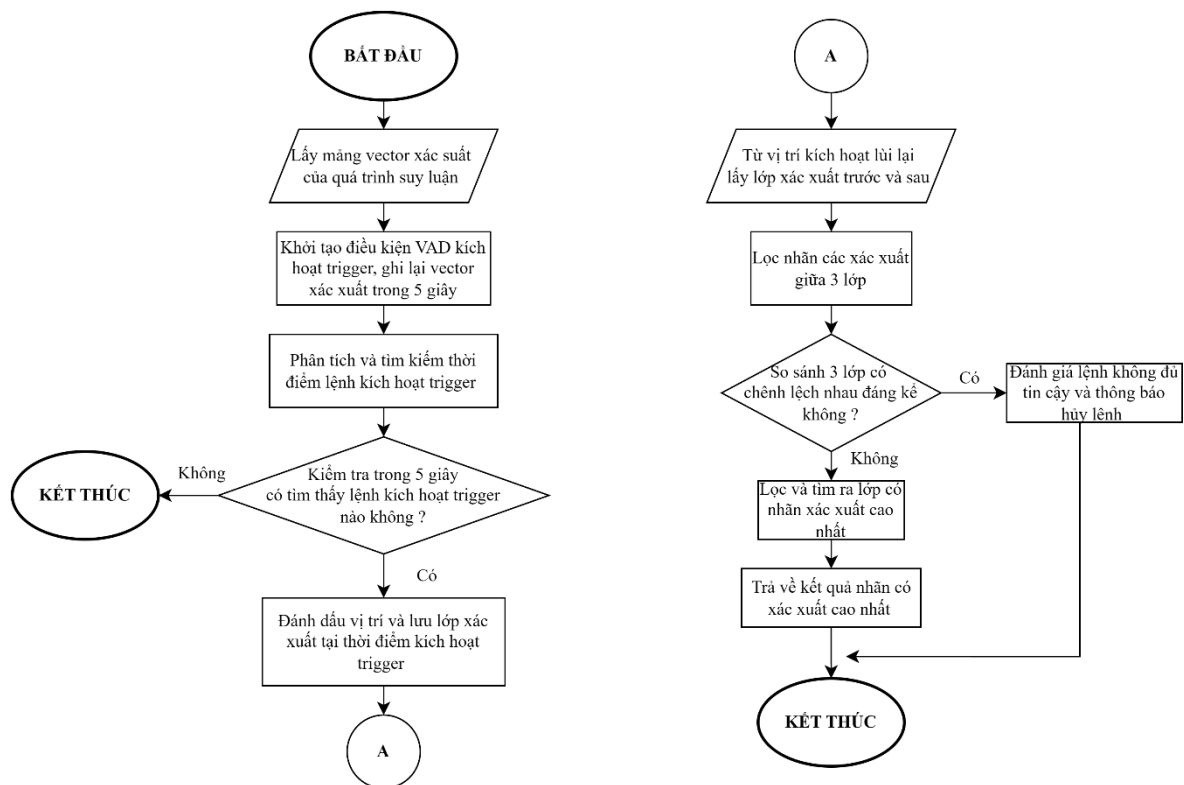
Nếu thỏa các điều kiện kích hoạt, hệ thống:

- Xác nhận đã nghe thấy từ khóa “hey esp”.
- Ghi lại thời điểm kích hoạt để phục vụ giới hạn thời gian nghe lệnh.

- Đặt trạng thái sang “Nhận lệnh”.
- Báo hiệu cho người dùng (LED hoặc âm báo ngắn) rằng hệ thống đã sẵn sàng nhận câu lệnh.

Sau khi lệnh kích hoạt được phát hiện, bước tiếp theo của quá trình nhận dạng giọng nói là nhận lệnh từ người dùng.

3.6.4.4 Thuật toán phát hiện lệnh điều khiển giọng nói từ người dùng



Hình 3. 18 Thuật toán phát hiện lệnh điều khiển giọng nói từ người dùng

Ngay khi chuyển sang trạng thái nhận lệnh điều khiển từ người dùng, thuật toán bắt đầu các bước:

1. Ghi tại thời điểm bắt đầu nghe lệnh để giới hạn thời gian chờ, khoảng 5 giây).

- Đặt cờ VAD_lệnh = False, nghĩa là chưa xác định được đoạn chứa câu lệnh.
- Xóa thông tin về lệnh tốt nhất (lớp, xác suất, mức nhất quán) của lần trước.

2. Phân tích vector xác suất và tìm ra thời điểm kích hoạt lệnh:

- Tìm lớp lệnh điều khiển có xác suất lớn nhất.

- So sánh xác suất này với một ngưỡng VAD để ước lượng xem đã xuất hiện tiếng nói lệnh hay chưa.

3. Xác định “có tiếng nói lệnh”, thuật toán bắt đầu kiểm tra các điều kiện và cho ra được nhãn có xác suất cao nhất, sau đó tiến hành đánh dấu vị trí kích hoạt VAD:

- Lưu vị trí (frame/mẫu) kích hoạt, lưu lớp và xác suất tại thời điểm này,
- Bật cờ VAD_lệnh = True, báo hiệu cho người dùng rằng hệ thống đang ghi nhận câu lệnh.

Nếu chưa thỏa điều kiện, hệ thống coi như hủy câu lệnh, chỉ trượt cửa sổ theo thời gian và chờ lần kích hoạt tiếp theo.

4. Sau khi xác định có lệnh, thuật toán bắt đầu phân tích để tìm ra lệnh chính xác:

Từ vị trí kích hoạt, thuật toán thu thập vector xác suất khoảng 0,5 – 1 giây (nếu còn dữ liệu) trước đó. Đồng thời cũng thu thập vector xác suất sau đó khoảng 0.5 – 1 giây nhằm bảo đảm bắt trọn đầu câu lệnh.

5. Sau khi thu thập được 3 vector xác suất: trước thời điểm kích hoạt, thời điểm kích hoạt, sau thời điểm kích hoạt. Thuật toán tiến hành so sánh sự chênh lệch xác suất của 3 vector có chênh lệch nhau đáng kể không (*Ví dụ*: 0.86 và 0.23).

Nếu có chênh lệch nhau đáng kể, rất có thể đây là lệnh sai thuật toán tiến hành hủy lệnh, và quay lại chờ lệnh kích hoạt lượt tiếp theo.

6. Nếu lệnh được xác nhận là không chênh lệch quá nhiều, thuật toán bắt đầu tiến hành lần lọc cuối cùng. Trích xuất ra nhãn có xác suất cao nhất trong 3 vector và tìm nhãn có xác suất cao nhất làm nhãn tin cậy.

7. Sau khi trả về quyết định nhãn có xác suất tốt nhất, thuật toán tiến hành xóa các dữ liệu và đặt các thông số điều kiện lại mặc định và kết thúc một vòng nhận diện.

3.7 Phân tích và thiết kế thuật toán điều phối IOT của hệ thống

Quá trình thiết kế thuật toán điều phối được tổ chức thành các nhóm chức năng tương đối độc lập. Mỗi nhóm đảm nhiệm một nhiệm vụ lõi trong hệ thống và các nhóm liên kết với nhau thông qua các luồng dữ liệu rõ ràng

3.7.1 Các nhóm chức năng trong thiết kế thuật toán điều phối cần có

Dựa trên các yêu cầu mà một hệ thống IOT cần có như: Đồng bộ thời gian thực, điều khiển đúng các thiết bị, dễ dàng mở rộng, tự động hóa với các trường hợp thực tế và điều phối lệnh điều khiển bằng giọng nói từ người dùng. Thuật toán chia làm 5 nhóm chính như sau:

Nhóm 1 – Hạ tầng giao tiếp MQTT (MQTT Infrastructure/Transport): Khởi tạo MQTT client, kết nối broker, subscribe topic, nhận bản tin, tiền xử lý và điều phối bản tin đến đúng nhóm nghiệp vụ; cung cấp hàm publish dùng chung để gửi lệnh xuống node.

Nhóm 2 – Cập nhật trạng thái & tình trạng online (State & Presence Update): Cập nhật trạng thái online/offline của node; cập nhật trạng thái actuator báo về (reportedValue); đồng thời có thể quản lý “tình trạng sống” của gateway thông qua heartbeat lên Firebase.

Nhóm 3 – Xử lý lệnh điều khiển thủ công (Manual Control): Nhận lệnh từ App (Firebase desiredValue/commandValue) và từ giọng nói (MQTT voice_command), chuẩn hoá/kiểm tra hợp lệ, ánh xạ đúng thiết bị đích, sau đó gửi lệnh điều khiển xuống node.

Nhóm 4 – Xử lý dữ liệu cảm biến thời gian thực & cảnh báo (Realtime Sensors & Alerts): Nhận dữ liệu cảm biến từ MQTT, cập nhật live state, so sánh ngưỡng để phát hiện cảnh báo, ghi lịch sử (notifications) và gửi FCM; đồng thời kích hoạt nhóm tự động hóa khi điều kiện thoả.

Nhóm 5 – Tự động hóa (Automation Engine + Mode Coordination): Lắng nghe cấu hình rule từ Firebase, đánh giá điều kiện trigger theo dữ liệu cảm biến hiện tại, thực

thi action bằng cách gửi lệnh xuống node; phối hợp chế độ AUTO/MANUAL theo Setting_Mode để cho phép hoặc vô hiệu hóa automation theo global và theo node.

3.7.2 Triển khai chi tiết từng nhóm – Nhóm 1: Hạ tầng giao tiếp MQTT

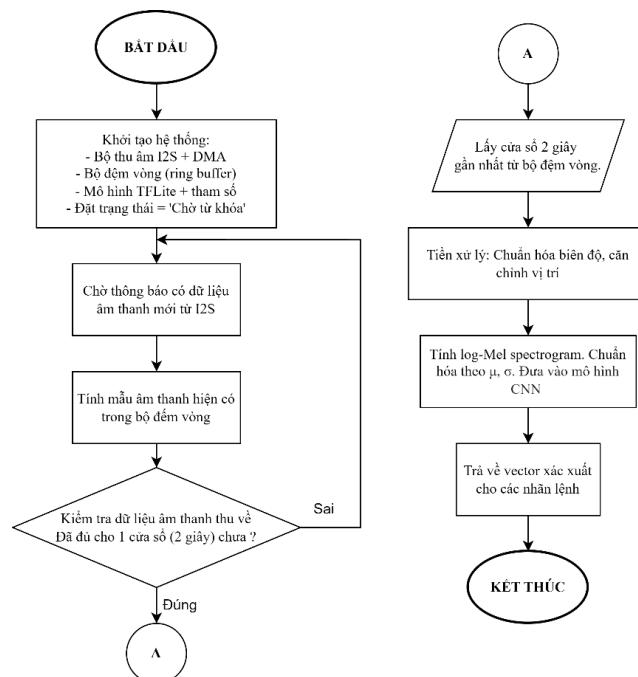
A) Các yêu cầu nhóm phải đảm bảo

Nhóm Hạ tầng giao tiếp MQTT là nền tảng để gateway giao tiếp với các node ESP32 thông qua broker MQTT. Nhóm này phải đảm bảo các yêu cầu sau:

- (A1) Gateway kết nối thành công tới MQTT Broker và duy trì kênh giao tiếp.
- (A2) Gateway đăng ký (subscribe) các topic cần thiết để thu nhận đầy đủ dữ liệu/sự kiện từ node.
- (A3) Khi có bản tin đến, gateway thực hiện tiền xử lý và xác thực đầu vào, sau đó điều phối bản tin về đúng nhóm nghiệp vụ (status/sensors/state/voice).
- (A4) Gateway cung cấp một cơ chế publish thống nhất để các nhóm khác gửi lệnh xuống node (đảm bảo log đồng nhất và dễ debug).

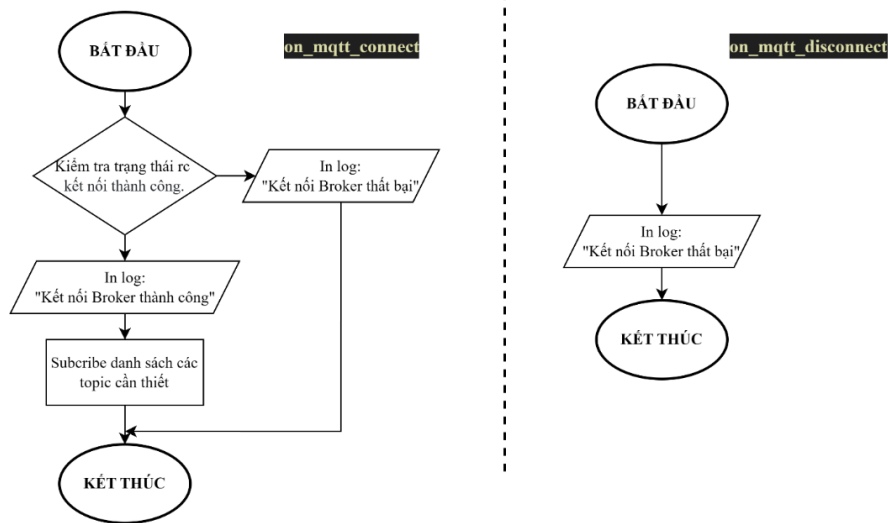
B) Thiết kế thuật toán/ Các lưu đồ con tương ứng với từng yêu cầu

Ứng với A1: Lưu đồ khởi tạo và kết nối MQTT



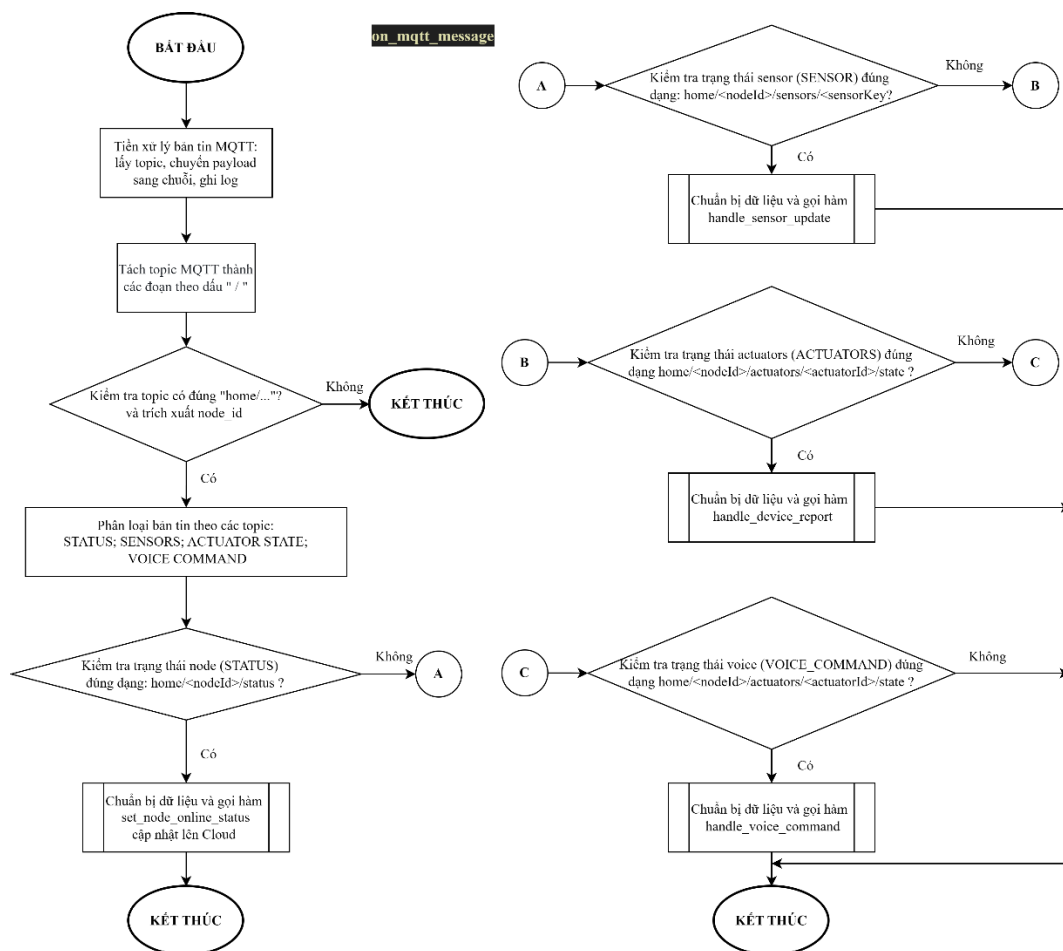
Hình 3. 19 Lưu đồ khởi tạo và kết nối MQTT

Ứng với A2: Lưu đề subscribe topic sau khi kết nối thành công



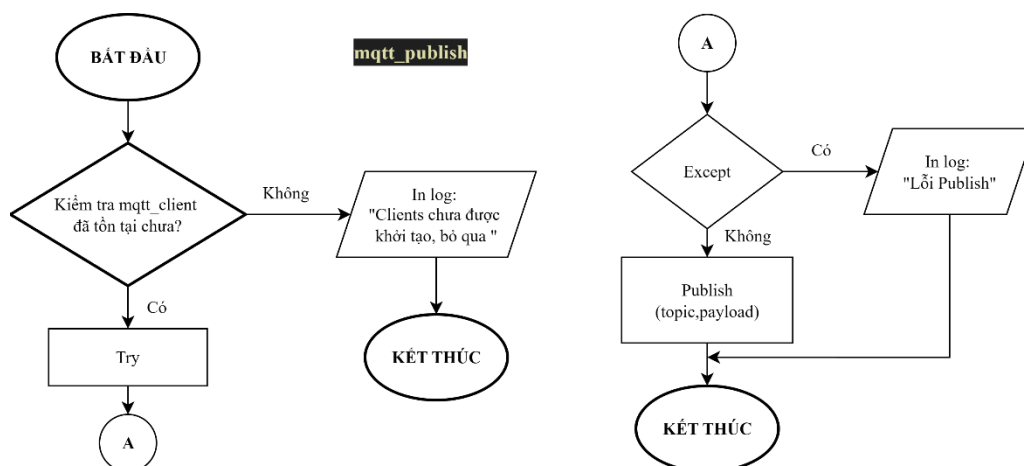
Hình 3. 20 Lưu đề subscribe topic MQTT

Ứng với A3: Lưu đề tiếp nhận bản tin và điều phối theo topic



Hình 3. 21 Lưu đề tiếp nhận bản tin và điều phối theo topic

Ứng với A4: Lưu đề publish lệnh xuống node



Hình 3. 22 Lưu đề publish lệnh qua MQTT

C) Chức năng các hàm trong nhóm và các bước liên kết logic giữa chúng

Bước 1 – Khởi tạo và kích hoạt kênh MQTT

Gateway gọi `start_mqtt_client()` (chức năng: tạo MQTT client, cấu hình auth, gắn callback và bắt đầu loop lắng nghe nền). Khi vòng lắng nghe nền chạy, gateway có thể nhận các sự kiện kết nối và bản tin theo cơ chế callback.

Bước 2 – Subscribe các topic ngay sau khi kết nối thành công

Khi broker xác nhận kết nối, `on_mqtt_connect()` được kích hoạt (chức năng: kiểm tra trạng thái kết nối và đăng ký subscribe các topic cần thiết). Nhờ subscribe này, mọi dữ liệu/sự kiện từ node sẽ “đổ về” gateway qua các topic quy ước.

Bước 3 – Tiếp nhận bản tin và chuẩn hoá dữ liệu đầu vào

Mỗi khi có message đến, `on_mqtt_message()` được gọi (chức năng: nhận bản tin, chuẩn hoá payload về dạng xử lý thống nhất và ghi log để truy vết). Bước này tạo tiền đề để các nhánh xử lý phía sau không phụ thuộc vào dạng dữ liệu thô.

Bước 4 – Xác thực topic và trích xuất thông tin định tuyến (`node_id`)

Trong `on_mqtt_message()`, gateway thực hiện kiểm tra định dạng/namespace topic (chức năng: đảm bảo topic thuộc hệ thống và lấy `node_id` làm ngữ cảnh xử lý). Nếu không hợp lệ thì bỏ qua; nếu hợp lệ thì chuyển sang bước điều phối.

Bước 5 – Điều phối bản tin theo loại sự kiện

Tiếp tục trong `on_mqtt_message()`, Gateway phân loại theo cấu trúc topic và gọi các handler nghiệp vụ tương ứng:

- `status` → cập nhật online/offline (nhóm 2 trạng thái)
- `sensors` → xử lý sensor/cảnh báo/automation (nhóm 4 sensor & alert)
- `actuator state` → cập nhật reported state (nhóm 3 trạng thái thiết bị)
- `voice_command` → xử lý lệnh giọng nói (nhóm 3 manual voice)

Bước 6 – Publish thống nhất cho các lệnh điều khiển

Khi các nhóm nghiệp vụ cần gửi lệnh xuống node, chúng đi qua `mqtt_publish()` (chức năng: publish tập trung, log thống nhất và bắt lỗi publish). Nhờ đó, Gateway duy trì được một “đầu ra MQTT” nhất quán, dễ kiểm soát và debug.

3.7.3 Triển khai chi tiết từng nhóm – Nhóm 2: Cập nhật trạng thái & tình trạng online

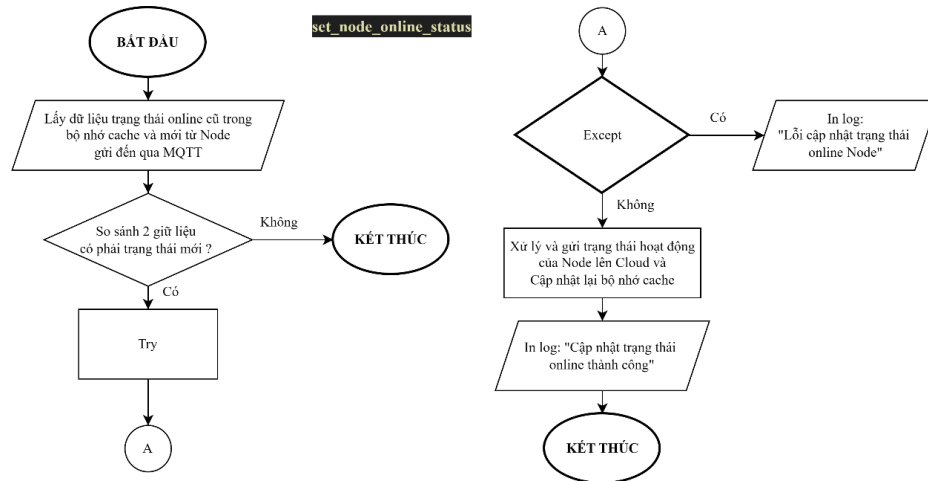
A) Các yêu cầu nhóm phải đảm bảo

Nhóm cập nhật trạng thái và tình trạng online chịu trách nhiệm đồng bộ “trạng thái thực tế” của hệ thống lên Cloud. Nhóm này phải đảm bảo các yêu cầu sau:

- (A1) Cập nhật tình trạng online/offline của node dựa trên bản tin trạng thái (`status`) gửi về từ MQTT.
- (A2) Cập nhật trạng thái thiết bị chấp hành (`actuator reported state`) dựa trên bản tin `state` gửi về từ MQTT, phản ánh trạng thái thực tế của thiết bị.
- (A3) Cập nhật tình trạng hoạt động của gateway theo cơ chế heartbeat định kỳ, nhằm giám sát gateway có đang chạy hay không.
- (A4) Đảm bảo tính ổn định dữ liệu: hạn chế cập nhật lặp không cần thiết, và mỗi lần cập nhật cần kèm dấu thời gian (`timestamp`) để phục vụ theo dõi.

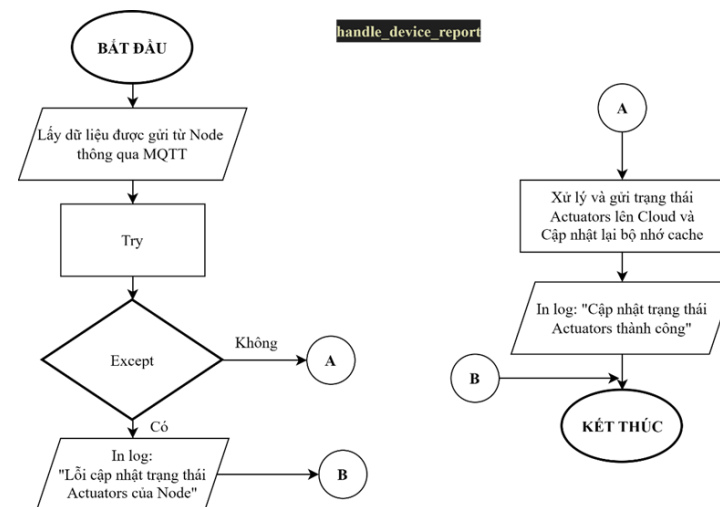
B) Thiết kế thuật toán/ Các lưu đồ con tương ứng với từng yêu cầu

Ứng với A1: Lưu đồ cập nhật online/offline của node



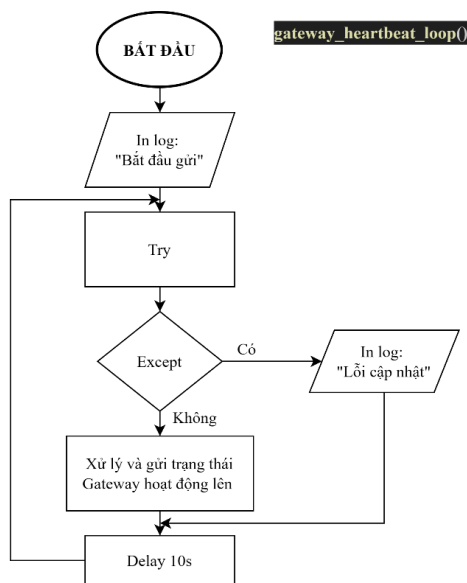
Hình 3. 23 Lưu đồ cập nhật online/offline của node

Ứng với A2: Lưu đồ cập nhật trạng thái báo cáo (reported state) của actuator



Hình 3. 24 Lưu đồ cập nhật trạng thái báo cáo (reported state) của actuator

Ứng với A3: Lưu đồ cập nhật trạng thái của Gateway



Hình 3

Hình 3. 25 Lưu đồ cập nhật trạng thái của Gateway

D) Chức năng các hàm trong nhóm và các bước liên kết logic giữa chúng

Bước 1 – Nhận sự kiện trạng thái từ MQTT và chuyển sang nhóm trạng thái

Trạng thái node/actuator được đưa vào gateway thông qua `on_mqtt_message()` (chức năng: điều phối bản tin theo topic).

Khi topic thuộc nhánh status hoặc actuator state, `on_mqtt_message()` sẽ gọi sang các hàm cập nhật tương ứng của nhóm 2.

Bước 2 – Cập nhật online/offline của node (presence node)

Hàm `set_node_online_status()` (chức năng: đồng bộ trạng thái online của node lên Firebase và cập nhật mốc thời gian) được gọi khi nhận bản tin status. Hàm này có cơ chế “chống ghi lặp” bằng cache để tránh cập nhật Cloud Firebase liên tục khi trạng thái không đổi, từ đó đảm bảo yêu cầu (A4).

Bước 3 – Cập nhật reported state của actuator (trạng thái thiết bị thực tế)

Khi node báo trạng thái actuator, gateway gọi `handle_device_report()` (chức năng: cập nhật `reportedValue` của actuator lên Cloud Firebase và gắn `timestamp`). Trạng thái này đại diện cho “thiết bị đang ở trạng thái gì thật”, giúp phân biệt với “lệnh mong muốn” (`desired`) ở nhóm điều khiển thủ công/tự động.

Bước 4 – Heartbeat gateway (presence gateway)

Song song với luồng MQTT, `gateway_heartbeat_loop()` (chức năng: định kỳ cập nhật trạng thái/`lastSeen` của gateway lên Cloud Firebase) chạy theo `thread/timer`. Luồng này không phụ thuộc message MQTT, nhằm đảm bảo yêu cầu (A3): Hệ thống có thể giám sát gateway ngay cả khi không có node gửi dữ liệu.

Bước 5 – Đồng bộ hoá đầu ra cho các nhóm khác

Sau khi các hàm ở bước 2–4 cập nhật Firebase, dữ liệu trạng thái trở nên chuẩn cho các thành phần khác: App hiển thị trạng thái, nhóm tự động hóa tham chiếu điều kiện, và nhóm điều khiển đối chiếu trạng thái thực tế sau khi gửi lệnh.

Như vậy, nhóm 2 đóng vai trò cầu nối giữa dữ liệu realtime và trạng thái lưu trữ trung tâm.

3.7.4 Triển khai chi tiết từng nhóm – Nhóm 3 – Xử lý lệnh điều khiển thủ công (App + Voice)

A) Các yêu cầu nhóm phải đảm bảo

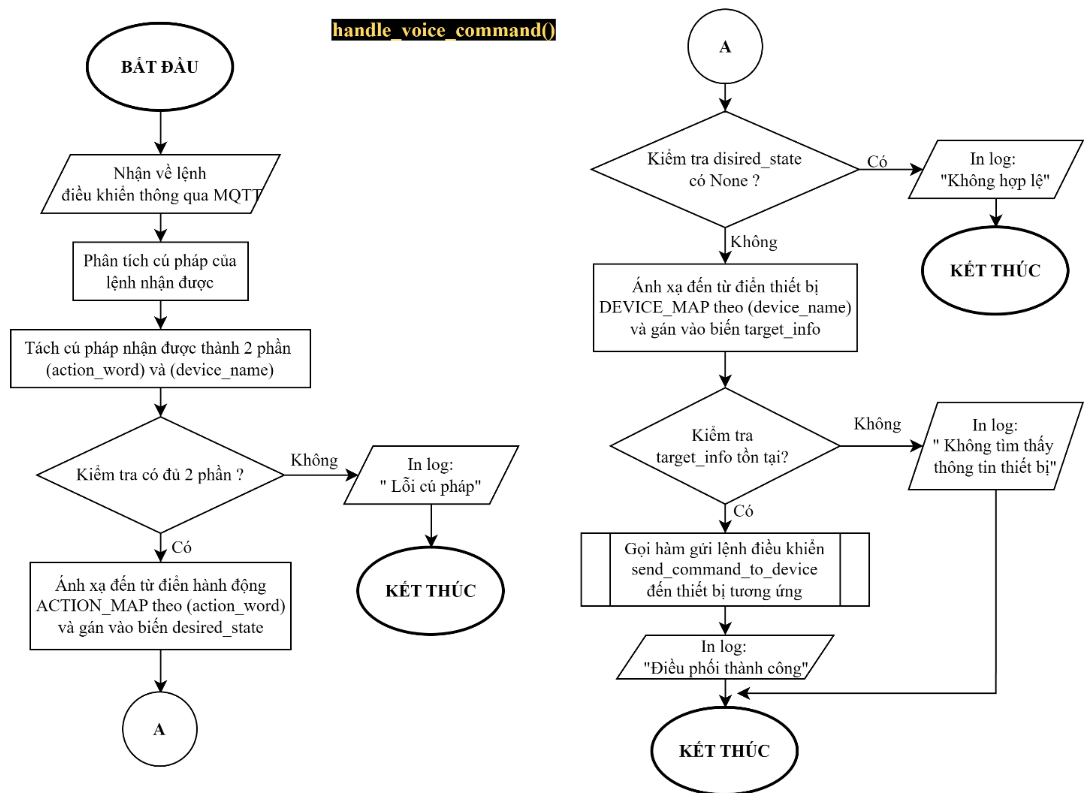
Nhóm điều khiển thủ công chịu trách nhiệm tiếp nhận từ người dùng và chuyển thành lệnh điều khiển thực thi trên thiết bị. Nhóm này phải đảm bảo các yêu cầu sau:

- (A1) Tiếp nhận lệnh điều khiển từ giọng nói (MQTT `voice_command`) và chuyển đổi thành hành động điều khiển tương ứng.
- (A2) Tiếp nhận lệnh điều khiển từ ứng dụng (Firebase `desiredValue/commandValue`) theo cơ chế realtime và xử lý liên tục.

- (A3) Gửi lệnh điều khiển xuống node theo chuẩn thống nhất, đảm bảo truy vết nguồn lệnh (MANUAL/VOICE) và tránh gửi sai đích.
- (A4) Sau khi phát lệnh, hệ thống có cơ sở để đối chiếu trạng thái thực tế (thông qua reported state ở Nhóm 2), nhằm đảm bảo phản hồi trạng thái đúng.

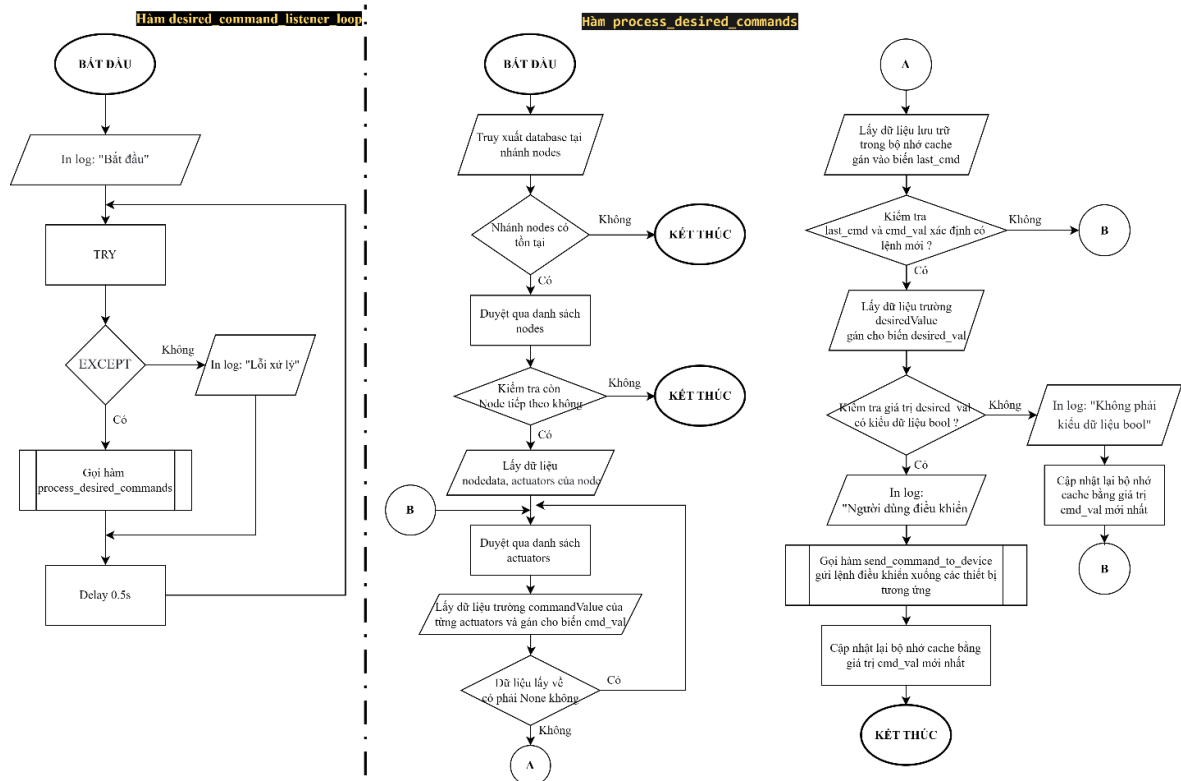
B) Thiết kế thuật toán/ Các lưu đồ con tương ứng với từng yêu cầu

Ứng với A1: Lưu đồ xử lý lệnh giọng nói từ MQTT



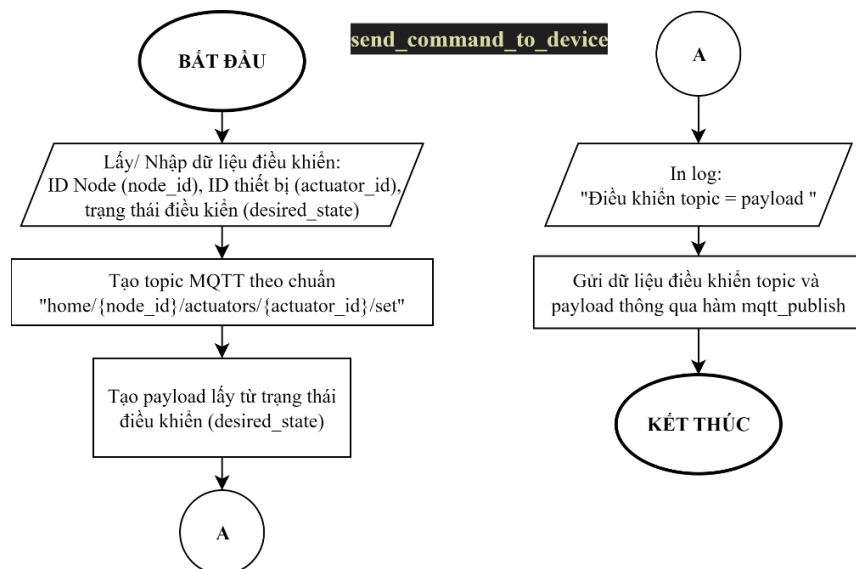
Hình 3. 26 Lưu đồ xử lý lệnh giọng nói từ MQTT

Ứng với A2: Lưu đề lắng nghe và xử lý lệnh từ App (Firebase desired/command)



Hình 3. 27 Lưu đề lắng nghe và xử lý lệnh từ App

Ứng với A3: Lưu đề gửi lệnh xuống Node



Hình 3. 28 Lưu đề gửi lệnh điều khiển xuống node qua MQTT

C) Chức năng các hàm trong nhóm và các bước liên kết logic giữa chúng

Bước 1 – Nhận lệnh từ App (đầu vào 1)

Hàm `desired_command_listener_loop()` giúp Gateway luôn theo dõi các thay đổi tạo trường `desired/ command` từ ứng dụng.

Trong vòng lặp, `process_desired_commands()` đọc yêu cầu điều khiển từ Cloud Firebase và chuyển thành lệnh điều khiển và chuẩn bị dữ liệu cần gửi xuống node.

Bước 2 – Nhận lệnh giọng nói từ MQTT (đầu vào 2)

Khi broker nhận bản tin `voice_command`, hàm `on_mqtt_message()` điều phối bản tin theo topic, dữ liệu được đưa sang hàm `handle_voice_command()` để phân tích câu lệnh và ánh xạ thành thiết bị đích + trạng thái mong muốn.

Bước 3 – Hợp nhất về một cơ chế phát lệnh thống nhất (đầu ra chung)

Dù lệnh điều khiển xuất phát từ App hay Voice, điểm hội tụ đều là hàm `send_command_to_device()`, tại đây hàm chịu trách nhiệm đóng gói lệnh điều khiển theo chuẩn topic/payload và gán “nguồn lệnh” để truy vết.

Sau đó hàm `send_command_to_device` sử dụng hàm `mqtt_publish()` để gửi lệnh đến broker vì vậy mỗi khi có lệnh Node nhận lệnh và thực thi. Bước này đáp ứng yêu cầu (A3) về chuẩn hoá đường phát lệnh và truy vết nguồn lệnh.

Bước 4 – Vòng phản hồi trạng thái thực tế sau điều khiển

Sau khi Node thực thi lệnh, trạng thái actuator tương ứng sẽ được gửi về qua MQTT. Bản tin state quay lại Gateway qua Nhóm 1 và được cập nhật bởi Nhóm 2 thông qua `handle_device_report ()` giúp cập nhật giá trị thực tế của thiết bị lên Cloud Firebase.

3.7.5 Triển khai chi tiết từng nhóm – Nhóm 4: Xử lý dữ liệu cảm biến thời gian thực & cảnh báo

A) Các yêu cầu nhóm phải đảm bảo

Đây là nhóm biến dữ liệu thô từ Node thành thông tin có ý nghĩa. Nhóm này phải đảm bảo:

- (A1) Tiếp nhận dữ liệu cảm biến realtime từ MQTT và cập nhật “giá trị hiện tại” (live state) của từng sensor theo từng Node.
- (A2) Thực hiện đánh giá ngưỡng/ cảnh báo theo cấu hình (threshold) và xác định trạng thái bình thường hay cảnh báo.
- (A3) Khi phát hiện cảnh báo, ghi nhận lịch sử/ nhật ký sự kiện (history/ notification log) để phục vụ truy vết và thống kê.
- (A4) Gửi thông báo đẩy (FCM) khi cần thiết, đồng thời có cơ chế hạn chế spam (không gửi lặp liên tục cùng một cảnh báo).
- (A5) Kích hoạt nhóm tự động hoá khi dữ liệu cảm biến thay đổi (nếu hệ thống đang ở chế độ cho phép automation).

B) Mô tả thiết kế phân cấp của nhóm (mô hình cha–con)

Nhóm xử lý dữ liệu cảm biến được tổ chức theo mô hình phân cấp cha – con, trong đó hàm `handle_sensor_update()` đóng vai trò hàm cha điều phối cho toàn bộ quy trình xử lý một bản tin cảm biến.

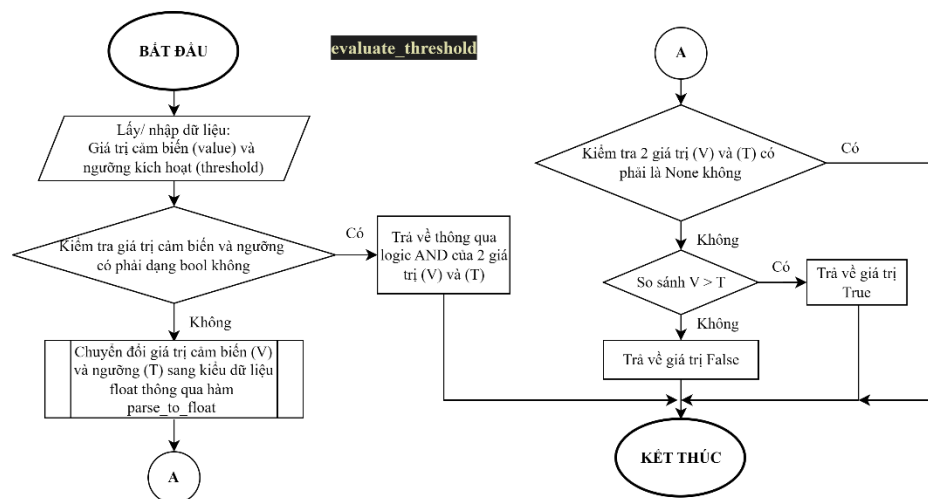
Bên trong hàm cha, các hàm con được thiết kế theo nguyên tắc tách nhiệm vụ: mỗi hàm con đảm trách một công đoạn rõ ràng (chuẩn hoá dữ liệu, đánh giá ngưỡng, cập nhật trạng thái hiện tại, chống lặp cảnh báo, ghi lịch sử thông báo, gửi thông báo FCM, và kích hoạt automation). Cách tổ chức này giúp quy trình xử lý được trình bày và kiểm soát như một “pipeline” và có thể sửa chữa không mà không ảnh hưởng các hàm chức năng khác.

C) Thiết kế thuật toán/ Các lưu đồ hàm con

Hàm chuẩn hoá dữ liệu cảm biến *parse_to_float(value)*

Hàm *parse_to_float* đảm nhiệm việc chuẩn hoá biểu diễn giá trị cảm biến về dạng số khi có thể. Ý nghĩa hàm con này tạo ra một giá trị số hợp lệ (hoặc trạng thái không chuyển đổi được). Đầu ra này giúp hàm cha có cơ sở thực hiện so sánh ngưỡng một cách ổn định, đồng thời hạn chế sai lệch do định dạng dữ liệu không đồng nhất giữa các loại cảm biến.

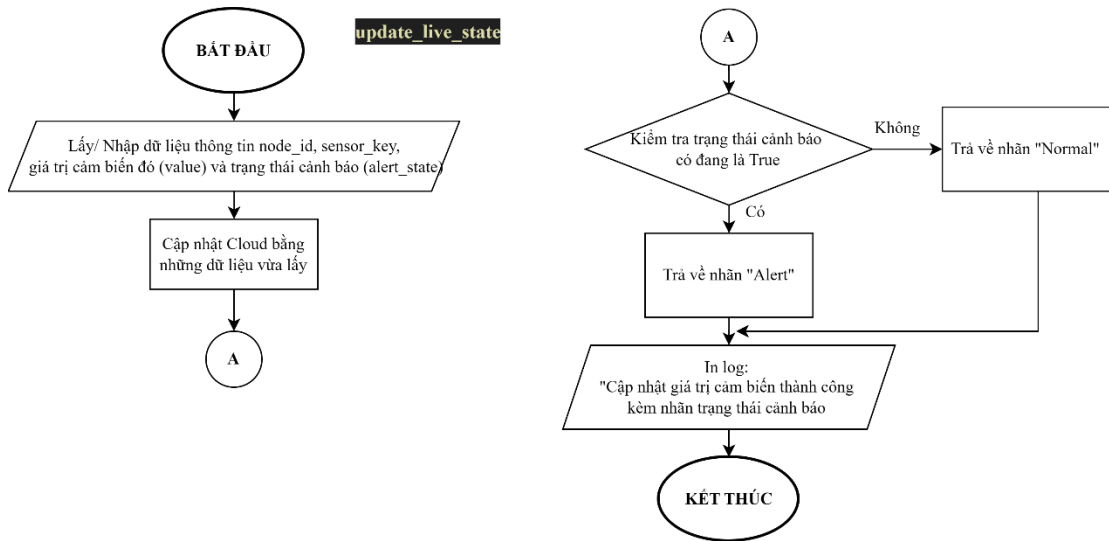
Hàm đánh giá ngưỡng/ cảnh báo



Hình 3. 29 Lưu đồ thuật toán hàm đánh giá ngưỡng cảnh báo

Hàm đóng vai trò quyết định trong cơ chế phản hồi của hệ thống bằng cách so sánh dữ liệu cảm biến với các ngưỡng cấu hình để xác định trạng thái cảnh báo *is_alert*. Giúp hàm cha có căn cứ để quyết định các nhánh xử lý cảnh báo chuyên sâu.

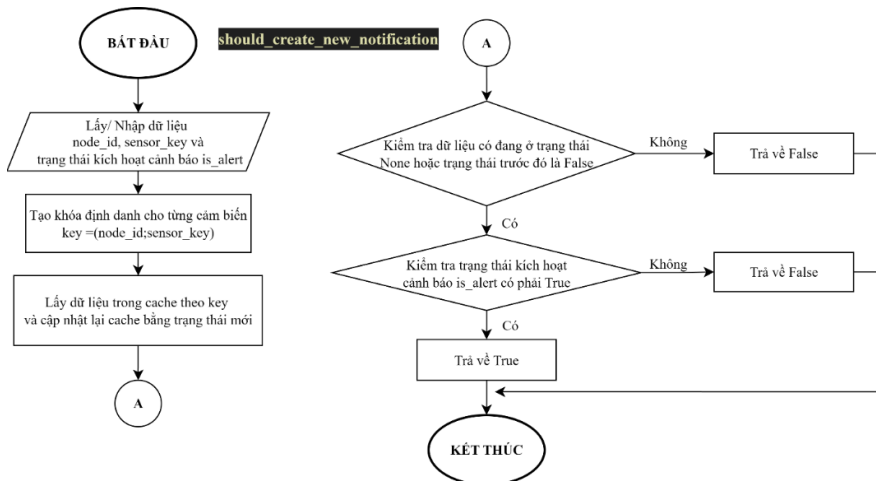
Hàm cập nhật giá trị cảm biến hiện tại lên Cloud Firebase



Hình 3. 30 Lưu đồ thuật toán hàm cập nhật giá trị cảm biến lên Cloud Firebase

Hàm `update_live_state` chịu trách nhiệm đồng bộ giá trị cảm biến mới nhất lên cơ sở dữ liệu trung tâm (Cloud Firebase) kèm theo trạng thái cảnh báo tương ứng. Giúp App có thể cập nhật và hiển thị trạng thái chính xác và realtime.

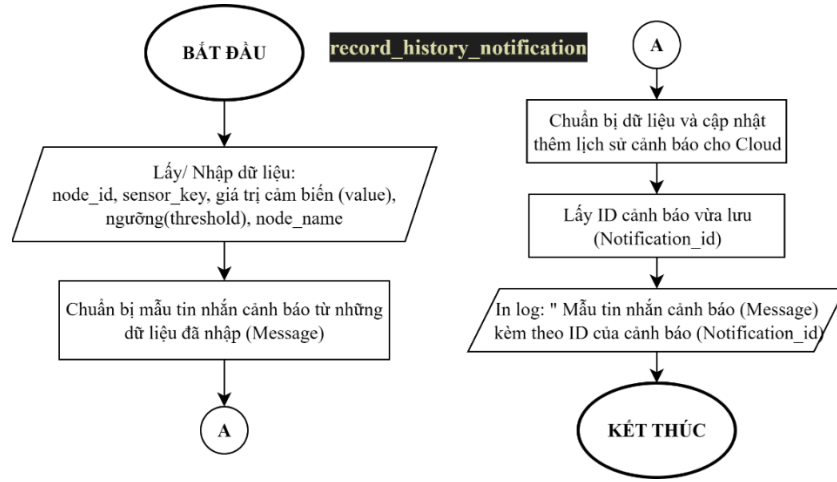
Hàm chống lặp cảnh báo



Hình 3. 31 Lưu đồ thuật toán hàm chống lặp cảnh báo

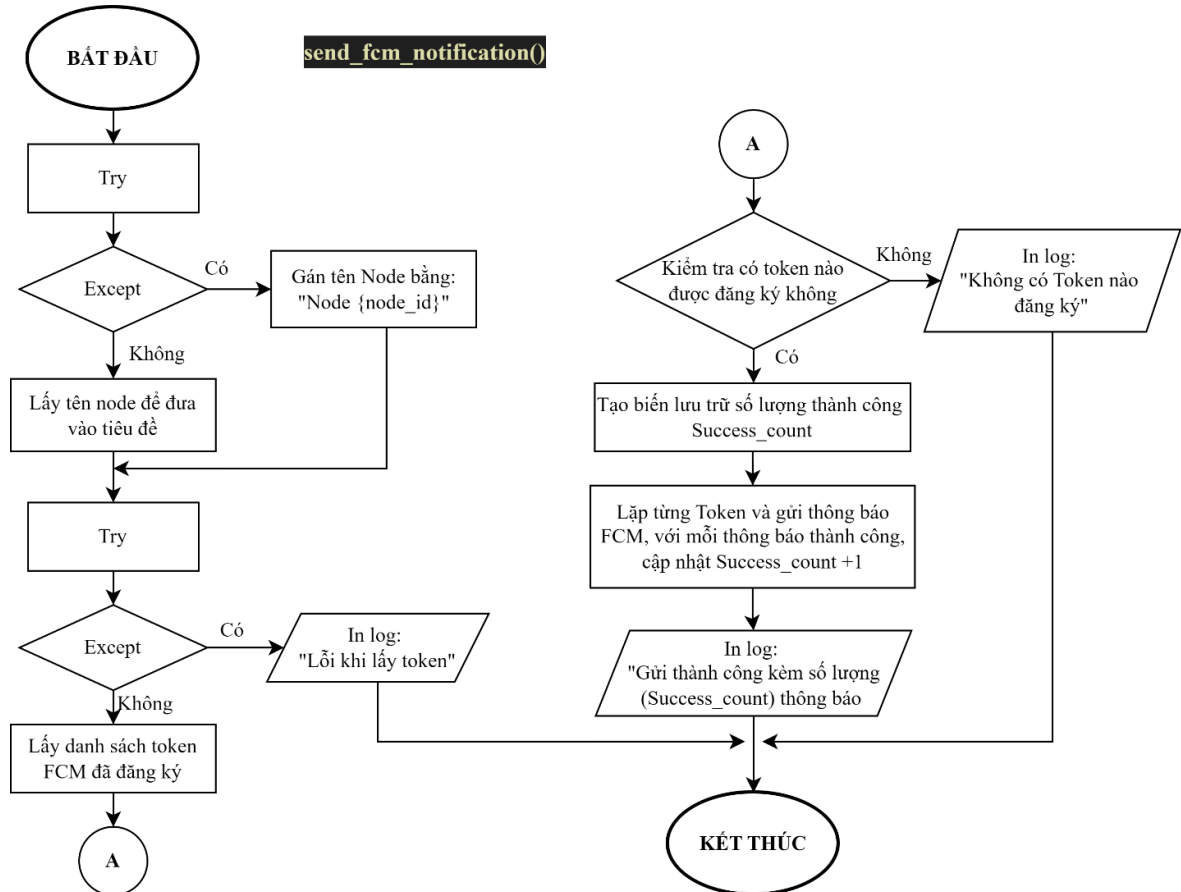
Dữ liệu cảm biến có thể dao động liên tục quanh ngưỡng, dẫn đến nguy cơ phát cảnh báo lặp với tần suất cao. Hàm `should_create_new_notification` được tách ra để cung cấp một quyết định điều khiển cho hàm cha, nhằm giới hạn việc này.

Hàm ghi lịch sử sự kiện thông báo



Hình 3. 32 Lưu đồ thuật toán hàm ghi lịch sử sự kiện thông báo

Hàm gửi thông báo đẩy FCM



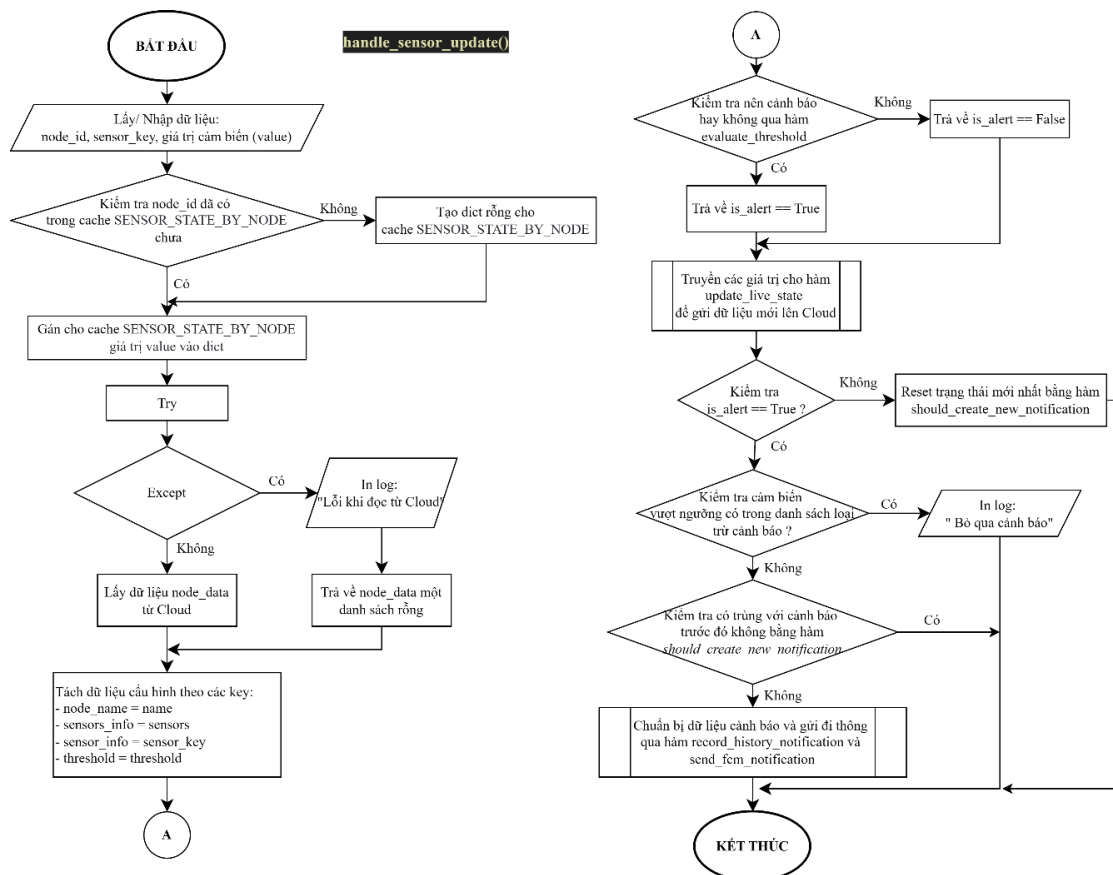
Hình 3. 33 Lưu đồ thuật toán hàm gửi thông báo đẩy FCM

Hàm `send_fcm_notification` phụ trách thông báo đẩy đến người dùng thông qua cơ chế FCM ((Firebase Cloud Messaging) của Google, cho phép hệ thống gửi thông báo đẩy đến ứng dụng di động trên IOS, Android một cách đáng tin cậy trong lúc người dùng không thao tác và theo dõi trên ứng dụng di động.

Kích hoạt tự động hoá sau cập nhật cảm biến

Hàm `run_automations_for_node` được thể hiện ở Hình 3. 39 Lưu đồ giải thuật nhóm 5 là cầu nối từ dữ liệu cảm biến sang cơ chế ra quyết định hành động (automation). Khả năng kích hoạt quá trình đánh giá rule dựa trên trạng thái cảm biến mới nhất và (nếu thoả điều kiện) dẫn tới hành động điều khiển thiết bị ở nhóm tự động hoá. Việc gọi bước này sau khi live state đã được cập nhật giúp automation luôn dựa trên “trạng thái hiện tại” nhất quán của hệ thống.

D) Thiết kế thuật toán/ Các lưu đồ hàm cha



Hình 3. 34 Lưu đồ giải thuật hàm cha: Xử lý dữ liệu cảm biến

Các bước liên kết logic giữa chúng

Quy trình xử lý một bản tin cảm biến được khởi phát khi Nhóm MQTT Transport phân loại được message thuộc nhánh sensors và chuyển giao bộ tham số định danh (node và loại cảm biến) kèm giá trị đo được. Tại thời điểm này, hàm `handle_sensor_update` tiếp nhận dữ liệu và điều phối theo các giai đoạn liên tiếp.

Trước hết, hàm cha sử dụng `parse_to_float` để tạo ra giá trị đã chuẩn hoá. Đầu ra chuẩn hoá này đóng vai trò nền tảng cho bước đánh giá ngưỡng vì nó cho phép so sánh theo cùng một hệ quy chiếu (numeric). Tiếp theo, hàm cha gọi `evaluate_threshold` để sinh ra kết quả đánh giá cảnh báo (trạng thái bình thường/cảnh báo). Chính kết quả này là điều kiện điều phối quan trọng: nó quyết định “luồng chỉ cập nhật trạng thái” hay “luồng phát sinh sự kiện cảnh báo”.

Song song với nhánh cảnh báo, hệ thống cần đảm bảo trạng thái hiện tại luôn được cập nhật, do đó `update_live_state` được sử dụng để ghi trạng thái cảm biến (live state) mới nhất lên Cloud Firestore. Khi live state đã được cập nhật, hệ thống có được một trạng thái đồng nhất để các thành phần khác đọc và sử dụng.

Trong trường hợp trạng thái đánh giá cho thấy có cảnh báo (hoặc có chuyển trạng thái đáng kể), hàm cha không phát thông báo ngay lập tức mà cần kiểm tra có nên phát thông báo hay không, sử dụng `should_create_new_notification` để tạo ra quyết định chống lặp. Đây là cơ chế đảm bảo hành vi ổn định: chỉ khi quyết định cho phép cảnh báo thì pipeline mới chuyển sang các bước tạo sự kiện. Khi được cho phép, `record_history_notification` tạo bản ghi lịch sử thông báo và `send_fcm_notification` phát thông báo đẩy tới người dùng. Cách điều phối này giúp hệ thống vừa “không bỏ sót cảnh báo”, vừa “không gây nhiễu do lặp”.

Cuối cùng, sau khi trạng thái cảm biến đã được cập nhật và các xử lý cảnh báo (nếu có) đã hoàn tất, `handle_sensor_update` kích hoạt `run_automations_for_node`. Việc kích hoạt ở cuối pipeline nhằm đảm bảo automation đánh giá trên dữ liệu live state đã được cập nhật, từ đó giảm rủi ro ra quyết định dựa trên trạng thái cũ. Nếu các kịch

bản thoả điều kiện, luồng tự động hoá sẽ tiếp tục ở Nhóm 5 và có thể phát sinh lệnh điều khiển xuống thiết bị thông qua cơ chế gửi lệnh thống nhất của hệ thống.

3.7.6 Triển khai chi tiết từng nhóm – Nhóm 5 – Tự động hoá

A) Các yêu cầu nhóm phải đảm bảo

Nhóm tự động hoá là lớp “ra quyết định” của thuật toán điều phối IoT, cho phép hệ thống phản ứng với ngữ cảnh (dữ liệu cảm biến, trạng thái thiết bị, cấu hình người dùng) hoàn toàn tự động mà không cần thao tác thủ công. Nhóm này phải đảm bảo các yêu cầu sau:

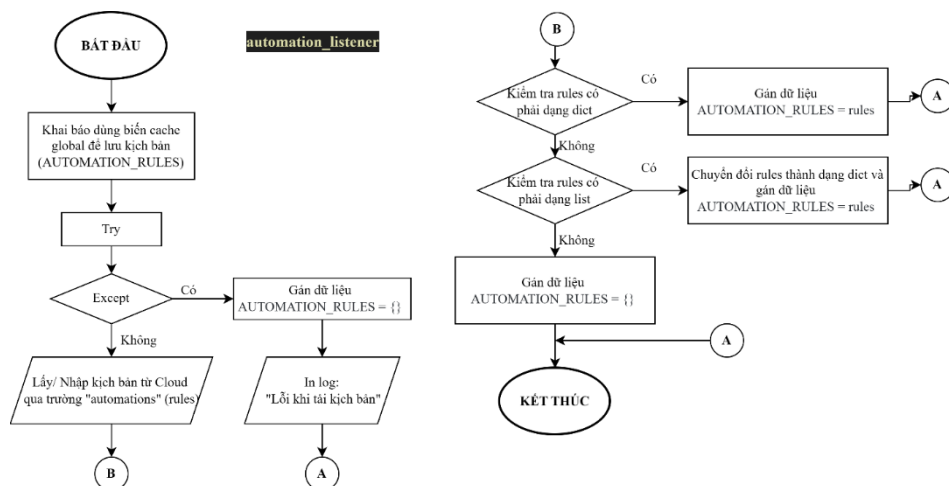
- (A1) Tiếp nhận và đồng bộ cấu hình tự động hoá (rule/automation) từ cơ sở dữ liệu trung tâm theo cơ chế realtime.
- (A2) Đánh giá điều kiện kích hoạt (trigger) của từng rule dựa trên trạng thái hiện tại của node.
- (A3) Tôn trọng cơ chế bật/tắt tự động hoá theo chế độ hệ thống và theo node (AUTO/MANUAL) để tránh xung đột với điều khiển thủ công.
- (A4) Khi rule thoả điều kiện, thực thi hành động (actions) một cách nhất quán thông qua cơ chế gửi lệnh thống nhất của gateway.
- (A5) Đảm bảo tính ổn định vận hành, đồng thời đảm bảo luồng tự động hoá có thể truy vết và kiểm soát.

B) Mô tả thiết kế phân cấp của nhóm (mô hình cha-con)

Tương tự nhóm 4, tại nhóm này cũng áp dụng phân cấp theo mô hình cha con, cho phép chủ động khi thay đổi cách đánh giá điều kiện kích hoạt hoặc cách thực thi hành động cụ thể mà không làm ảnh hưởng cơ chế đồng bộ rule và luồng điều phối tổng.

C) Thiết kế thuật toán/ Các lưu đồ hàm con

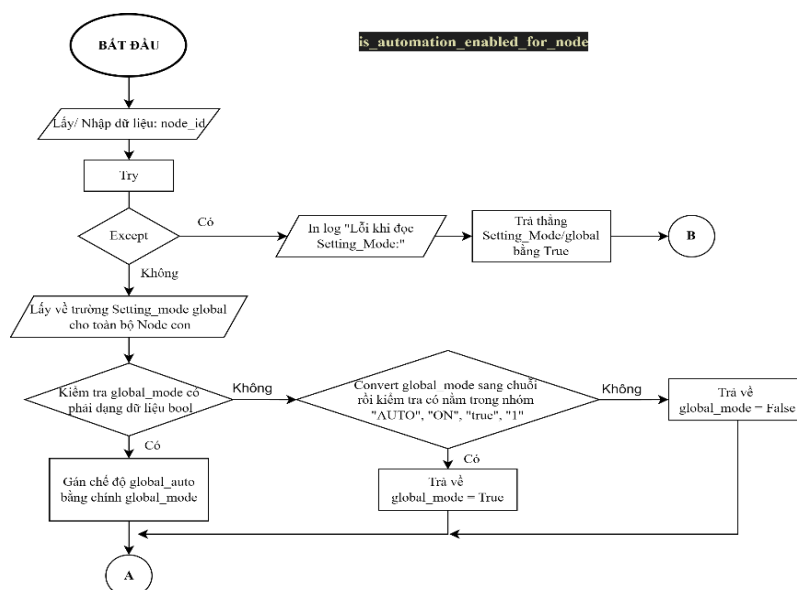
Hàm đồng bộ rule tự động hoá từ Cloud Firebase



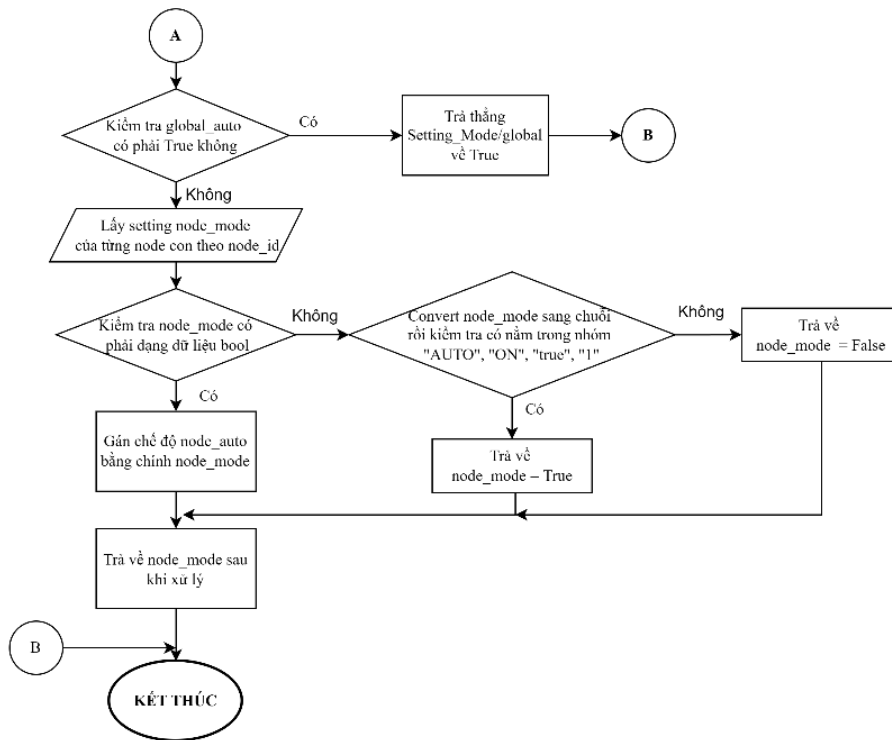
Hình 3. 35 Lưu đồ giải thuật hàm đồng bộ kịch bản từ Cloud Firebase

Hàm `automation_listener` được sử dụng để thiết lập cơ chế lắng nghe và cập nhật cấu hình tự động theo thời gian thực. Bảo đảm rằng các quy tắc tự động hoá luôn phản ánh đúng trạng thái cấu hình hiện hành, từ đó hạn chế rủi ro hệ thống thực thi theo cấu hình lỗi thời.

Kiểm soát quyền kích hoạt tự động hoá theo chế độ vận hành



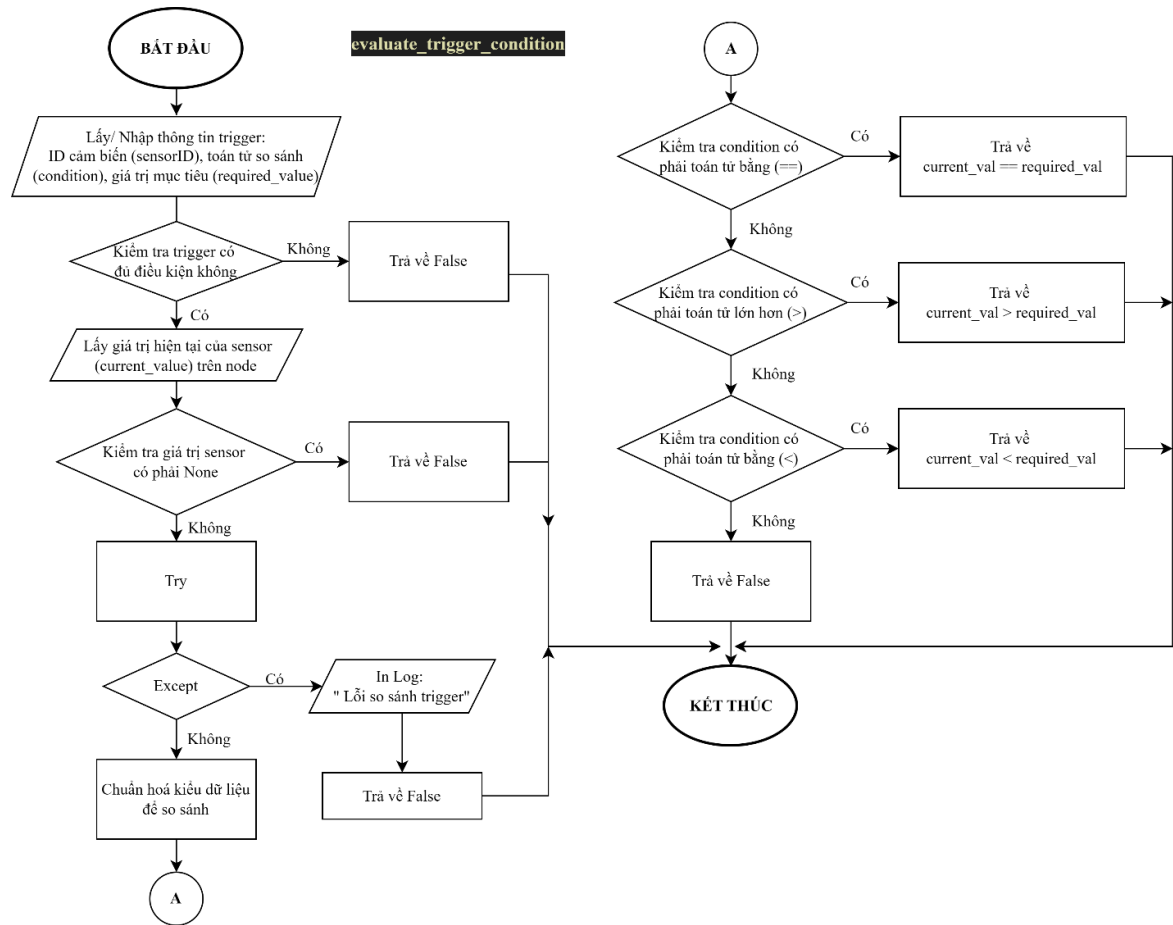
Hình 3. 36 Lưu đồ giải thuật hàm kiểm soát quyền kích hoạt tự động hoá



Hình 3. 37 Lưu đồ giải thuật hàm kiểm soát quyền kích hoạt tự động hoá theo chế độ vận hành (Hình tiếp theo của Hình 3.36)

Hàm `is_automation_enabled_for_node` đảm nhiệm chức năng kiểm soát chế độ vận hành, nhằm bảo đảm tự động hoá chỉ hoạt động khi được cho phép. Cụ thể, hàm thực hiện kiểm tra các điều kiện liên quan đến chế độ AUTO/MANUAL (ở mức hệ thống và/hoặc mức node), qua đó tạo ra một cơ chế “chặn sớm” trước khi tiến hành đánh giá quy tắc. Thiết kế này có ý nghĩa quan trọng trong thực tế vận hành: nó giảm nguy cơ xung đột giữa điều khiển thủ công và điều khiển tự động, đồng thời tối ưu tài nguyên xử lý bằng cách tránh đánh giá rule khi chế độ không cho phép.

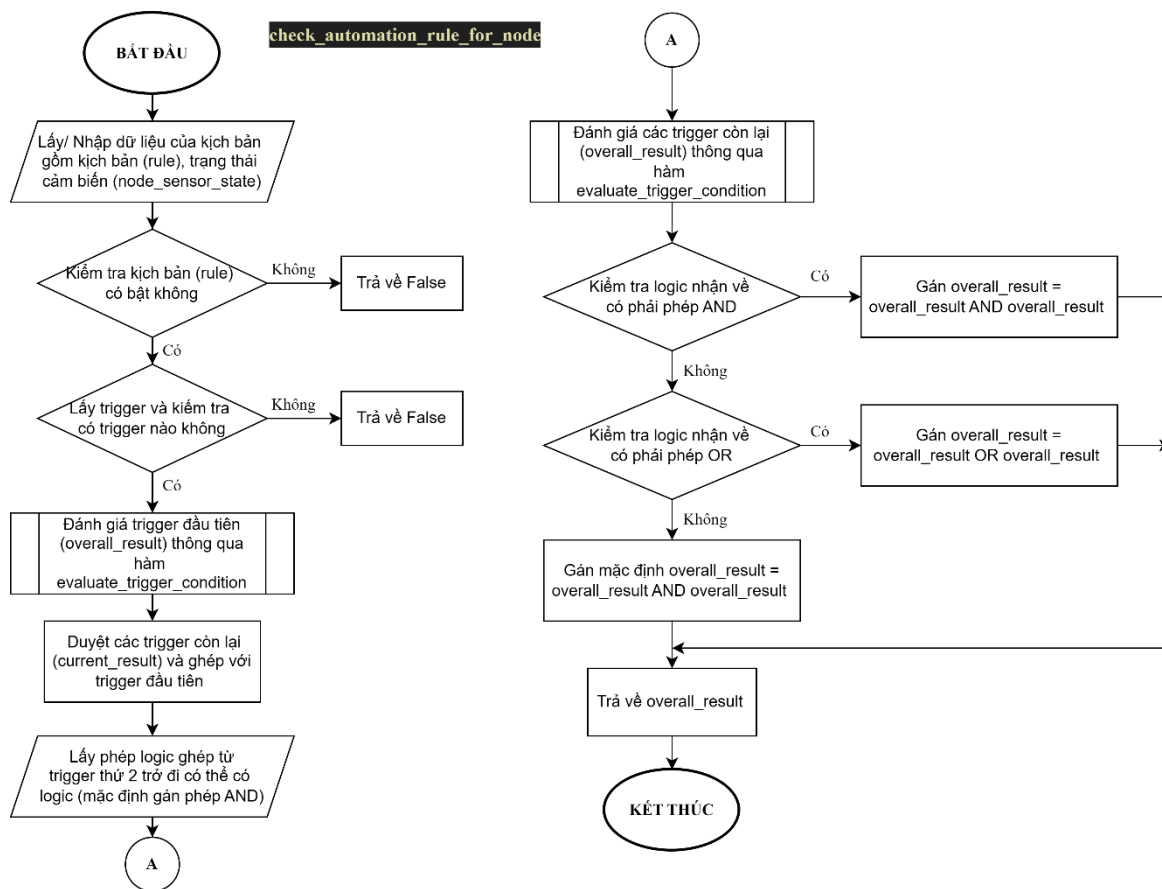
Đánh giá điều kiện kích hoạt của một trigger



Hình 3. 38 Lưu đồ giải thuật hàm đánh giá điều kiện kích hoạt của một trigger

Hàm `evaluate_trigger_condition` là đơn vị đánh giá điều kiện ở mức thấp nhất. Đầu ra của hàm là kết quả thoả/không thoả của một trigger (và có thể kèm theo thông tin phụ như mức so sánh). Một trigger đúng phải bao gồm: phép so sánh lớn hơn/nhỏ hơn, khoảng giá trị, điều kiện kết hợp (nếu nhiều hơn 2 trigger).

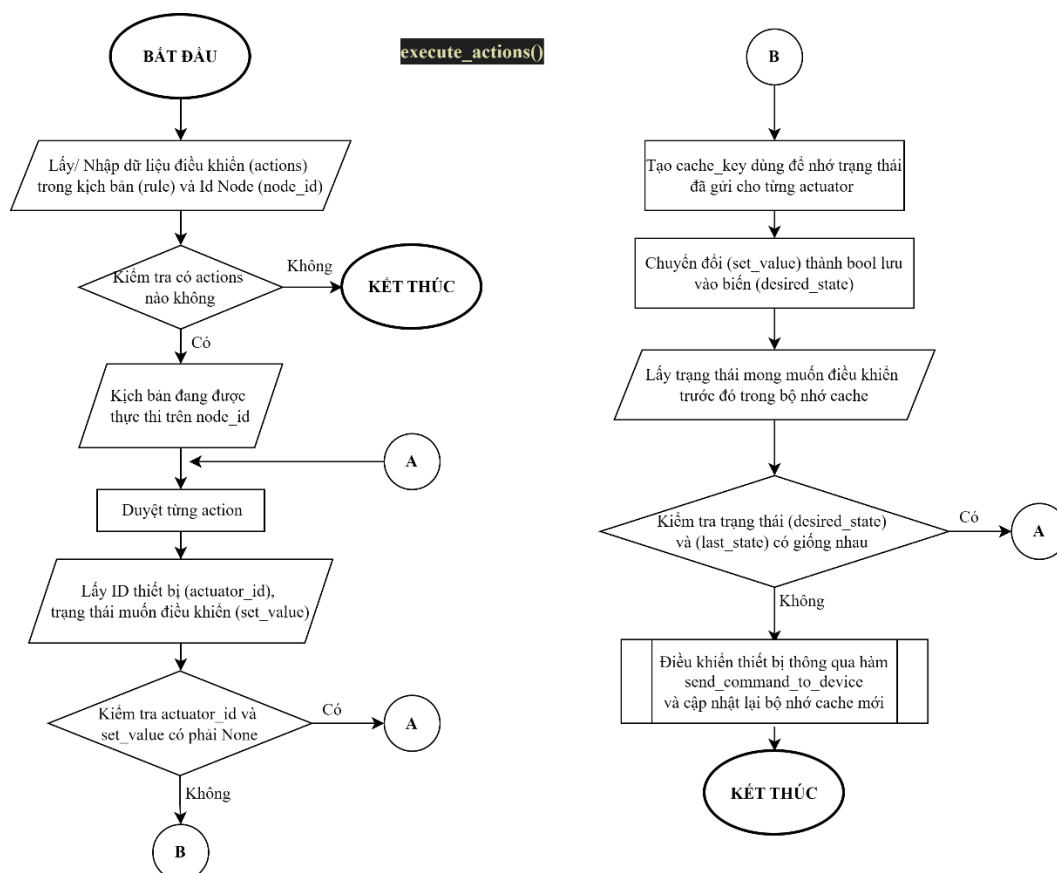
Kiểm tra một kịch bản (rule) có đủ điều kiện để thực hiện



Hình 3. 39 Lưu đồ giải thuật kiểm tra một kịch bản (rule) có đủ điều kiện để thực hiện

Trong cơ chế tự động hoá, trigger là một điều kiện đơn lẻ (ví dụ: nhiệt độ > ngưỡng), còn rule là một quy tắc hoàn chỉnh gồm tập trigger (có thể nhiều trigger kết hợp AND/OR và ràng buộc khác) và tập hành động (actions) sẽ thực thi khi điều kiện thoả. Vì vậy, hàm `check_automation_rule_for_node` thực hiện đánh giá tổng hợp kết quả các trigger theo logic của rule để đưa ra kết luận cuối cùng rule có được kích hoạt tại thời điểm hiện tại hay không.

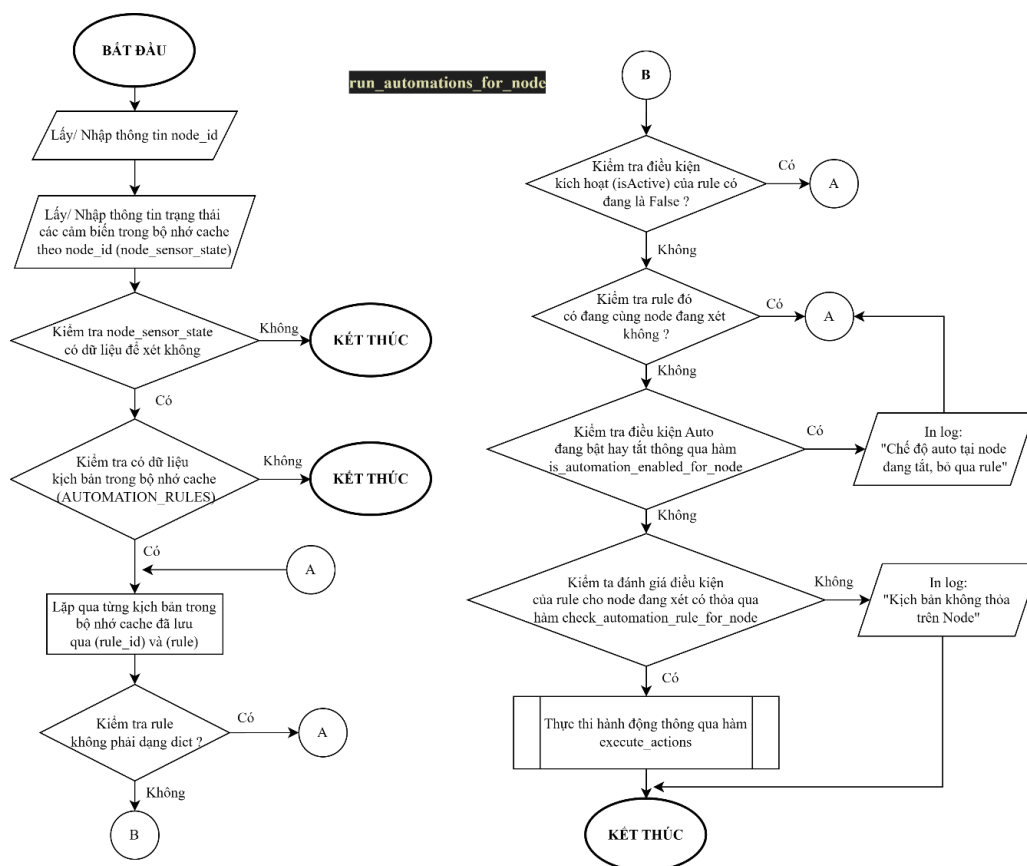
Thực thi hành động điều khiển khi rule được kích hoạt



Hình 3. 40 Lưu đồ giải thuật hàm thực thi hành động điều khiển khi rule được kích hoạt

Hàm `execute_actions` chịu trách nhiệm thực thi danh sách hành động (actions) đi kèm rule, theo đúng trình tự và nội dung đã được cấu hình. Về mặt thiết kế điều phối, hàm này là nơi gắn kết cơ chế tự động hoá với cơ chế điều khiển thiết bị của gateway: hành động được chuyển hoá thành các lệnh điều khiển và được gửi xuống node thông qua kênh điều khiển thống nhất của hệ thống. Nhờ vậy, tự động hoá và điều khiển thủ công sử dụng cùng một cơ chế phát lệnh, giúp hệ thống có hành vi nhất quán, dễ kiểm soát và thuận lợi cho truy vết vận hành.

D) Thiết kế thuật toán/ Các lưu đồ hàm cha



Hình 3. 41 Lưu đồ thuật toán điều phối toàn bộ logic tự động hóa

Các bước liên kết logic giữa chúng

Quá trình tự động hoá thường được kích hoạt sau khi trạng thái của node thay đổi, điển hình là sau khi Nhóm 4 cập nhật trạng thái dữ liệu cảm biến và gọi `run_automations_for_node(node_id)`. Tại thời điểm này, hàm cha tiếp cận hai nguồn thông tin cần thiết: Tập quy tắc tự động hoá hiện hành (được cập nhật bởi cơ chế lắng nghe cấu hình) và trạng thái hiện tại của node (đặc biệt là các giá trị cảm biến mới nhất).

Trước khi đánh giá các quy tắc có trong kịch bản, hàm `run_automations_for_node` gọi `is_automation_enabled_for_node(node_id)` để kiểm tra việc cho phép chạy tự động hoá theo chế độ vận hành (AUTO/MANUAL). Nếu tự động hoá không được

cho phép, hàm kết thúc sớm nhằm tránh xung đột với điều khiển thủ công và giảm xử lý không cần thiết.

Khi tự động hoá được cho phép, hàm cha duyệt qua danh sách rule áp dụng cho node và gọi `check_automation_rule_for_node()` cho từng rule. Ở đây, “kiểm tra ở mức rule” được hiểu là: Hàm tổng hợp kết quả của các trigger (có thể nhiều trigger kết hợp theo logic AND/OR và các ràng buộc kèm theo) để đưa ra kết luận cuối cùng rằng rule có được kích hoạt trong trạng thái hiện tại hay không. Bên trong bước này, hàm `evaluate_trigger_condition()` được sử dụng để đánh giá từng trigger riêng lẻ, sau đó các kết quả trigger được kết hợp theo cấu trúc logic của rule.

Nếu một rule được kết luận là kích hoạt, `run_automations_for_node` chuyển sang nhánh thực thi và gọi `execute_actions()` để thực hiện danh sách hành động đã cấu hình. Các hành động này được chuyển hoá thành lệnh điều khiển và được gửi xuống thiết bị thông qua cơ chế phát lệnh thống nhất của gateway, đảm bảo tự động hoá và điều khiển thủ công sử dụng cùng một kênh điều khiển, từ đó thuận lợi cho giám sát và truy vết vận hành.

3.7.7 Kết luận chung quá trình thiết kế thuật toán điều phối cho toàn bộ hệ thống IOT

Hệ thống được thiết kế theo hướng phân rã mô-đun và điều phối theo luồng, giúp tách biệt hoàn toàn các lớp xử lý dữ liệu và điều khiển. Thay vì tập trung logic vào một khối duy nhất, thuật toán được tổ chức thành các nhóm thành phần độc lập nhưng gắn kết chặt chẽ với giá trị kỹ thuật đạt được: Cách tiếp cận phân cấp này giúp hệ thống đạt được các tiêu chí quan trọng trong triển khai thực tế:

- Tách biệt mối quan tâm: Giảm phụ thuộc chéo, giúp việc kiểm thử và bảo trì trở nên đơn giản.
- Khả năng mở rộng: Dễ dàng tích hợp thêm thiết bị hoặc quy tắc tự động hóa mới mà không ảnh hưởng đến cấu trúc nền tảng.
- Tính bền vững: Đảm bảo khả năng tái sử dụng mã nguồn và vận hành ổn định trong môi trường IoT có tính động và phân tán cao.

3.8 Thiết kế ứng dụng

3.8.1 Màn hình đăng nhập

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.1 Màn hình đăng nhập*

3.8.2 Màn hình Dashboard

Đây là màn hình mặc định sau khi đăng nhập thành công, đóng vai trò trung tâm điều phối và giám sát tổng quan. Màn hình này nằm cùng cấp độ điều hướng với *Màn hình Thông báo* và *Màn hình Cài đặt hệ thống*.

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.2 Màn hình Dashboard*

Và *Phụ lục B – Hình 5.3 Màn hình tùy chọn hiển thị*

A) Đặc tả chức năng và Tương tác người dùng

- *Điều hướng toàn cục*: Thanh điều hướng dưới đáy (Bottom Navigation Bar) cho phép chuyển nhanh sang các tab *Thông báo* hoặc *Cài đặt chung*.
- *Tương tác tại thẻ Node*: Người dùng nhấn vào nút "Cài đặt" trên thẻ Node, Màn hình chuyển sang màn hình con "Tùy chọn hiển thị"

Tại màn hình này, người dùng tích chọn các cảm biến/thiết bị muốn đưa ra Dashboard.

B) Logic xử lý & Ràng buộc hệ thống.

Cơ chế truy vấn dữ liệu

Khi khởi tạo Dashboard, App đọc dữ liệu cấu hình cục bộ (Local Config/Shared Preferences) để lấy danh sách ID của 4 thiết bị được người dùng chọn cho mỗi Node, sau đó tải dữ liệu từ Cloud tương ứng với các ID này để tối ưu băng thông.

Logic kiểm tra ràng buộc

Tại màn hình "Tùy chọn hiển thị", hệ thống tích hợp bộ đếm thời gian thực (Real-time Counter). Khi người dùng tích chọn thiết bị, biến đếm *selected_count* tăng lên.

Tại màn hình dashboard sẽ kiểm tra nếu *selected_count* > 4, ứng dụng sẽ kích hoạt sự kiện chặn và chỉ hiển thị 4 thiết bị được tích chọn đầu tiên, kèm thông báo "*Vui lòng chỉ chọn hiển thị 4 mục. Nhấn cấu hình để xem toàn bộ mục đã chọn*".

3.8.3 Màn hình Chi tiết Node (Node Detail Screen)

Màn hình này cung cấp giao diện giám sát chuyên sâu và điều khiển thời gian thực cho một Node cụ thể được chọn từ Dashboard. Tại đây, người dùng có thể theo dõi trạng thái kết nối của thiết bị, xem dữ liệu các cảm biến và điều khiển các thiết bị chấp hành tương ứng với Node.

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.4 Màn hình Chi tiết Node*

A) Đặc tả chức năng và Tương tác người dùng

1. Chức năng đổi tên phòng và thiết bị:

Khi nhấn vào biểu tượng chỉnh sửa, App hiển thị một hộp thoại (Dialog) yêu cầu nhập tên mới. Sau khi xác nhận, tên mới được cập nhật vào cơ sở dữ liệu cục bộ và đồng bộ lên Cloud.

2. Cài đặt ngưỡng cảnh báo (Threshold Setting):

Thao tác: Sử dụng sự kiện Nhấn giữ (Long Click/Long Press) vào một ô giá trị cảm biến bất kỳ (ví dụ: ô Nhiệt độ).

Phản hồi: Ứng dụng xuất hiện Pop-up "Cài đặt ngưỡng cảnh báo".

Người dùng nhập giá trị Ngưỡng trên (Max). Nếu giá trị cảm biến vượt quá khoảng này, App sẽ kích hoạt hệ thống thông báo đẩy (Push Notification) để cảnh báo người dùng.

3. Điều khiển thiết bị (Device Control):

Người dùng chạm vào nút công tắc để Bật/Tắt thiết bị. Trạng thái nút nhấn sẽ dữ nguyên trạng thái tắt cho đến khi nhận được gói tin xác thực từ phần cứng gửi về, đảm bảo trạng thái trên App luôn đồng nhất với thực tế.

B) Logic xử lý và Cơ chế ràng buộc.

Đây là phần xử lý logic quan trọng để đảm bảo độ tin cậy của hệ thống:

1. Cơ chế hiển thị trạng thái kép (Dual Status Logic):

App phân tích gói tin trạng thái (Heartbeat) để hiển thị:

- Nếu Gateway = Online và Node = Online -> Hiển thị "Hệ thống Sẵn sàng" (Màu Tím).
- Nếu Gateway = Offline -> Hiển thị "Mất kết nối Gateway" (Màu đỏ), ngầm định Node cũng mất kết nối.
- Nếu Gateway = Online nhưng Node = Offline -> Hiển thị "Lỗi kết nối Node" (Màu Cam), báo hiệu người dùng kiểm tra nguồn điện hoặc sóng tại Node đó.

2. Cơ chế Khóa điều khiển (Control Lock/Interlock Mechanism):

Tránh trường hợp người dùng nhấn nút điều khiển liên tục (spam) khi thiết bị đang mất kết nối, gây tắc nghẽn hàng đợi lệnh khi mạng có lại. Hệ thống liên tục lắng nghe biến trạng thái kết nối. Ngay khi phát hiện trạng thái là Offline (ở Gateway hoặc Node), App lập tức kích hoạt khóa nút điều khiển

Hành vi giao diện:

- Toàn bộ các nút công tắc điều khiển bị làm mờ (Dimmed/Greyscale).
- Vô hiệu hóa tương tác cảm ứng (Touch Disabled).
- Nút nhấn chỉ được kích hoạt lại (Enable) khi trạng thái chuyển về Online.

3.8.4 Màn hình Lịch sử Thông báo (Notification History)

Màn hình này đóng vai trò là nhật ký hoạt động của hệ thống, tập trung lưu trữ và hiển thị các cảnh báo an toàn khi thông số môi trường vượt ngưỡng cài đặt. Đây là thành phần quan trọng giúp người dùng truy vết sự cố theo thời gian thực.

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.5 Màn hình Lịch sử thông báo*

A) Đặc tả chức năng và Tương tác người dùng

1. Chức năng xóa thông báo

Xóa đã đọc: Hệ thống lọc các Item có trạng thái "Đã đọc" và loại bỏ khỏi danh sách hiển thị.

Xóa toàn bộ: Gửi lệnh xóa sạch mảng dữ liệu thông báo liên quan đến User trên Cloud và làm mới giao diện về trạng thái rỗng.

2. Chức năng điều hướng màn hình

App tự động chuyển màn hình sang Màn hình Chi tiết Node tương ứng với Node phát ra cảnh báo đó, giúp người dùng kiểm tra ngay lập tức các thông số hiện tại.

D) Logic xử lý và Cơ chế ràng buộc.

1. Cơ chế đồng bộ dữ liệu

Ứng dụng lắng nghe sự thay đổi của trường notification trên Cloud theo thời gian thực. Dữ liệu trả về là một mảng JSON chứa các ID thông báo. App thực hiện vòng lặp duyệt qua từng ID để render ra danh sách hiển thị trên màn hình.

2. Cơ chế phân loại trạng thái

Mỗi thông báo sẽ đi kèm theo một cờ trạng thái để đánh dấu thông báo đó đã được đọc hay chưa, mỗi khi người dùng nhấn vào thông báo sẽ cập nhật lại trạng thái thành đã đọc cho cờ trạng thái. App sẽ kiểm tra và hiển thị màu tương ứng với trạng thái.

3.8.5 Màn hình Cài đặt Hệ thống

Đóng vai trò là trung tâm điều hướng (Navigation Hub) cho các cấu hình chuyên sâu cấp hệ thống. Chuyển tiếp đến các màn hình cài đặt kịch bản tự động và màn hình chế độ hoạt động cho hệ thống.

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.6 Màn hình cài đặt hệ thống*

A) Đặc tả chức năng và Tương tác người dùng

- Nhóm Cài đặt Tự động hóa (Automation Settings): Nút nhấn chuyển đến màn hình “*Danh sách kịch bản tự động*”
- Nhóm Chế độ Hoạt động (Auto/Manual): Nút nhấn chuyển đến màn hình “*Cài đặt chế độ hoạt động*”

3.8.6 Màn hình quản lý danh sách kịch bản tự động

Đây là giao diện trung tâm cho phép người dùng quản lý các luật thông minh (If-Then) của hệ thống. Tại đây, người dùng có thể xem danh sách các kịch bản đang có, nhanh chóng bật/tắt hiệu lực của chúng hoặc thêm mới/chỉnh sửa các luồng tự động hóa mà không cần can thiệp vào Code.

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.7 Màn hình danh sách kịch bản tự động*

A) Đặc tả chức năng và Tương tác người dùng

Chức năng Bật/ Tắt Kịch bản nhanh: Cho phép người dùng Bật/Tắt nhanh kịch bản ngay trên danh sách mà không cần vào chi tiết. Khi gạt tắt, kịch bản sẽ tạm ngưng hoạt động trên

Chức năng thêm mới: Khi nhấn vào biểu tượng dấu “+”, ứng dụng chuyển tiếp sang “Màn hình Cập nhật/ Tạo kịch bản mới”, bắt đầu quy trình thiết lập quy tắc tự động hóa.

3.8.7 Màn hình Cập nhật/ Tạo kịch bản tự động

Cung cấp giao diện trực quan để người dùng thiết lập các luồng logic tự động hóa theo nguyên tắc "IF - THEN" (Nếu - Thì). Màn hình này được sử dụng chung cho cả hai tác vụ: Tạo mới kịch bản và Cập nhật kịch bản đã có.

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.8 Màn hình Tạo và chỉnh sửa kịch bản tự động*

A) Đặc tả chức năng và Tương tác người dùng

Chức năng phân biệt ngữ cảnh (Contextual Action):

Trường hợp Tạo mới: Màn hình trống dữ liệu, nút tác vụ cuối màn hình hiển thị nhãn "LƯU KỊCH BẢN". Khi nhấn, hệ thống tạo một ID kịch bản mới.

Trường hợp Cập nhật: Màn hình tự động điền sẵn toàn bộ dữ liệu của kịch bản cũ. Nút tác vụ hiển thị nhãn "CẬP NHẬT KỊCH BẢN". Khi nhấn, hệ thống ghi đè thông tin mới lên ID kịch bản hiện tại

3.8.8 Màn hình Chế độ hoạt động

Cung cấp giao diện trực quan để người dùng thiết lập chế độ hoạt động dành cho hệ thống, với các nút gạt và ghi chú giải thích về tác dụng của chúng

Tham khảo giao diện thực tế tại *Phụ lục B – Hình 5.9 Màn hình chế độ hoạt động*

A) Đặc tả chức năng và Tương tác người dùng

Chế độ Tổng thể (Global Mode): Đóng vai trò là "công tắc tổng" cho kịch bản tự động toàn nhà. Khi ở trạng thái "Thủ công", mọi kịch bản tự động tại các Node đơn lẻ sẽ bị vô hiệu hóa.

Chế độ theo Node (Local Mode): Cho phép bật/tắt logic tự động riêng biệt cho từng khu vực (Phòng khách, Bếp, Phòng ngủ) dựa trên nhu cầu sử dụng thực tế tại mỗi phòng.

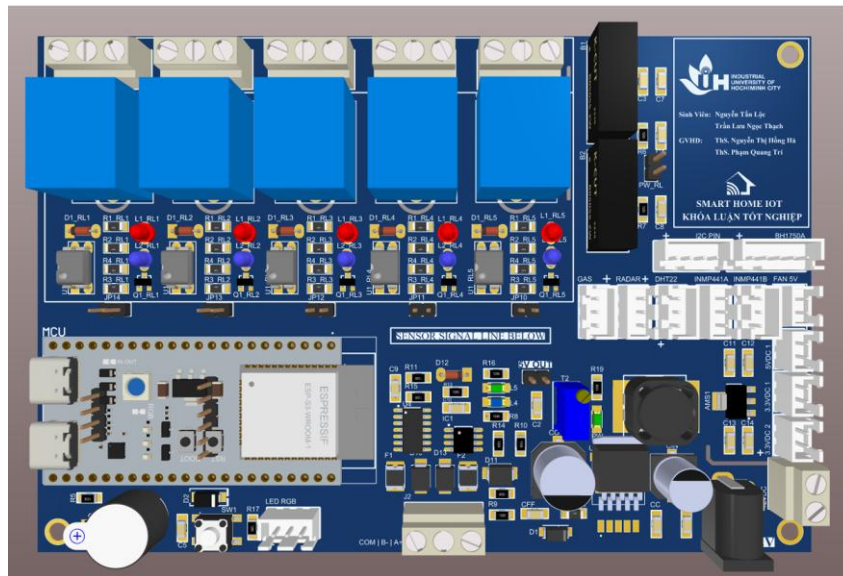
Cơ chế Đồng bộ Kép: Đảm bảo tính nhất quán dữ liệu. Nếu một Node bất kỳ chuyển sang "Thủ công", Global Mode sẽ tự động chuyển về "Thủ công". Ngược lại, Global Mode chỉ kích hoạt "Tự động" khi tất cả các Node thành phần đều ở trạng thái này.

Tương tác Người dùng: Sử dụng các công tắc gạt (Switch) để thay đổi trạng thái, kèm theo phản hồi văn bản tức thời ngay dưới tiêu đề từng mục.

CHƯƠNG 4 THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

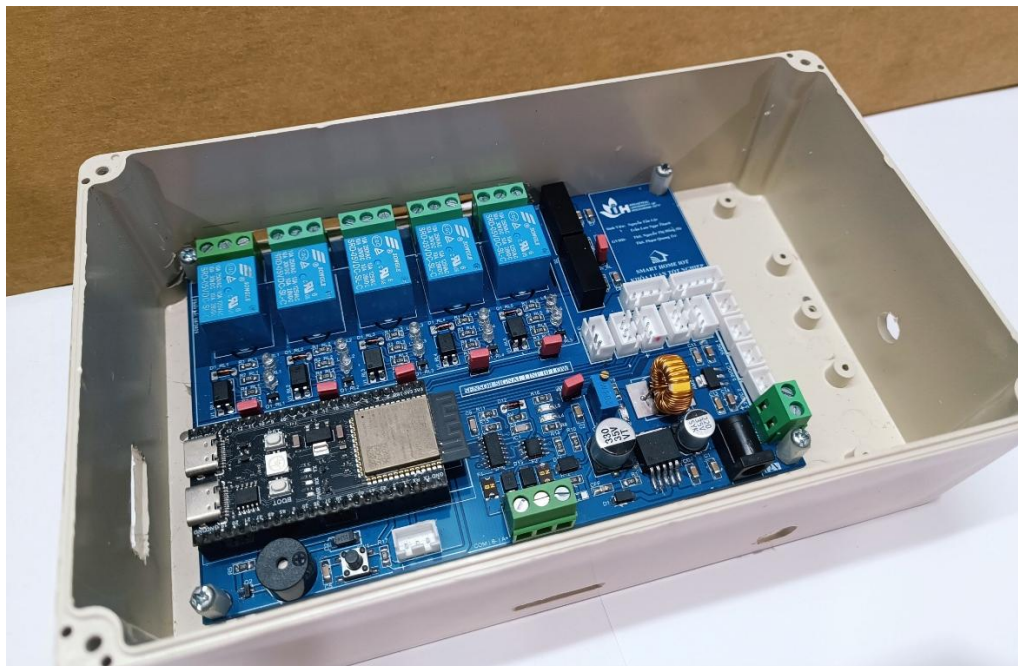
4.1 Thực nghiệm

4.1.1 Thiết kế mạch in PCB



Hình 4. 1 Mạch in PCB 3D

4.1.2 Lắp đặt/ hàn linh kiện phần cứng các Node



Hình 4. 2 Mạch PCB phần cứng cho Node thực tế

4.1.3 Thiết kế mô hình tương ứng với các Node

A) Node phòng khách

Mô hình được thiết kế theo số lượng thiết bị được liệt kê tại mục 3.5.7 bao gồm các cảm biến: 1 cảm biến DHT22, 1 cảm biến âm thanh INMP441 và thiết bị chấp hành gồm: 4 đèn LED AC 220V tượng trưng các thiết bị điện trong nhà, 1 Node rèm cửa thông minh được trình bày tại mục B 3.5.4



Hình 4. 3 Mô hình Node phòng khách

B) Node phòng bếp

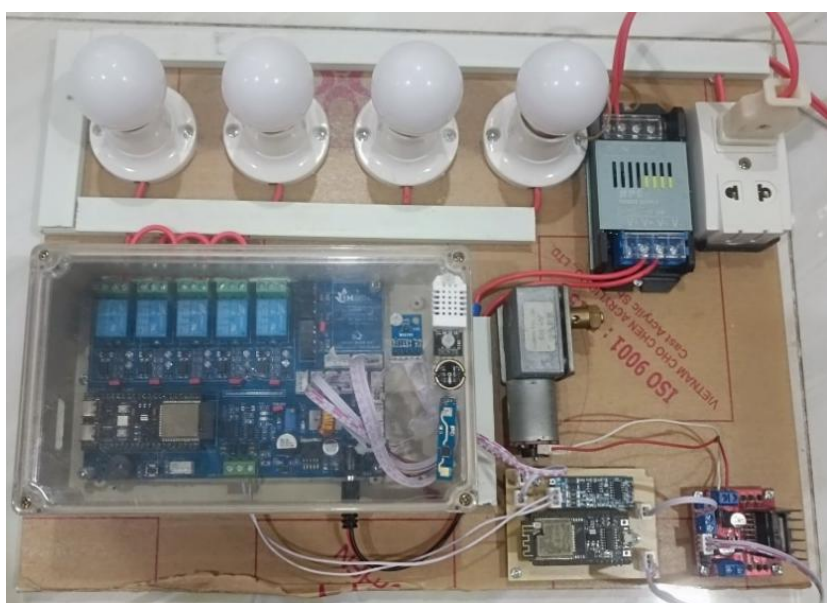
Mô hình được thiết kế theo số lượng thiết bị được liệt kê tại mục 3.5.7 bao gồm các cảm biến: 1 cảm biến DHT22, 1 cảm biến âm thanh INMP441, 1 cảm biến khí gas MQ2 và thiết bị chấp hành gồm: 4 đèn LED AC 220V tượng trưng các thiết bị điện.



Hình 4. 4 Mô hình Node phòng bếp

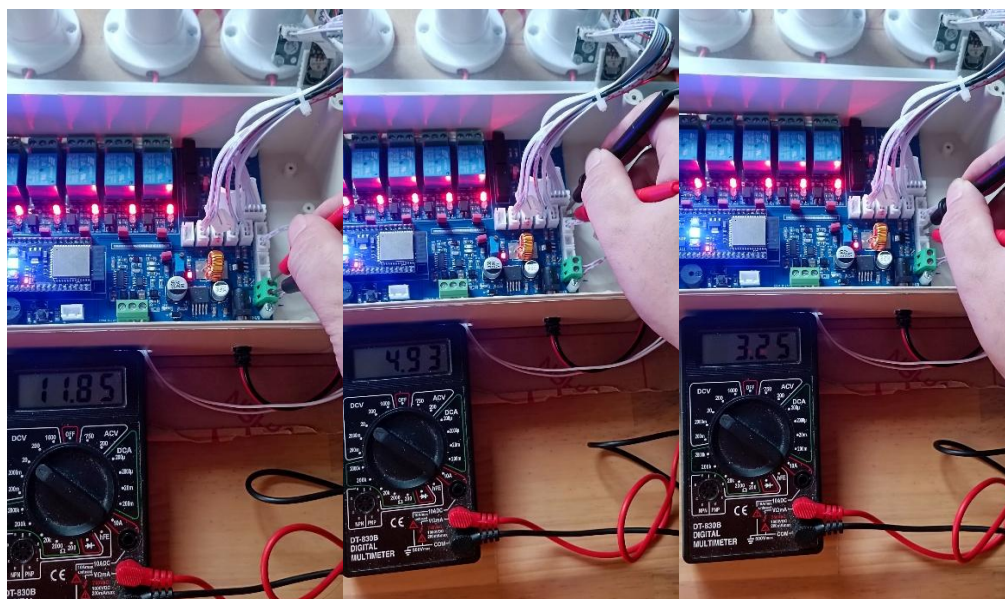
D) Node phòng ngủ

Mô hình được thiết kế theo số lượng thiết bị được liệt kê tại (mục 3.5.7) bao gồm các cảm biến: 1 cảm biến DHT22, 1 cảm biến âm thanh INMP441, 1 cảm biến ánh sáng BH1750, 1 cảm biến Radar và thiết bị chấp hành gồm: 4 đèn LED AC 220V tương trưng các thiết bị điện trong nhà. 1 Node rèm cửa thông minh được trình bày tại (mục B 3.5.4).



Hình 4. 5 Mô hình Node phòng ngủ

4.1.4 Kiểm thử các kết quả tính toán cho mạch nguồn



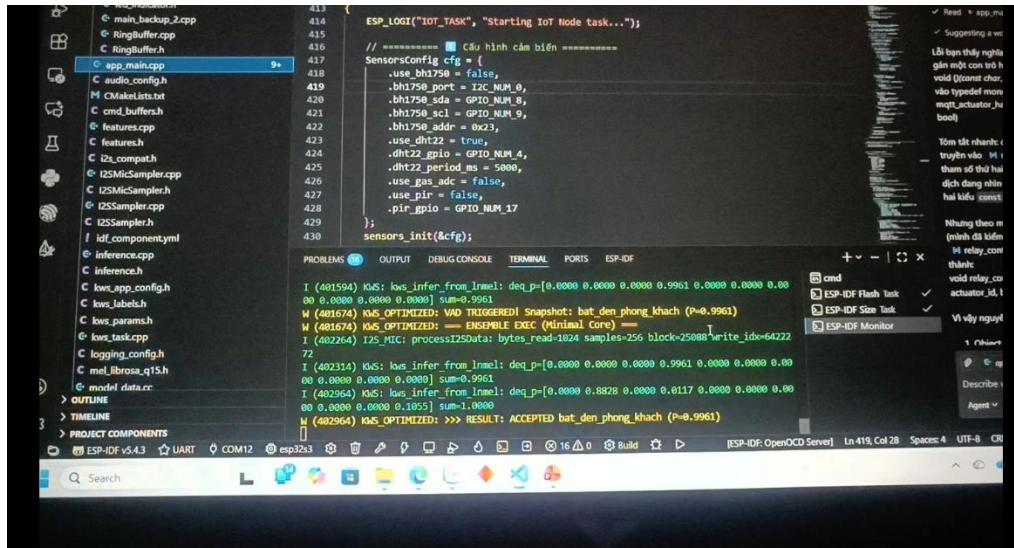
Hình 4. 6 Kết đo đo điện áp đầu ra thực tế của mạch

Dựa vào những tính toán tại Mục 3.5.10.1 Sơ đồ nguyên lý mạch nguồn giảm áp 12V xuống 5V sử dụng IC XL4015 cho kết quả đầu ra tương ứng với mức điện áp xấp xỉ 5V và tại Mục 3.5.10.5 Sơ đồ nguyên lý mạch giảm áp 5V xuống 3.3V sử dụng IC AMS1117 cho ra kết quả điện áp đầu ra ở mức xấp xỉ 3.3V

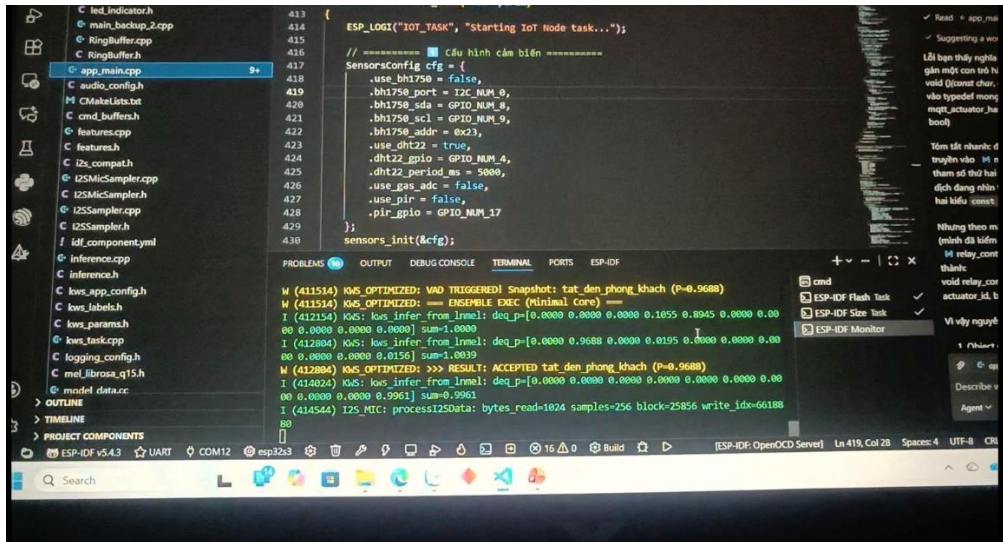
4.1.5 Thử nghiệm nhận diện giọng nói và quyết định nhận chính xác

Hệ thống được đánh giá trực tiếp trên mô hình trong điều kiện sử dụng thực tế. Tuy nhiên, do số lượng nhãn dán nhiều vì thế không thể đưa vào báo cáo hết được, bảng khảo sát sẽ thực hiện dựa trên cặp lệnh có độ khác nhau ít như: *bật đèn phòng khách* và *tắt đèn phòng khách*. Lệnh giọng nói được khảo sát như sau:

- Thu 25 lần ở khoảng cách 0.5–1 m.
- Cùng một người nói.
- Trong không gian ít nhiễu (không gian thu thập dữ liệu huấn luyện trước đó).
- Kiểm tra trực tiếp trên mô hình và truyền về máy tính để quan sát.



Hình 4. 7 Nhận diện giọng nói và quyết định nhấn *bật_đèn_phòng_khách*



Hình 4. 8 Nhận diện giọng nói và quyết định nhấn *tắt_đèn_phòng_khách*

Quá trình nhận diện lệnh giọng nói được thực hiện theo đúng trình tự đã trình bày tại Mục 3.6.4.4. Trước hết, hệ thống phát hiện cửa sổ chứa lệnh giọng nói bằng cách chọn nhấn có xác suất cao nhất tại thời điểm kích hoạt (trigger). Sau đó, bước lọc hậu xử lý được áp dụng bằng cách so sánh ba cửa sổ liên tiếp gồm cửa sổ trước thời điểm trigger, tại thời điểm trigger và sau thời điểm trigger. Thuật toán lựa chọn giá trị xác suất lớn nhất trong ba cửa sổ này để đưa ra kết quả quyết định cuối cùng được chấp nhận.

Bảng 4. 1 Bảng khảo sát số lần nhận chính xác nhãn *bật_đèn_phòng_khách*

Lượt	Nhãn khi nói	Nhãn trước khi lọc	Nhãn sau khi lọc
1	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
2	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
3	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
4	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	Hủy lệnh
5	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
6	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_ngủ</i>	<i>bật_đèn_phòng_khách</i>
7	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	Hủy lệnh
8	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_bếp</i>	<i>bật_đèn_phòng_khách</i>
9	<i>bật_đèn_phòng_khách</i>	Không nhận lệnh	Hủy lệnh
10	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_bếp</i>	<i>bật_đèn_phòng_khách</i>
11	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
12	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_bếp</i>	<i>bật_đèn_phòng_khách</i>
13	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_bếp</i>
14	<i>bật_đèn_phòng_khách</i>	Không nhận lệnh	Hủy lệnh
15	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
16	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
17	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
18	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_bếp</i>	Hủy lệnh
19	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
20	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
21	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
22	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_bếp</i>	<i>bật_đèn_phòng_khách</i>
23	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
24	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
25	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
Tổng số lần quyết định đúng: bao gồm việc hủy lệnh			21

Bảng 4. 2 Bảng khảo sát số lần nhận chính xác nhãn *tắt_đèn_phòng_khách*

Lượt	Nhãn khi nói	Nhãn trước khi lọc	Nhãn sau khi lọc
1	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
2	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
3	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_bếp</i>	<i>tắt_đèn_phòng_khách</i>
4	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_bếp</i>	Hủy lệnh
5	<i>tắt_đèn_phòng_khách</i>	<i>tắt_quạt_phòng_ngủ</i>	<i>tắt_quạt_phòng_ngủ</i>
6	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
7	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
8	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	Hủy lệnh
9	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_bếp</i>	Hủy lệnh
10	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
11	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_bếp</i>	<i>bật_đèn_phòng_khách</i>
12	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
13	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
14	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
15	<i>tắt_đèn_phòng_khách</i>	Không nhận lệnh	Hủy lệnh
16	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_bếp</i>	<i>tắt_đèn_phòng_khách</i>
17	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
18	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
19	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_bếp</i>	Hủy lệnh
20	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>bật_đèn_phòng_khách</i>
21	<i>tắt_đèn_phòng_khách</i>	Không nhận lệnh	Hủy lệnh
22	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
23	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_bếp</i>
24	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
25	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>	<i>tắt_đèn_phòng_khách</i>
Tổng số lần quyết định đúng: bao gồm việc hủy lệnh			20

- **Đối với nhãn `bật_đèn_phòng_khách`**

Tỉ lệ quyết định đúng của mô hình và thuật toán lọc là $\frac{21}{25} \times 100\% = 84\%$

Trong đó, các nhãn bị nhầm lẫn chủ yếu là `tắt_đèn_phòng_khách` và `bật_đèn_phòng_bếp`. Nguyên nhân xuất phát từ sự tương đồng cao về đặc trưng âm học giữa các từ “tắt” và “bếp”, đặc biệt trong một số cách phát âm và điều kiện thu âm thực tế. Sự tương đồng này dẫn đến các biểu diễn Log-Mel Spectrogram của các lệnh có phân bố phổ gần nhau, khiến mô hình CNN khó phân biệt rõ ràng giữa các nhãn, từ đó làm tăng xác suất suy luận sai.

- **Đối với nhãn `tắt_đèn_phòng_khách`**

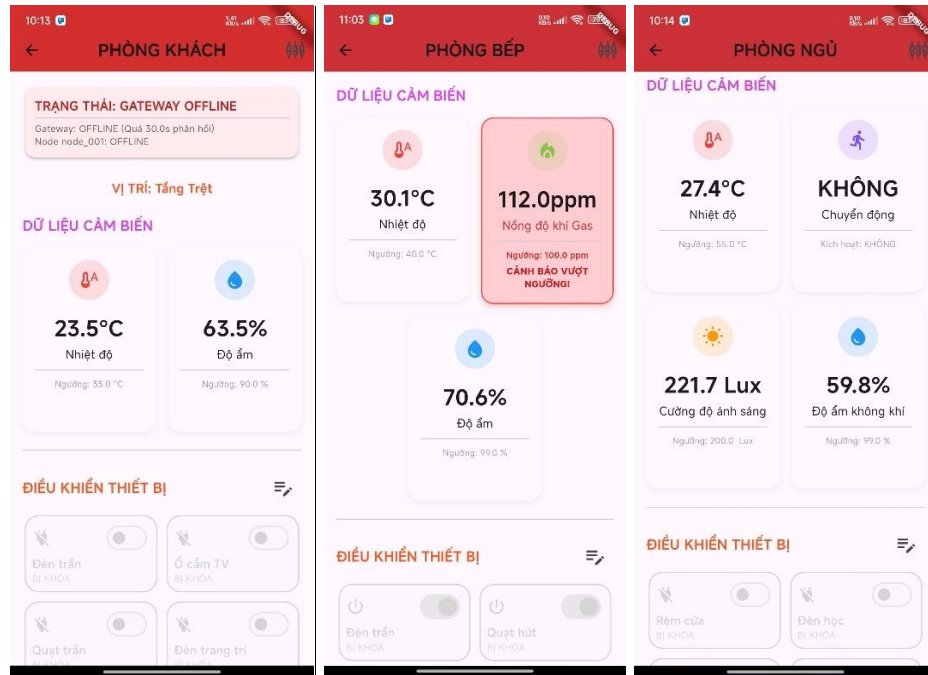
Tỉ lệ quyết định đúng của mô hình và thuật toán lọc là $\frac{20}{25} \times 100\% = 80\%$

Kết quả phân tích cho thấy nhãn `tắt_đèn_phòng_khách` thường bị nhận nhầm thành `bật_đèn_phòng_khách`. Nguyên nhân chủ yếu đến từ sự tương đồng cao về đặc trưng âm học giữa hai câu lệnh, khi phần lớn âm tiết là giống nhau và chỉ khác biệt ở từ khóa ngắn “bật/tắt” ở đầu câu. Do thời lượng và năng lượng của từ khóa này thấp, mô hình CNN khó phân biệt trong điều kiện nhiễu hoặc phát âm không rõ, dẫn đến xu hướng dự đoán theo phần câu chiếm ưu thế.

Nhận xét: Các nhãn có đặc trưng âm học gần tương đồng thường bị nhầm lẫn do chỉ khác biệt ở một số âm tiết ngắn, trong khi phần còn lại của câu lệnh có cấu trúc tương tự nhau. Biểu diễn Log-Mel Spectrogram của các nhãn này có phân bố gần nhau, dẫn đến tỉ lệ nhận nhầm tương đương và không có sự chênh lệch rõ rệt. Hiện tượng này phản ánh giới hạn của mô hình tinh gọn khi triển khai trên hệ thống nhúng.

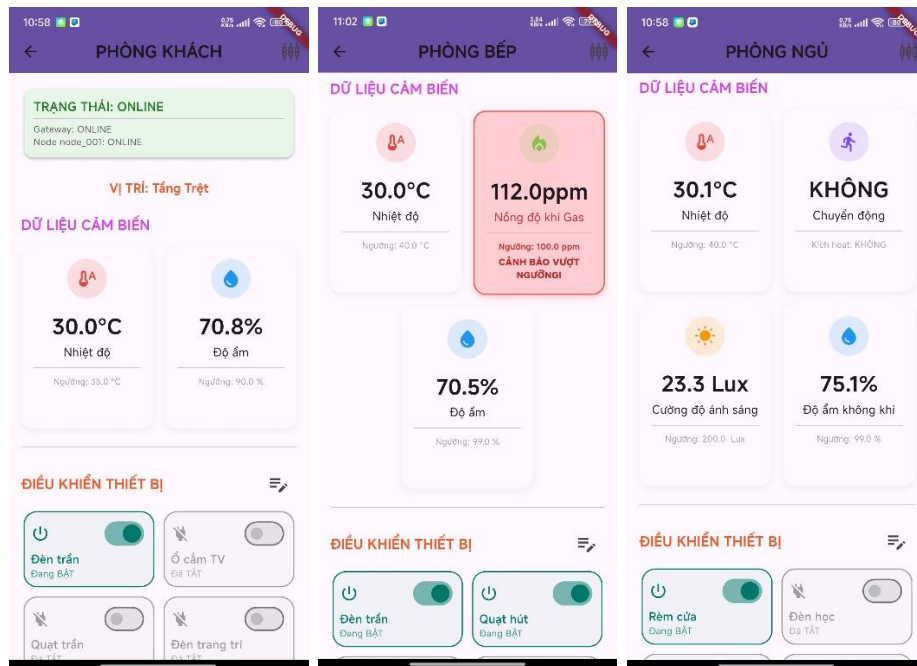
4.1.6 Kiểm thử chức năng theo dõi môi trường và điều khiển trạng thái các node

Tại mục này sẽ kiểm tra khả năng hiển thị dữ liệu đo được từ cảm biến trên giao diện ứng dụng App, đồng thời kiểm thử các chức năng khác như: Trạng thái online/offline các Node, bảng điều khiển bị vô hiệu hóa khi offline không.



Hình 4. 9 Trạng thái hiển thị của các Node khi Offline

Khi Gateway và Node offline màn hình hiển thị sẽ tự động khóa các nút nhấn điều khiển thiết bị và đổi màu giao diện chủ đạo sang màu đỏ theo đúng phân tích tại Mục B 3.8.3 Màn hình Chi tiết Node (Node Detail Screen)



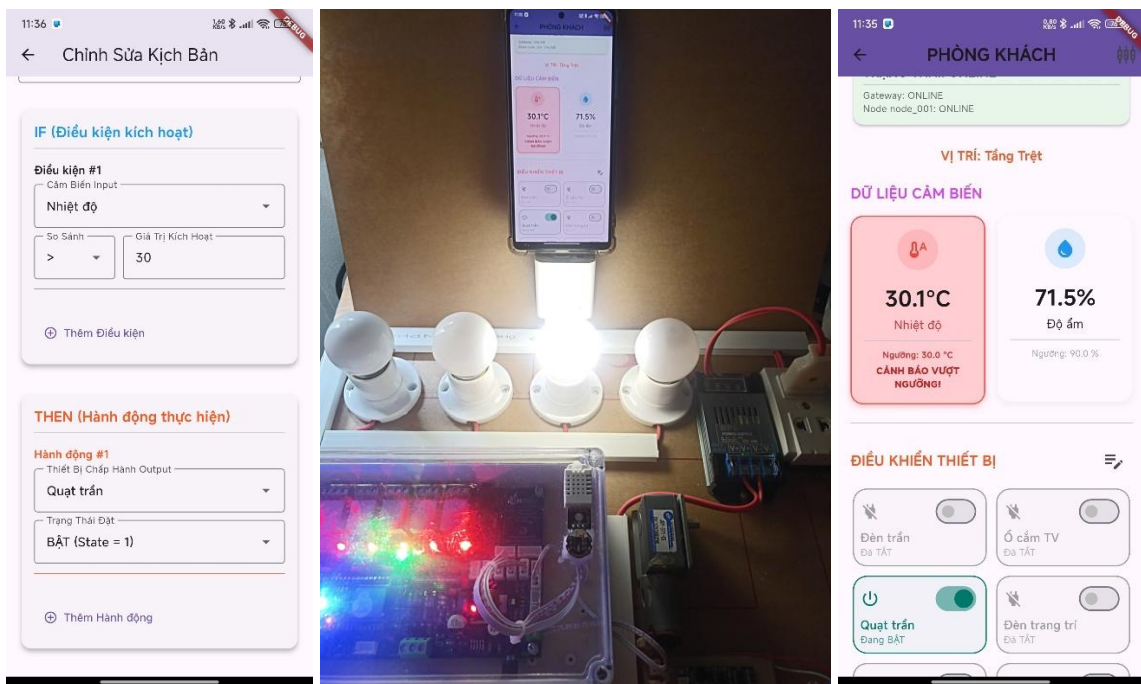
Hình 4. 10 Trạng thái hiển thị của các Node khi Online

Khi Node online, giao diện cho phép người dùng quan sát giá trị của các cảm biến tại thời điểm đó và điều khiển thiết bị một cách trực quan, đồng thời có thể cài đặt các ngưỡng để cảnh báo. Khi cảm biến đó được vượt quá ngưỡng cài đặt, màn hình sẽ tô đỏ nền tại cảm biến đó để người dùng dễ dàng nhận thấy.

4.1.7 Kiểm thử chức năng tự động hóa

4.1.7.1 Kịch bản đơn (1 cảm biến - 1 chấp hành)

Giả sử tạo kịch bản đơn (1 cảm biến - 1 chấp hành) có nội dung như sau tại Node phòng khách: Nếu nhiệt độ tại phòng đo được lớn hơn 30 độ, thì quạt trần bật Relay thứ 3 như hình minh họa:



Hình 4. 11 Kiểm thử chức năng tự động hóa kịch bản đơn

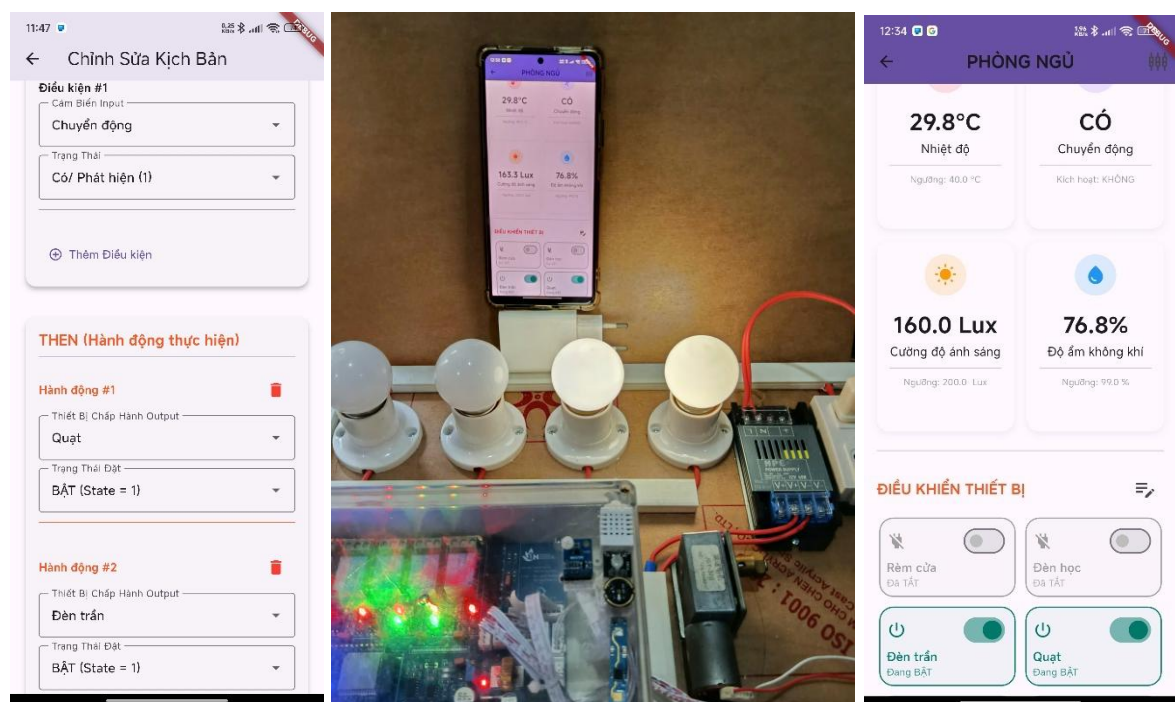
Nhận xét: Kết quả thực nghiệm cho thấy hệ thống hoạt động đúng theo kịch bản đã thiết kế và tuân thủ đầy đủ chế độ vận hành do người dùng thiết lập.

Ở chế độ **Auto**, hệ thống ưu tiên điều kiện môi trường, do đó ngay cả khi người dùng đã tắt thiết bị thủ công, thiết bị vẫn sẽ được tự động bật trở lại nếu các lần đo tiếp theo vượt ngưỡng nhiệt độ cho phép.

Ngược lại, ở chế độ **Manual**, hệ thống ưu tiên thao tác của người dùng; khi thiết bị đã được tắt, các lần đo sau dù vượt ngưỡng cũng không kích hoạt lại thiết bị cho đến khi có lệnh điều khiển mới từ người dùng.

4.1.7.2 Kịch bản kép (1 cảm biến - 2 chấp hành) hoặc nhiều hơn (nhiều cảm biến – nhiều chấp hành)

Giả sử tạo kịch bản kép (1 cảm biến – 2 chấp hành) có nội dung như sau tại Node phòng ngủ: Nếu cảm biến phát hiện sự hiện diện con người, thì kích hoạt bật đèn (Relay thứ 3) và bật quạt (Relay thứ 4) như hình minh họa:



Hình 4. 12 Kiểm thử chức năng tự động hóa kịch bản kép

Nhận xét: Kết quả thực nghiệm cho thấy kịch bản kép tại Node phòng ngủ hoạt động đúng theo thiết kế. Khi cảm biến phát hiện sự hiện diện của con người, hệ thống đồng thời kích hoạt hai cơ cấu chấp hành gồm đèn (Relay 3) và quạt (Relay 4). Việc điều khiển đồng thời nhiều thiết bị từ một cảm biến được thực hiện ổn định, không xảy ra xung đột lệnh hay độ trễ đáng kể.

KẾT LUẬN VÀ KIẾN NGHỊ

1. Kết quả đạt được

Trong phạm vi đề tài, nhóm đã xây dựng và triển khai thành công module nhận dạng lệnh giọng nói tiếng Việt chạy trực tiếp trên thiết bị (on-device), phục vụ điều khiển hệ thống nhà thông minh theo mô hình IoT. Hệ thống được thiết kế theo chuỗi xử lý gồm: thu nhận tín hiệu âm thanh, tiền xử lý và trích xuất đặc trưng (log-mel), suy luận bằng mô hình học máy gọn nhẹ, kết hợp cơ chế quyết định theo trạng thái nhằm đảm bảo tính ổn định khi vận hành trên thiết bị nhúng.

Module giọng nói hoạt động theo cơ chế hai giai đoạn: phát hiện từ khóa kích hoạt và chuyển sang chế độ nhận lệnh trong một khoảng thời gian giới hạn. Cách tiếp cận này giúp giảm kích hoạt sai, đồng thời chỉ xử lý sâu khi có tương tác chủ động từ người dùng, phù hợp với yêu cầu tiết kiệm tài nguyên và phản hồi nhanh trong hệ thống IoT.

Về mặt IoT và điều khiển, hệ thống hỗ trợ hai chế độ vận hành song song: Điều khiển thủ công từ ứng dụng, cho phép người dùng giám sát cảm biến và trạng thái thiết bị và gửi lệnh điều khiển trực tiếp thông qua mạng IoT. Điều khiển tự động theo kịch bản, trong đó các thiết bị được kích hoạt dựa trên điều kiện định sẵn hoặc lệnh giọng nói, giảm sự phụ thuộc vào thao tác thủ công.

Nhìn chung hệ thống đảm bảo khả năng phản hồi nhanh, vận hành ổn định và phù hợp với các yêu cầu điều khiển cơ bản trong bối cảnh nhà thông minh.

2. Hạn chế của đề tài

Mặc dù đạt được kết quả mong muốn, hệ thống vẫn tồn tại một số hạn chế:

- Hạn chế về lưu trữ và quản lý dữ liệu IoT: Hệ thống hiện chưa tích hợp cơ chế lưu trữ dữ liệu trên cloud, do đó chưa hỗ trợ xem lại lịch sử lệnh điều khiển,

trạng thái thiết bị hoặc thống kê hoạt động theo thời gian. Điều này làm giảm khả năng giám sát và phân tích hệ thống trong vận hành dài hạn.

- Hạn chế về phạm vi nhận dạng giọng nói: Mô hình nhận dạng được tối ưu cho tập lệnh và điều kiện triển khai cụ thể. Trong môi trường nhiễu mạnh hoặc với người dùng có giọng nói khác biệt so với dữ liệu huấn luyện, độ chính xác có thể suy giảm. Đây là hạn chế phổ biến của các hệ thống nhận dạng giọng nói on-device trong bối cảnh IoT nhúng.

Về phần cứng, hệ thống hoạt động ổn định và không ghi nhận hạn chế đáng kể ảnh hưởng trực tiếp đến mục tiêu đề tài.

3. Kiến nghị và hướng phát triển

Để nâng cao tính ứng dụng và mở rộng hệ thống trong tương lai, nhóm đề xuất một số hướng phát triển sau:

- Bổ sung cơ chế lưu trữ và nhật ký IoT: Ghi nhận lịch sử lệnh điều khiển và trạng thái thiết bị tại gateway hoặc trên cloud nhằm phục vụ giám sát, truy vết và phân tích hệ thống.
- Nâng cao độ bền vững của nhận dạng giọng nói: Mở rộng dữ liệu huấn luyện với nhiều giọng nói và điều kiện nhiễu; bổ sung dữ liệu unknown/noise và điều chỉnh ngưỡng nhận dạng thích nghi theo môi trường triển khai.
- Tối ưu phản hồi và vận hành thời gian thực: Hoàn thiện cơ chế phản hồi cho người dùng (đèn báo/âm thanh) và tối ưu các tham số như ngưỡng tin cậy, timeout và làm mượt theo thời gian để phù hợp hơn với hệ thống IoT thời gian thực.
- Mở rộng chức năng điều khiển bằng giọng nói: Mở rộng tập lệnh, chuẩn hóa quy tắc đặt lệnh và bổ sung cơ chế xác nhận đối với các thao tác quan trọng trong hệ thống nhà thông minh.

TÀI LIỆU THAM KHẢO

- [1] T. N. Sainath and C. Parada, "Convolutional Neural Networks for Small-footprint Keyword Spotting," Proc. Interspeech, pp. 1478-1482, 2015.
- [2] Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," arXiv:1711.07128, 2017.
- [3] P. M. Sørensen et al., "A depthwise separable convolutional neural network for keyword spotting on an embedded system," EURASIP J. Audio Speech Music Process., vol. 2020, no. 1, 2020.
- [4] J. Bae et al., "FPGA Implementation of Keyword Spotting System Using Depthwise Separable Binarized and Ternarized Neural Networks," Sensors, vol. 23, no. 12, 5701, 2023.
- [5] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," arXiv:1804.03209, 2018.
- [6] "TinyML-Based Vietnamese Keyword Recognition for Smart Home Voice Control," J. Sci. Technol. Inf. Commun. (JSTIC), 2024. [Online]. Available: <https://jstic.ptit.edu.vn/jstic-ptit/index.php/jstic/article/view/1668> (truy cập: 17/12/2025).
- [7] G. Ciccarelli et al., "Challenges and Opportunities in Multi-device Speech Processing," Proc. Interspeech, 2022.
- [8] H. T. Luong and H. Q. Vu, "A non-expert Kaldi recipe for Vietnamese Speech Recognition System," Proc. WLSI/OIAF4HLT, 2016.
- [9] N. C. Ngôn, T. T. Hùng, và L. C. Quyền, "Ứng dụng mạng nơ-ron nhân tạo để điều khiển thiết bị bằng giọng nói tiếng Việt," Tạp chí Khoa học Đại học Cần Thơ, số 12, 2009.

- [10] P. D. Hung, L. T. Kien, and N. D. Huy, "Vietnamese Speech Command Recognition using Recurrent Neural Networks," *Int. J. Adv. Comput. Sci. Appl. (IJACSA)*, vol. 10, no. 7, 2019.
- [11] Q. H. Nguyen and T. D. Cao, "A Novel Method for Recognizing Vietnamese Voice Commands on Smartphones with Support Vector Machine and Convolutional Neural Networks," *Hum.-centric Comput. Inf. Sci.*, vol. 10, 2020.
- [12] "Nhận dạng giọng nói tiếng Việt bằng Logic mờ," *Tạp chí Khoa học Công nghệ*. [Online]. Available: <https://media.neliti.com/media/publications/448410-using-fuzzy-logic-in-vietnamese-speech-r-72093e6c.pdf> (truy cập: 17/12/2025).
- [13] "Nhà thông minh điều khiển thiết bị bằng Web Server và giọng nói trên Raspberry Pi," *Tạp chí Khoa học Công nghệ - ĐH Công nghiệp Hà Nội*. [Online]. Available: <https://khcn.hau.edu.vn/media/30/uffile-upload-no-title30775.pdf> (truy cập: 18/12/2025).
- [14] "Deep Learning-based Automatic Control System Using Audio Signal," *TNU J. Sci. Technol.*, 2023. [Online]. Available: <https://jst.tnu.edu.vn/jst/article/download/12357/pdf> (truy cập: 21/12/2025).
- [15] M. Xu and X. Zhang, "Depthwise Separable Convolutional ResNet with Squeeze-and-Excitation Blocks for Small-Footprint Keyword Spotting," *Proc. Interspeech*, 2020.
- [16] J. S. Albuquerque et al., "A review of Internet of Things for smart home: Challenges and solutions," *J. Cleaner Prod.*, vol. 140, pp. 1454-1461, 2017.
- [17] F. Seraj et al., "Smart Home Automation—Use Cases of a Secure and Integrated Voice-Control System," *Systems*, vol. 9, no. 4, 77, 2021.
- [18] Google, *Firebase Admin SDK Documentation*, Google Developers. [Online]. Available: <https://firebase.google.com/docs/admin>. (truy cập: 21/12/2025).

- [19] Google, Firebase Client SDK Documentation, Google Developers. [Online]. Available: <https://firebase.google.com/docs>. (truy cập: 21/12/2025).
- [20] Google, Firebase Cloud Messaging (FCM) Documentation, Google Developers. [Online]. Available: <https://firebase.google.com/docs/cloud-messaging> (truy cập: 21/12/2025).
- [21] R. David et al., “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems,” Proceedings of the 5th Conference on Machine Learning and Systems (MLSys), 2021.

PHỤ LỤC

A - PHẦN CỨNG

1) Tính toán các giá trị cho Relay

Hai điện trở (R_1 và R_2) được tính toán dựa trên công thức: $R_{LED} = \frac{V_{nguồn} - V_f}{I_f}$ (với $V_{nguồn} = 5 V$; $V_f = 2 V$ là là điện áp tối thiểu để cho phép dòng điện chạy qua linh kiện theo chiều thuận; $I_f = 10 mA$ là dòng điện tối ưu cho LED 5V).

$$R_{LED} = \frac{5 V - 2 V}{0.010 A} = 300 \text{ Ohm}$$

Lựa chọn giá trị chuẩn gần nhất cho (R_1 và R_2) là 330 Ohm.

Điện trở Đầu vào Optocoupler (Chân 1 PC817: $R_{in} = 470 \text{ Ohm}$) dựa trên công thức:

$$I_f = \frac{V_{nguồn} - V_f}{R_{in}}; \text{ Với } V_{nguồn} = 3.3 V; V_f = 1.2 V$$

$$I_f = \frac{3.3 V - 1.2 V}{470 \text{ Ohm}} = 4.15 mA$$

Dòng điện này cung cấp đủ năng lượng để kích hoạt PC817 mà vẫn giữ được mức tiêu thụ thấp, phù hợp với tiêu chuẩn cấp nguồn của chân GPIO trên ESP32

Điện trở R_4 hay R_{base} Được tính toán dựa trên sự đảm bảo chuyển mạch Relay dứt khoát, giảm thiểu nhiệt năng tiêu tán, và cung cấp khả năng chống nhiễu điện từ vượt trội và tin cậy. Nó thiết lập hệ số an toàn bão hòa sâu của transistor

Thông qua xác định hệ số bão hòa tối thiểu (β_{sat_min}) của transistor 2N3904 được công bố từ datasheet từ nhà sản xuất dùng để đảm bảo bão hòa

$$\beta_{sat_min} = \frac{I_C}{I_B} = \frac{50 mA}{5 mA} = 10$$

$$\text{Tính toán Dòng Base Tối thiểu } I_{B_min} = \frac{I_{C_actual}}{\beta_{sat_min}} = \frac{70 mA}{10} = 7 mA$$

Quyết định hệ số an toàn thiết kế để đạt bão hòa sâu và chống nhiễu ta chọn hệ số dự phòng là 2 lần so với mức tối thiểu

Dòng Base mục tiêu $I_{B_target} = 2 \times I_{B_min} = 2 \times 7 \text{ mA} = 14 \text{ mA}$

Tính toán $R_{base} = \frac{V_{in} - V_{BE,sat}}{I_{B_target}} = \frac{4.8 \text{ V} - 0.95 \text{ V}}{0.014 \text{ A}} = 275 \text{ Ohm}$ (Với $V_{in} = 5 \text{ V} - 0.2 \text{ V}$ của PC817 khi kích; $V_{BE,sat} = 0.95 \text{ V}$ theo công bố của nhà sản xuất)

Giá trị điện trở chuẩn được lựa chọn là $R_{base} = 220 \text{ Ohm}$ (Giá trị chuẩn gần nhất và nhỏ hơn 275 Ohm).

Việc lựa chọn R_{base} là một quyết định kỹ thuật tối ưu. Nó thiết lập Hệ số An toàn Thiết kế là $\beta = 4$ (Bão hòa Sâu), giúp Transistor 2N3904 hoạt động chắc chắn trong vùng bão hòa.

2) Tính toán phân tích dự phòng dòng điện

Giả sử công suất tiêu thụ đỉnh của mỗi Node được tính $P_{peak} = 5 \text{ W}$ tại mức điện áp đầu ra 5 V.

Phương án 1: Cấp nguồn trực tiếp 5 V

Dòng điện mà bộ nguồn Adapter 5 V phải cung cấp được xác định theo:

$$I_{5V} = \frac{P_{peak}}{5} = \frac{5}{5} = 1 \text{ A}$$

Trong trường hợp này, bộ nguồn làm việc gần với giới hạn dòng danh định, do đó biên dự phòng dòng (current margin) nhỏ, đặc biệt bất lợi khi tải xuất hiện các xung dòng ngắn hạn do vi điều khiển hoặc relay đóng cắt.

Phương án 2: Cấp nguồn 12 V và hạ áp bằng Buck Converter

Với cùng mức công suất đỉnh $P_{peak} = 5 \text{ W}$, giả sử hiệu suất của mạch hạ áp là: $\eta \approx 90\%$

Dòng điện rút ra từ nguồn 12 V được tính như sau:

$$I_{12V} = \frac{P_{\text{peak}}}{12 \times \eta} = \frac{5}{12 \times 0.90} \approx 0.463 \text{ A}$$

Tỷ lệ dòng điện giữa hai phương án:

$$\frac{I_{5V}}{I_{12V}} = \frac{1}{0.463} \approx 2.16$$

Kết luận: Dòng điện phía 12 V chỉ bằng ~46% so với cấp trực tiếp 5 V.

3) Độ ổn định Điện áp (Voltage Stability).

Sụt áp trên đường dây tuân theo:

$$\Delta V = I \times R_{\text{line}}$$

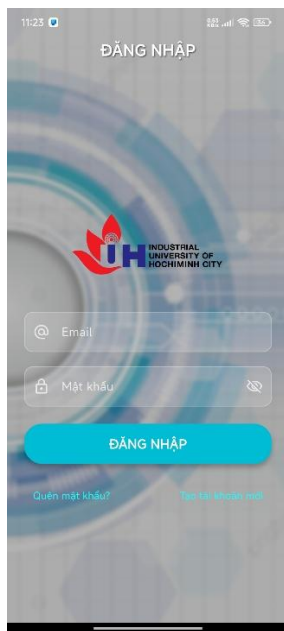
Với cùng điện trở dây dẫn:

- Đường 5 V: $\Delta V_5 \propto 1 \text{ A}$
- Đường 12 V: $\Delta V_{12} \propto 0.463 \text{ A}$

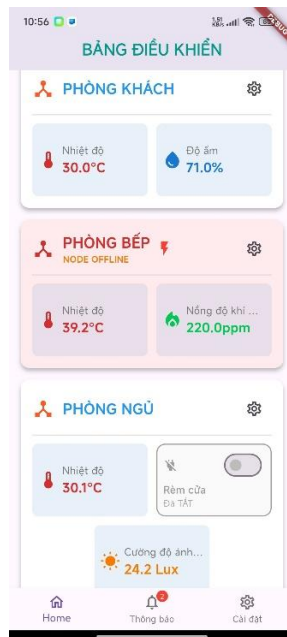
$$\Rightarrow \Delta V_{12} \approx 0.46 \cdot \Delta V_5$$

Như vậy: sụt áp trên đường 12 V nhỏ hơn hơn 2 lần, giúp điện áp đầu vào của Buck Converter ổn định hơn khi tải thay đổi.

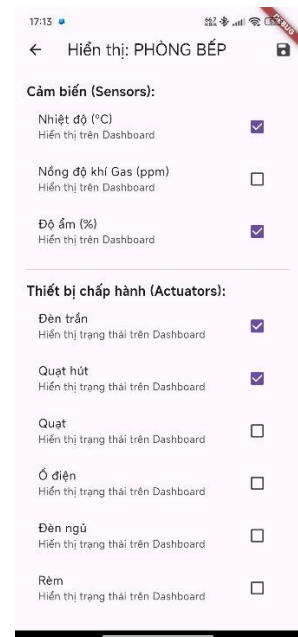
B - Thiết kế chi tiết UI



Hình 5. 1 Màn hình đăng nhập



Hình 5. 2 Màn hình Dashboard



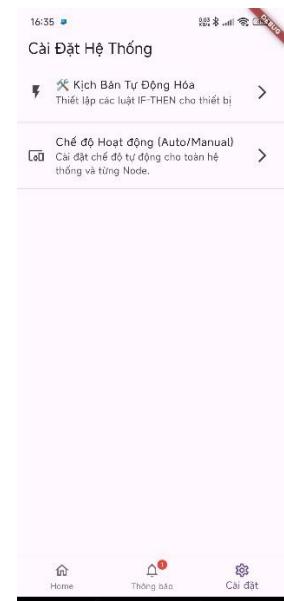
Hình 5. 3 Màn hình tùy chọn hiển thị



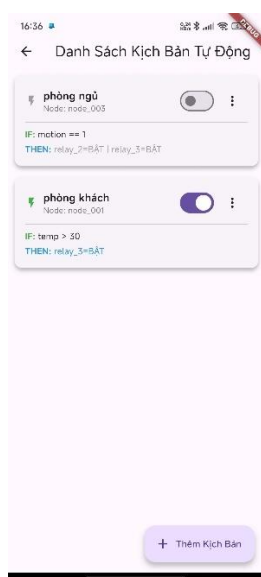
Hình 5. 4 Màn hình chi tiết Node



Hình 5. 5 Màn hình Lịch sử Thông báo



Hình 5. 6 Màn hình cài đặt Hệ thống



Hình 5. 7 Màn hình danh sách kịch bản tự động



Hình 5. 8 Màn hình tạo và chỉnh sửa kịch bản



Hình 5. 9 Màn hình chế độ hoạt động